Interview Guide

Associate Lead SDET — LSEG (London Stock Exchange Group)

Udani Weerasinghe

March 16, 2026

Prepared with care for the most radiant SDET London's ever seen. ☀️

Table of Contents

1. Job Description 3
2. Interviewer: Amy Lee 5
3. Interviewer: Andy Purtell 7
4. Personalised Interview Guide 10
5. Question Bank Index 27
6. Testing Fundamentals 29
7. Test Automation & Programming 34
8. Performance Testing 43
9. Infrastructure, Cloud & Networking 47
10. Financial Domain 51
11. DevOps & SDLC 56
12. Defect Management 60
13. Leadership & Soft Skills 63
14. Scenario-Based Questions 67
15. Personal & Career 72
16. AI & Modern QA 77
17. Security Testing 82
18. General SDET Questions 88
19. AWS Cloud Guide 112
20. DevOps Questions 123

Job Description

Job ID: R0098555 **Company:** LSEG (London Stock Exchange Group)

Job Description

SDET Engineers are involved in requirements analysis, test planning, writing test cases/scripts, test case automation, and execution of all testing (which may include application integration, system, system/network integration, performance testing and acceptance testing) required to ensure that the software is of the highest quality. After the project is released to production, these teammates are also involved to providing support to our internal customer, the technical operation group as well as service management. Besides, this role also plays a meaningful part of internal process audit and improvement to build efficiencies.

Scope and Impact

- Typically work in a single QA team, as a member of a DevOps team.
- Work on projects/changes, typically requires a general understanding of coordinated system and/or of at least one development environment.
- Accountable for the quality of own work, delivered within agreed procedures and timeframes and passionate about internal and/or external customer needs.
- Work independently within defined parameters, taking ownership of problems within own area of knowledge.
- Decisions taken are made within guidelines.
- Make recommendations for process improvements to immediate manager.
- Some roles are responsible for limited co-ordination of the work of others.
- Work under general supervision, carry out procedures to validate that all information systems products and services meet minimum company standards and end-user requirements.
- Develop scripts and test cases, tests modules of larger software products or smaller stand-alone software products to ensure accurate operation and freedom from defects. Document all problems and works to resolve them. Report progress on problem resolution to management.
- Implement pre-determined improvements to current procedures and reports on success of changes. Perform workflow analysis and recommends quality improvements.

Required Education/Experience

- Bachelor’s degree or related experience (or above) in Computer Science, Software Engineering, Electrical Engineering or related subject area.
- Minimum of 3 years testing or other development experience.
- Shown expertise with software testing methodologies and tools.
- Excellent English language skill.
- Proficiency in Windows server and/or Unix/Linux.
- Familiarity with Cloud technologies such as AWS/Azure.
- Experience of formal software development lifecycle and good knowledge on process such as CMM/CMMI.
- Good awareness of customer service and be an active teammate!
- In-depth analysis capability.
- Test automation and Programming languages skills, proficient in one or two of coding languages i.e., Shell/Python/Perl/Robot and/or C/C#/Java/VB, at least 2 years of experience in automation, 2-5 years programming experience.
- Some experience on Networking, TCP/IP, financial markets, or project management is a nice to have.

About LSEG

LSEG is a leading global financial markets infrastructure and data provider. Our purpose is driving financial stability, empowering economies and enabling customers to create sustainable growth.

Our purpose is the foundation on which our culture is built. Our values of Integrity, Partnership, Excellence and Change underpin our purpose and set the standard for everything we do, every day. They go to the heart of who we are and guide our decision making and everyday actions.

Working with us means that you will be part of a dynamic organisation of 25,000 people across 65 countries. However, we will value your individuality and enable you to bring your true self to work so you can help enrich our diverse workforce. You will be part of a collaborative and creative culture where we encourage new ideas and are committed to sustainability across our global business. You will experience the critical role we have in helping to re-engineer the financial ecosystem to support and drive sustainable economic growth. Together, we are aiming to achieve this growth by accelerating the just transition to net zero, enabling growth of the green economy and creating inclusive economic opportunity.

Benefits

LSEG offers a range of tailored benefits and support, including healthcare, retirement planning, paid volunteering days and wellbeing initiatives.

Equal Opportunities

We are proud to be an equal opportunities employer. This means that we do not discriminate on the basis of anyone's race, religion, colour, national origin, gender, sexual orientation, gender identity, gender expression, age, marital status, veteran status, pregnancy or disability, or any other basis protected under applicable law. Conforming with applicable law, we can reasonably accommodate applicants' and employees' religious practices and beliefs, as well as mental health or physical disability needs.

Interviewer: Amy Lee

Role: Director, Quality Engineering **Company:** LSEG (London Stock Exchange Group) **Location:** St. Louis, Missouri, United States **LinkedIn:** linkedin.com/in/amy-lee-60b7317

Current Position

Amy Lee serves as **Director of Quality Engineering** at LSEG, with a focus on AI-driven test automation. Her LinkedIn headline references AI-driven test strategies, suggesting she is actively involved in modernising QA practices within LSEG — likely exploring generative AI for test generation, intelligent test selection, or AI-augmented quality processes.

She has also been noted as holding a **Principal Quality Engineer** title at LSEG, indicating a possible recent title transition or dual responsibility across strategic (Director) and hands-on technical (Principal Engineer) dimensions.

Career Trajectory

Amy's career follows the organisational lineage of **Thomson Reuters** → **Refinitiv** → **LSEG**, a path shared by many long-tenured employees in LSEG's data and analytics division:

Period	Organisation	Role
Earlier career	Thomson Reuters	Software Quality Assurance Engineer / Systems roles
Mid career	Thomson Reuters (Finance & Risk)	Manager — Global QA team for high-speed data delivery
Post-2018	Refinitiv (spun off from Thomson Reuters)	QA Director
Post-2021	LSEG (acquired Refinitiv)	Director, Quality Engineering

Key Career Themes

- **Long tenure in financial data infrastructure** — Amy has spent a significant portion of her career within the same organisational lineage, giving her deep institutional knowledge of LSEG's data delivery systems, products, and quality challenges.
- **Managed global QA teams** — She led a global team of software quality testers for Thomson Reuters' high-speed data delivery infrastructure in the Finance & Risk division. This means she has experience managing distributed teams across time zones.
- **Systems and infrastructure background** — Before moving into QA leadership, Amy's specialties included **Unix Administration, Server Management, and Web Development**. This is a notable differentiator — she understands the infrastructure layer, not just the application layer.

Technical Background

Based on available information, Amy's technical skillset spans:

- **Unix/Linux system administration** — deep OS-level knowledge
- **Windows and Unix server management** — installation, configuration, maintenance
- **Web development and hosting** — earlier career experience
- **High-speed data delivery systems** — core domain at Thomson Reuters/Refinitiv
- **AI-driven test automation** — current focus area at LSEG

Her infrastructure background is significant: it means she likely values candidates who understand systems beyond just application-level testing — networking, server configuration, deployment pipelines, and performance at the infrastructure level.

Education

- **Columbia College** — specific degree details not publicly available, but her background is in engineering and computer science.

What This Means for the Interview

Likely Interview Focus Areas

1. **Infrastructure and systems knowledge** — Given her Unix/server background, Amy may probe the candidate's understanding of operating systems, server environments, and how testing interacts with infrastructure. The JD's requirement for "Windows server and/or Unix/Linux proficiency" likely reflects her team's actual needs.
2. **AI in QA** — Her LinkedIn headline emphasises AI-driven test automation. She may ask about the candidate's awareness of or experience with AI/ML in testing — intelligent test selection, AI-generated test cases, predictive analytics for quality.
3. **Global team dynamics** — Having managed global teams, she understands the challenges of distributed QA. She may assess the candidate's experience working across time zones, with offshore/onshore models, or with culturally diverse teams.
4. **Real-time data systems** — Her career has centred on high-speed financial data delivery. She may ask about testing approaches for latency-sensitive, high-throughput systems — performance testing, network-level testing, data integrity validation.
5. **Leadership and strategic thinking** — As a Director, she's likely assessing whether the candidate can think beyond test execution — test strategy, quality metrics, process improvement, stakeholder communication.

Communication Style Considerations

- Amy has deep technical roots (Unix admin, server management) combined with leadership experience. She likely respects candidates who can speak to both technical depth and strategic breadth.
- Her long tenure within the Thomson Reuters → LSEG lineage suggests she values stability, institutional knowledge, and long-term thinking.
- The AI-driven focus suggests she is forward-looking and may be interested in candidates who show curiosity about emerging QA technologies.

Sources

- Amy Lee — LinkedIn
- Amy Lee — Wiza Profile
- LSEG St. Louis Office

Interviewer: Andy Purtell

Role: Manager, Quality Engineering (QE Manager) **Company:** LSEG (London Stock Exchange Group) **Location:** Missouri, United States **LinkedIn:** linkedin.com/in/andypurtell

Current Position

Andy Purtell is a **Quality Engineering Manager** at LSEG, operating within the lending and brokerage sector of the business. As a QE Manager, he is the likely **direct hiring manager** for this Associate Lead SDET role — meaning his assessment will carry significant weight in the hiring decision. He would be evaluating the candidate as a potential direct report or close team member.

Career Trajectory

Like Amy Lee, Andy's career follows the **Thomson Reuters** → **Refinitiv** → **LSEG** organisational lineage:

Period	Organisation	Role
Earlier career	Thomson Reuters	Elektron Integration Support QA Engineer
Mid career	Thomson Reuters / Refinitiv	Senior Software Quality Assurance Engineer
Later	Refinitiv	Technical Leadership roles in QA
Current	LSEG	Manager, Quality Engineering

Key Career Themes

- **Grew from hands-on QA into management** — Andy's progression from QA Engineer → Senior QA Engineer → Technical Lead → QE Manager means he has walked the same career path the candidate is on. He understands the day-to-day realities of test automation work because he has done it himself.
- **Deep Elektron/real-time data expertise** — His role as **Elektron Integration Support QA Engineer** at Thomson Reuters placed him at the heart of the company's next-generation real-time financial data delivery network. Elektron (now LSEG Real-Time) is the backbone infrastructure for distributing market data to financial institutions globally.
- **Multi-environment testing at scale** — Colleagues have noted his work maintaining technical documentation and test frameworks across **three environments (Windows, Linux, Cloud)** scaling to **500+ individual products**. This indicates experience with complex, large-scale test infrastructure where consistency across platforms is a major challenge.
- **Mentoring and team development** — Professional references highlight his mentoring abilities and contributions to developing other QA engineers, suggesting he values and actively invests in growing his team members.

Technical Background

Based on available information, Andy's technical experience includes:

- **Real-time financial data systems** — Elektron platform, market data delivery, low-latency systems
- **Multi-platform testing** — Windows, Linux, and Cloud environments
- **Large-scale test frameworks** — managing test infrastructure across 500+ products
- **Integration and system testing** — his Elektron role focused on integration support

- **Technical documentation** — consistently cited for documentation quality across environments
- **Financial data protocols and standards** — deep domain knowledge of how financial data moves through systems

What This Means for the Interview

Likely Interview Focus Areas

1. **Hands-on technical depth** — Andy came up through the QA engineering ranks. He will likely ask practical, technical questions about test automation: framework design, debugging flaky tests, handling test data, parallel execution strategies, CI/CD integration. He can spot superficial answers because he has done this work himself.
2. **Multi-platform testing** — Given his experience across Windows, Linux, and Cloud environments, he may ask about cross-platform testing challenges, environment-specific issues, and how the candidate handles testing across different operating systems. This directly connects to the JD's "Windows server and/or Unix/Linux" requirement.
3. **Scale and complexity** — Managing test frameworks for 500+ products means he understands the challenges of testing at scale. He may ask about how the candidate has handled growing test suites, maintaining test stability, managing test infrastructure, or dealing with combinatorial complexity.
4. **Integration testing** — His Elektron Integration Support background means he likely values strong integration and system testing skills. He may ask about testing approaches for distributed systems, API integrations, message queues, or data pipelines.
5. **Financial data domain** — Andy has spent his career in financial data delivery. He may assess whether the candidate understands (or can quickly learn) financial market concepts — market data feeds, trading systems, order management, or regulatory requirements.
6. **Team fit and growth potential** — As the likely direct manager, he'll be evaluating whether the candidate would be a good fit for his team culture, whether they can work independently (as the JD states), and whether they show potential for growth into more senior roles.

Communication Style Considerations

- Andy's progression from hands-on engineer to manager suggests he values practical, grounded answers over theoretical or buzzword-heavy responses. Demonstrating real experience with specific examples will resonate more than abstract claims.
- His reputation as a mentor suggests he is supportive of learning and growth. The candidate should feel comfortable being honest about areas where they are still developing, as long as they show willingness and ability to learn.
- His documentation focus suggests he values thoroughness and attention to detail. Candidates who can speak to how they document test plans, maintain test frameworks, or communicate quality status will likely impress.
- He may ask scenario-based questions drawn from his own experience — e.g., "How would you approach testing a real-time data feed across multiple platforms?"

Relationship to Amy Lee

Andy reports within Amy Lee's Quality Engineering organisation. The interview structure (Director + Manager) suggests:

- **Amy** will likely focus on **strategic fit, leadership potential, and cultural alignment** — does this person think like a quality leader?
- **Andy** will likely focus on **technical competence, practical skills, and team fit** — can this person do the work and grow within my team?

The candidate should be prepared to adjust their communication style between the two interviews: more strategic and visionary with Amy, more practical and detailed with Andy.

Sources

- Andy Purtell — LinkedIn
- Andy Purtell — ZoomInfo
- LSEG Careers

Personalised Interview Guide

Target Role: Associate Lead, SDET — LSEG (R0098555) **Interviewers:** Amy Lee (Director, Quality Engineering) & Andy Purtell (QE Manager) **Date Prepared:** 2026-03-15

Table of Contents

1. Interview Panel Dynamics
2. The Career Gap — Your Biggest Challenge
3. Your Narrative Arc — The Story to Tell
4. Technical Question Areas
5. Behavioural and Situational Questions
6. Financial Domain — Bridging the Gap
7. Infrastructure and Systems Questions
8. Leadership and Team Questions
9. AI and Modern QA Practices
10. Questions About LSEG Specifically
11. Handling Your Weak Areas
12. Questions You Should Ask Them
13. Practical Tips and Do/Don't Lists
14. Pre-Interview Checklist

1. Interview Panel Dynamics

Understanding the two interviewers and tailoring your approach to each is critical.

Amy Lee — Director, Quality Engineering

Attribute	Detail
Role	Director-level; strategic decision-maker
Background	Unix admin → Server management → QA leadership; Thomson Reuters → Refinitiv → LSEG
Current Focus	AI-driven test automation
What she's evaluating	Strategic thinking, leadership potential, cultural fit, long-term vision
Communication style	She has deep technical roots but now operates at a strategic level. Speak in terms of outcomes, quality strategy, and business impact — not just tools and frameworks

How to approach Amy: - Lead with *why*, not *what*. Instead of "I used Selenium," say "I chose Selenium for this context because… and the outcome was…" - Show you think about quality at the organisational level, not just the test case level. - Connect your MSc and MBA to how you think about quality as a business enabler. - Show genuine curiosity about AI in QA — this is her current passion project. Ask about it. - Demonstrate you can work across global teams (she's managed global QA teams for years).

Andy Purtell — QE Manager

Attribute	Detail
Role	Likely your direct manager if hired
Background	QA Engineer → Senior QA → Tech Lead → QE Manager; deep Elektron/real-time data experience
Current Focus	Multi-platform testing at scale (Windows, Linux, Cloud), 500+ products
What he's evaluating	Technical depth, hands-on capability, team fit, growth potential
Communication style	Grew up through the ranks; values practical, specific, grounded answers. Can detect superficial responses

How to approach Andy: - Lead with *specifics*. He wants to hear about real problems you solved, real frameworks you built, real bugs you caught. - Be honest about what you don't know — his reputation as a mentor means he values coachability over perfection. - Show you can work independently (the JD emphasises this) while communicating effectively. - Demonstrate attention to detail, especially around documentation and test maintenance. - Be ready for scenario-based questions drawn from his real-time data testing experience.

The Dynamic Between Them

Amy and Andy likely have a coordinated evaluation approach:

- **Amy screens for**: Will this person grow into a quality leader? Do they think strategically? Will they fit our culture?
- **Andy screens for**: Can this person actually do the work? Will they fit my team? Can I develop them?

You need to pass **both** filters. A candidate who impresses Amy with vision but can't demonstrate practical skills to Andy won't get through — and vice versa.

---

2. The Career Gap — Your Biggest Challenge

This will almost certainly come up. Both interviewers will notice that your last QA role ended in October 2023 — approximately 2.5 years ago. How you handle this question could make or break the interview.

What They're Really Asking

When they ask about the gap, they're actually evaluating three things:

1. **Are your skills current?** Test automation has evolved significantly since 2023. Do you still know what you're doing?
2. **Are you committed to QA as a career?** Did you leave the field intentionally? Will you leave again?
3. **Can you be productive quickly?** Or will there be a long ramp-up period?

The Narrative to Prepare

Frame the gap as a **deliberate investment**, not an absence:

> *"After 3.5 years of intensive QA work at Sysco LABS, I made a deliberate decision to invest in my education — completing my MSc in Cloud Native Computing with First Class Honours at TUS in Ireland. This was a strategic choice: I saw cloud-native systems becoming the standard for enterprise software, and I wanted to understand that architecture deeply, not just test the surface. During my time in Ireland, I also stayed connected to the QA field by [mention anything: personal projects, online courses, ISTQB Advanced certification study, reading, community involvement]. My ISTQB Advanced Level TTA certification in April 2025 demonstrates that I've kept my testing knowledge not just current, but deepened it."*

Critical: Prepare Specific Evidence of Continued Learning

You **must** have concrete examples of what you did during 2023-2026 to stay technically sharp. If you don't have them, create them before the interview:

- **ISTQB CTAL-TTA (April 2025)** — This is your strongest evidence. Emphasise the study process, what you learned, and how the advanced syllabus covers topics beyond your work experience.
- **MSc coursework** — What testing-relevant topics did your Cloud Native Computing MSc cover? Docker? Kubernetes? Microservices testing? CI/CD pipelines? Even if the degree was completed in 2024, the knowledge is current.
- **Personal projects** — Even a small automation project on GitHub would help enormously. If you don't have one, consider building a small Selenium/Playwright + REST Assured + Jenkins pipeline project before the interview.
- **Online courses or certifications** — Any Udemy, Coursera, or platform courses in relevant areas (performance testing, AI in testing, financial markets basics).
- **Community involvement** — QA meetups, conferences attended, articles read, online forums participated in.

What NOT to Say

- Don't be defensive or apologetic about the gap.
- Don't say "I was looking for the right opportunity" — this sounds passive.
- Don't pretend the Student Experience Assistant role was technical.
- Don't inflate the timeline — if asked directly, be honest about 3.5 years of QA experience.

---

3. Your Narrative Arc — The Story to Tell

Every interview is a story. Yours should follow this arc:

Opening (Who You Are)

> *"I'm a quality engineering professional who combines hands-on test automation expertise with strategic thinking. I've spent 3.5 years building and leading test automation across complex enterprise platforms — SAP, Salesforce, and E-Commerce systems — and I've recently deepened my knowledge with a First Class MSc in Cloud Native Computing and an ISTQB Advanced Level certification."*

Middle (What Makes You Different)

> *"What sets me apart is the combination of technical depth and business perspective. My MBA helps me understand quality from a business impact standpoint — not just 'did the test pass' but 'does this quality strategy serve the business goals.' My MSc gives me the architectural understanding of cloud-native systems that modern SDET work demands. And my hands-on experience gives me the practical skills to actually build and maintain automation at scale."*

Bridge to LSEG (Why This Role)

> *"LSEG represents exactly the kind of challenge I'm looking for — a global financial infrastructure company where quality isn't just nice to have, it's mission-critical. The scale of your systems, the real-time data delivery, the regulatory requirements — these create testing challenges that genuinely excite me. I'm particularly interested in how your quality engineering team is approaching AI-driven test automation and how I can contribute to that evolution."*

---

4. Technical Question Areas

4.1 Test Automation Framework Design

Likely asked by: Andy (primary), Amy (at a higher level)

Expected questions:

- *"Walk me through how you would design a test automation framework from scratch."*
- *"How did you architect your automation framework at Sysco LABS?"*
- *"What's the difference between a test automation framework and a test automation script?"*
- *"How do you decide what to automate and what to test manually?"*

How to answer:

Use the **Page Object Model (POM)** as your foundation, since this is what most enterprise Selenium frameworks use:

```
Framework Architecture:
├── Test Layer (TestNG/JUnit test classes)
├── Business Logic Layer (step definitions, workflows)
├── Page Object Layer (page classes, element locators)
├── Utility Layer (config, logging, reporting, data providers)
├── Data Layer (test data, environment configs)
└── CI/CD Integration (Jenkins pipeline, reports)
```

Speak to specific design decisions you made at Sysco LABS: - Why you chose Selenium + TestNG over alternatives. - How you handled test data management. - How you managed environment configurations (dev, staging, production). - How you handled cross-browser testing (mention BrowserStack). - How you integrated with CI/CD (Jenkins + Git + AWS).

Key point for Andy: He manages test frameworks across 500+ products on 3 platforms. Show that you understand the *maintenance* challenge, not just the initial build. Talk about: - How you kept locators stable when the UI changed. - How you managed framework versioning. - How you reduced flaky tests. - How you onboarded new team members to the framework.

4.2 API Testing and REST Assured

Likely asked by: Andy

Expected questions:

- *"How did you use REST Assured in your projects?"*
- *"How do you validate API responses beyond just status codes?"*
- *"How do you handle API authentication in test automation?"*
- *"What's your approach to testing API integrations between systems?"*

How to answer:

Draw from your Salesforce/SAP integration testing experience: - You tested APIs that connected Salesforce, SAP, and E-Commerce systems. - You validated response payloads, headers, status codes, and response times. - You tested error scenarios — invalid inputs, authentication failures, timeout handling. - You used REST Assured's fluent API for readable, maintainable test code.

Example to prepare:

> *"In the Vendor Invoice Management project, we had APIs that connected SAP to Salesforce for invoice processing. I automated the entire API flow — creating invoices in SAP, verifying they appeared correctly in Salesforce via the integration API, and validating the data transformation at each step. This caught several data mapping issues before they reached production, including a currency conversion bug that would have affected invoice totals."*

4.3 CI/CD Integration

Likely asked by: Andy (implementation details), Amy (strategic value)

Expected questions:

- *"How did you integrate your test automation into CI/CD pipelines?"*
- *"What quality gates did you define in your pipeline?"*
- *"How did you handle test failures in the pipeline?"*
- *"What's your approach to determining which tests run at each pipeline stage?"*

How to answer:

Describe a tiered testing approach in Jenkins: 1. **Commit stage:** Unit tests + critical smoke tests (fast, blocking). 2. **Build stage:** API tests + core functional tests (medium, blocking). 3. **Deploy to staging:** Full regression suite (longer, non-blocking but alerts on failure). 4. **Pre-production:** Smoke tests on production-like environment.

Mention specific Jenkins features you used: - Parameterised builds for environment selection. - Pipeline-as-code (Jenkinsfile). - Integration with Git for trigger-on-push. - SonarQube quality gates for code quality. - Test report publishing and trend analysis.

4.4 Selenium-Specific Deep Dive

Likely asked by: Andy

Expected questions:

- *"How do you handle dynamic elements in Selenium?"*
- *"What's your approach to waits? Explain implicit vs explicit vs fluent waits."*
- *"How do you handle iframes, shadow DOM, or multi-window scenarios?"*
- *"How did you handle Salesforce testing with Selenium? What were the challenges?"*

Salesforce + Selenium is a strong differentiator — Salesforce's Lightning framework is notoriously difficult to automate due to dynamic element IDs, shadow DOM, and iframes. If you have specific strategies you used, prepare them in detail:

- Custom wait conditions for Lightning component loading.
- JavaScript executor for interacting with shadow DOM elements.
- Handling Salesforce's dynamic record IDs in test data.
- Managing Salesforce sandbox environments for testing.

4.5 Programming and Coding

Likely asked by: Andy

Expected questions:

- *"Write a function to [simple algorithm or data structure task]."*
- *"How would you parse a JSON response and validate specific fields?"*
- *"Write a method that retries a flaky API call up to 3 times with exponential backoff."*
- *"Review this test code — what would you improve?"*

Preparation:

- Be comfortable writing Java code on a whiteboard or shared screen.
- Practice common patterns: reading/writing files, parsing JSON/XML, string manipulation, collections operations.
- Know Java OOP concepts well — inheritance, polymorphism, interfaces, abstraction — and how they apply to framework design (e.g., base page class, interface for page actions).
- Be ready to write a simple TestNG test with assertions.
- Know the difference between `assertEquals`, `assertTrue`, `assertThat` and when to use each.

4.6 Test Case Design and Strategy

Likely asked by: Both (Amy at strategic level, Andy at practical level)

Expected questions:

- *"How do you decide test coverage priorities for a new feature?"*
- *"Explain boundary value analysis and equivalence partitioning with an example."*
- *"How do you approach risk-based testing?"*
- *"What's your approach to regression test selection — do you run everything every time?"*

Leverage your ISTQB CTAL-TTA here. The Advanced Level syllabus covers: - Condition testing, decision testing, modified condition/decision coverage (MC/DC). - Statement coverage vs branch coverage vs path coverage. - White-box test techniques. - Test design based on risk analysis.

Prepare a concrete example of risk-based testing from your Sysco LABS experience:

> *"When we had limited time before a Salesforce release, I used risk-based test prioritisation. I mapped each test case to business risk (impact × likelihood of failure) and technical risk (code change proximity × complexity). High-risk cases ran first and were automated; medium-risk cases were included in the regression suite; low-risk cases were deferred to the next cycle. This approach helped us maintain quality while meeting aggressive release timelines."*

4.7 Performance Testing

Likely asked by: Andy (specific tools/experience), Amy (strategic understanding)

This is a gap area for you. The JD mentions performance testing, and LSEG's real-time data delivery systems make performance critical.

Expected questions:

- *"Do you have experience with performance testing? What tools have you used?"*
- *"How would you approach performance testing for a real-time data system?"*
- *"What's the difference between load testing, stress testing, and endurance testing?"*

Honest approach:

If you haven't done significant performance testing, be honest but show awareness:

> *"My direct experience with dedicated performance testing tools is limited — my work focused primarily on functional and integration test automation. However, I understand the fundamentals: load testing to validate capacity, stress testing to find breaking points, endurance testing for memory leaks and degradation over time. I incorporated basic performance validation in my API tests using REST Assured — response time assertions and throughput checks — but I haven't done large-scale load testing with tools like JMeter or Gatling. It's an area I'm actively working to develop, especially given how critical performance is in financial markets infrastructure."*

Preparation to reduce this gap before the interview:

- Install JMeter and run a basic load test against a public API. Familiarise yourself with thread groups, samplers, listeners, and assertions.
- Understand key performance metrics: response time (mean, median, p95, p99), throughput (requests/sec), error rate, concurrent users.
- Learn the basic concepts of how real-time market data systems work and why latency matters (a 1ms delay in trading systems can mean millions in losses).

4.8 Database and SQL Testing

Likely asked by: Andy

Expected questions:

- *"How do you validate data in the database as part of your testing?"*
- *"How do you set up and manage test data?"*

- *"Write a SQL query to find duplicate records in a table."*
- *"How did you validate data integrity between SAP and Salesforce?"*

How to answer:

Draw from your SAP/Salesforce/E-Commerce integration work: - You likely validated data transformations between systems — an invoice created in SAP should appear with correct values in Salesforce. - You used SQL to verify data in staging databases (PostgreSQL, Oracle). - You managed test data creation and cleanup to ensure test independence.

Prepare basic SQL patterns:

```sql
-- Find duplicates
SELECT column, COUNT(*) FROM table GROUP BY column HAVING COUNT(*) > 1;

-- Validate data between tables (join-based validation)
SELECT a.id, a.amount, b.amount
FROM sap_invoices a
JOIN sf_invoices b ON a.invoice_id = b.external_id
WHERE a.amount != b.amount;

-- Check for null or missing data
SELECT * FROM orders WHERE customer_id IS NULL OR order_date IS NULL;
```

---

5. Behavioural and Situational Questions

5.1 The STAR Method

For all behavioural questions, use the **STAR** framework:

- **S**ituation — set the context (brief).
- **T**ask — what was your specific responsibility.
- **A**ction — what you specifically did (this is the longest part).
- **R**esult — the measurable outcome.

5.2 Common Behavioural Questions and Suggested Answers

"Tell me about a time you found a critical bug late in the release cycle."

Your angle: Use the E-Commerce project where you *"identified 20 critical defects prior to production release."*

> ***S:*** *"In the Hybris E-Commerce project, we were three days from a major release when I was running the final regression cycle."* ***T:*** *"I needed to validate the entire order flow end-to-end, including payment processing and inventory updates."* ***A:*** *"During automation execution, I noticed that one specific combination of discount code + promotional pricing was calculating the final amount incorrectly — a race condition between the discount engine and the pricing service. I immediately raised it with the dev team, provided the exact reproduction steps from my automated test logs, and worked with the developer to identify that the issue was in the async event handling between the two services."* ***R:*** *"The fix was deployed within 24 hours, and we shipped on time. Without the automated test catching this, customers would have been charged incorrect amounts — a direct revenue and trust impact."*

"Tell me about a time you had to push back on a deadline or scope."

Your angle: Frame around risk-based testing decisions.

> ***S:*** *"During a Salesforce Service Cloud release, the product owner wanted to add three new features to an already-packed sprint."* ***T:*** *"As the QA lead for that phase, I needed to assess whether we could maintain quality with the expanded scope."* ***A:*** *"I did a quick risk assessment and presented the product owner with three options: (1) extend the timeline by one sprint, (2) reduce scope to the two highest-priority features, or (3) proceed with all three but accept that we'd only have automated coverage for the critical paths and manual*

> *testing for edge cases. I quantified the risk of each option in terms of defect escape probability."* ***R:*** *"The product owner chose option 2, and we delivered two solid, well-tested features rather than three fragile ones. The third feature was delivered the following sprint with full test coverage."*

"Tell me about a time you mentored someone."

Your angle: Use your mentoring at Sysco LABS and/or your Teaching Assistant role.

> ***S:*** *"At Sysco LABS, I had a junior tester join my team who was strong in manual testing but had no automation experience."* ***T:*** *"I was responsible for getting them up to speed on our Selenium + TestNG framework within their first month."* ***A:*** *"Rather than just pointing them to documentation, I created a structured 4-week onboarding plan: Week 1 was Java fundamentals and our codebase walkthrough. Week 2 was writing their first page objects. Week 3 was creating their first end-to-end test. Week 4 was code review and refactoring. I paired with them daily for the first two weeks, gradually reducing to weekly check-ins. I also had them present their work to the team at the end of each week."* ***R:*** *"By month two, they were independently writing automation scripts and contributing to code reviews. Within six months, they were one of our most productive automation engineers. The experience also helped me develop a reusable onboarding template that we used for future hires."*

"Tell me about a time you dealt with a conflict in the team."

> ***S:*** *"A developer on our team consistently pushed back on bugs I raised, arguing they were 'by design' or 'edge cases not worth fixing.'"* ***T:*** *"I needed to maintain a productive working relationship while ensuring quality standards were met."* ***A:*** *"Instead of escalating immediately, I changed my approach. For each defect, I started including business impact analysis — not just 'this field shows incorrect data' but 'this affects invoice reconciliation for 15% of our UK customers, which could result in payment disputes.' I also started inviting the developer to participate in test case reviews before I tested, so they could see the scenarios I'd be covering and flag concerns early."* ***R:*** *"The relationship improved significantly. The developer started proactively asking about test coverage for their features, and our defect escape rate dropped by about 30% in the following quarter."*

"Why are you looking to move? Why LSEG?"

Critical question. Your answer must address the career gap positively:

> *"After my intensive period at Sysco LABS, I invested in my education — completing my MSc in Cloud Native Computing and then my ISTQB Advanced certification. Now I'm ready to bring that deeper knowledge back to a challenging technical environment. LSEG represents that challenge: you're operating at the intersection of financial markets, real-time data, and global infrastructure — this is where quality engineering has the highest stakes and the greatest impact. I'm particularly drawn to LSEG's investment in AI-driven quality engineering, and I believe my combination of hands-on automation experience, cloud-native architecture knowledge, and business understanding from my MBA positions me well to contribute meaningfully."*

"Where do you see yourself in 3-5 years?"

> *"I see myself growing into a senior quality engineering leader — someone who shapes QA strategy for a product area, not just individual test suites. In 3 years, I'd like to be leading a QA stream, driving test automation architecture decisions, and mentoring the next generation of SDETs. In 5 years, I'd like to be influencing quality practices at an organisational level — perhaps driving initiatives like AI-augmented testing or quality metrics that connect directly to business outcomes. LSEG's scale and the complexity of financial markets infrastructure gives me the right environment to develop those capabilities."*

---

6. Financial Domain — Bridging the Gap

This is a known weakness. Both interviewers have spent their entire careers in financial data. Here's how to bridge this.

What You Must Know Before the Interview

About LSEG

- **What LSEG is:** London Stock Exchange Group — a global financial markets infrastructure and data provider. Not just the London Stock Exchange; LSEG also owns Refinitiv (data analytics), FTSE Russell (indexes), and LCH (clearing house).
- **Scale:** 25,000 employees, 65 countries, processes billions of transactions.
- **Core products:** Real-time market data delivery (Elektron/LSEG Real-Time), trading platforms, index calculation (FTSE 100, Russell 2000), post-trade clearing and settlement.
- **Values:** Integrity, Partnership, Excellence, Change.

About Elektron / LSEG Real-Time

This is particularly important because both interviewers have worked on this:

- **What it is:** LSEG Real-Time (formerly Elektron) is the backbone infrastructure for delivering real-time market data to financial institutions globally.
- **What it does:** Distributes real-time prices, trades, news, and analytics for financial instruments across the world.
- **Why quality matters:** A data error or latency spike in market data can cause incorrect trading decisions worth millions. Quality in this context is not just about "no bugs" — it's about data integrity, latency, throughput, and availability at near-100% uptime.
- **Scale:** Handles data for millions of financial instruments across global markets.

Financial Markets Basics

- **Market data:** Real-time prices of stocks, bonds, currencies, commodities. This is the core of what LSEG delivers.
- **Latency:** The time it takes for data to travel from exchange to consumer. In financial markets, microseconds matter.
- **FIX protocol:** Financial Information eXchange — the standard messaging protocol for trading. Know it exists even if you haven't used it.
- **Regulatory compliance:** Financial systems must comply with regulations (MiFID II in Europe, SEC rules in the US). Testing must account for regulatory requirements.
- **Market hours and global markets:** Different exchanges operate in different time zones. Systems must handle opening, closing, and continuous trading phases.

How to Leverage What You Have

Your Chartered Accountancy background is an untapped asset here. Most SDET candidates have zero financial knowledge. You have:

- Business Level completed in Chartered Accountancy — you understand financial statements, accounting principles, and business processes.
- This gives you a foundation for understanding financial data, reporting, and compliance that pure engineers lack.

Frame it like this:

> *"While I haven't worked directly in financial markets, I do have a foundation in financial systems through my Chartered Accountancy studies — I completed the Business Level at CA Sri Lanka. This gives me a working understanding of financial data, reporting, and compliance requirements. Combined with my experience testing enterprise integration systems (SAP → Salesforce), I'm familiar with the complexities of financial data flows, data transformation, and the importance of data integrity. I'm eager to deepen this into financial markets specifically."*

Questions They May Ask About Financial Domain

- *"What do you know about LSEG?"* — Prepare a 60-second summary (see above).

- *"Have you worked with financial systems before?"* — Bridge with CA background and SAP experience.
- *"How would testing in a financial markets context differ from your previous experience?"* — Emphasise: higher stakes, stricter data integrity requirements, latency sensitivity, regulatory compliance, 24/7 availability expectations.
- *"What do you know about real-time data delivery?"* — Show awareness of Elektron/LSEG Real-Time and the importance of low-latency, high-throughput data distribution.

---

7. Infrastructure and Systems Questions

Both interviewers come from infrastructure backgrounds. Amy was a Unix admin. Andy tested across Windows, Linux, and Cloud. The JD requires "proficiency in Windows server and/or Unix/Linux."

Expected Questions

- *"Are you comfortable working in a Linux/Unix environment?"*
- *"How do you set up and maintain test environments?"*
- *"How would you troubleshoot a test failure that only occurs on Linux but not on Windows?"*
- *"How do you manage test environments in the cloud?"*

How to Answer

Leverage your MSc in Cloud Native Computing. Even if your Sysco LABS work was primarily application-level, your MSc almost certainly covered:

- Linux fundamentals (most cloud-native systems run on Linux).
- Docker containers and Kubernetes orchestration.
- Cloud infrastructure (AWS/Azure — both on your CV).
- Networking basics (how services communicate in cloud-native architectures).

Frame it like this:

> *"At Sysco LABS, my test automation ran on cloud-hosted environments in AWS, and I managed environment setup and test data configuration. My MSc in Cloud Native Computing significantly deepened my infrastructure knowledge — I worked with Docker containers, Kubernetes orchestration, Linux-based systems, and cloud-native deployment patterns. While I haven't done full-time Unix system administration, I'm comfortable working in Linux environments for test execution, log analysis, and basic scripting."*

What to Brush Up On Before the Interview

- **Basic Linux commands:** `grep`, `awk`, `sed`, `tail`, `ps`, `top`, `df`, `chmod`, `ssh`, `scp`. Be able to explain what each does.
- **Log analysis:** How to find errors in log files (`grep -i error /var/log/app.log`, `tail -f` for real-time monitoring).
- **Process management:** How to check running processes, kill stuck processes, check resource usage.
- **Basic networking:** `ping`, `curl`, `netstat` / `ss`, `traceroute`. Understanding TCP/IP at a basic level — what happens when you type a URL in a browser (DNS → TCP handshake → HTTP request → response).
- **Docker basics:** `docker ps`, `docker logs`, `docker exec`, Dockerfile structure. If your MSc covered this, refresh it.

---

8. Leadership and Team Questions

The role title says "Associate Lead." Even though the JD describes more of an IC role, both interviewers will assess leadership capability.

Expected Questions

From Amy (strategic leadership):

- *"How do you define a QA strategy for a new project?"*
- *"How do you measure quality? What metrics do you use?"*
- *"How do you influence developers to care about quality?"*
- *"How do you decide what to invest automation effort in vs. what stays manual?"*

Quality Metrics to know and discuss:

Metric	What It Measures	Why It Matters
Defect escape rate	Bugs found in production vs. found in testing	Measures testing effectiveness
Automation coverage	% of test cases automated	Measures automation maturity
Test execution time	How long the regression suite takes	Impacts release velocity
Flaky test rate	% of tests that intermittently fail	Measures automation reliability
Mean time to detect (MTTD)	How quickly defects are found after introduction	Measures shift-left effectiveness
Defect density	Defects per KLOC or per feature	Measures code quality

From Andy (team-level leadership):

- *"How do you onboard a new tester onto an existing automation framework?"*
- *"How do you handle a team member who isn't meeting quality standards in their test code?"*
- *"How do you prioritise work when multiple teams need QA support simultaneously?"*
- *"How do you conduct effective code reviews for test automation?"*

Your strengths here: - You mentored junior testers at Sysco LABS. - You delivered workshops on test automation. - You led QA phases across multiple development teams. - You were Teaching Assistant at Open University. - You were Deputy Tribe Chief at Sysco LABS.

Prepare 2-3 specific stories that demonstrate: 1. A time you taught someone a technical skill and they improved measurably. 2. A time you coordinated QA work across multiple teams or projects. 3. A time you made a process improvement recommendation that was adopted.

---

9. AI and Modern QA Practices

This is particularly important for Amy Lee, whose LinkedIn headline focuses on AI-driven test automation.

Expected Questions

- *"What's your view on AI in test automation?"*
- *"Have you used any AI-assisted testing tools?"*
- *"How do you think generative AI will change the SDET role?"*
- *"What emerging QA practices are you most excited about?"*

How to Answer

Your CV lists "Gen AI driven QA and test automation" as an interest. Be prepared to back this up with specifics:

AI in QA — What to Know:

AI Application	What It Does	Example Tools
AI-generated test cases		Diffblue Cover, CodiumAI

AI Application	What It Does	Example Tools
	Uses AI to generate test scenarios from requirements or code	
Intelligent test selection	Selects which tests to run based on code changes	Launchable, Predictive Test Selection
Visual testing AI	Uses AI to detect visual regressions	Applitools Eyes, Percy
Self-healing locators	Automatically updates element locators when UI changes	Healenium, Testim
AI-assisted code review	Reviews test code for quality and best practices	GitHub Copilot, SonarQube AI
Log analysis AI	Uses AI to identify patterns in test failure logs	Elastic AI, Splunk ITSI

A good answer:

> *"I'm genuinely excited about AI's potential in QA. I see three transformative areas: First, intelligent test selection — using ML to predict which tests are most likely to fail based on code changes, so we run the right tests faster rather than the entire suite every time. Second, AI-assisted test generation — tools like CodiumAI can generate test cases from code, which doesn't replace human test design but accelerates coverage. Third, self-healing test frameworks — where AI automatically adapts element locators when the UI changes, which is one of the biggest maintenance headaches in Selenium automation. I haven't had the opportunity to implement these at scale yet, but I've been following the space closely, and I'd be very keen to contribute to LSEG's AI-driven quality engineering initiatives."*

Then ask Amy about it:

> *"I noticed that your team is investing in AI-driven test automation — could you tell me more about what that looks like at LSEG? I'd love to understand where you are on that journey and how I might contribute."*

---

10. Questions About LSEG Specifically

Expected Questions

- *"Why LSEG?"*
- *"What do you know about our products?"*
- *"How do you think quality engineering at a financial markets company differs from your previous experience?"*

How to Answer "Why LSEG?"

Structure around three points:

1. **Scale and impact:** "LSEG operates critical global financial infrastructure. The quality bar isn't just 'no bugs' — it's 'billions of dollars of transactions depend on this working perfectly.' That level of stakes drives the kind of rigorous quality engineering I want to practice."
2. **Technical challenge:** "Testing real-time data delivery systems at global scale, across multiple platforms and regulatory environments, is a fundamentally different challenge from testing web applications. I want to grow into that complexity."
3. **Growth and culture:** "LSEG's investment in AI-driven quality engineering, combined with its global presence and engineering culture, offers the kind of environment where I can both contribute and continue developing as a quality leader."

LSEG Values — Be Ready to Reference Them

- **Integrity** — Connect to quality: "Quality engineering is fundamentally about integrity — ensuring our customers receive accurate, reliable data."
- **Partnership** — Connect to collaboration: "My experience collaborating with product owners, developers, and global QA teams has taught me that quality is a shared responsibility, not a QA team's solo mission."
- **Excellence** — Connect to standards: "I'm not satisfied with 'good enough' — my track record of reducing regression times by 45% and achieving 98% test coverage shows my drive toward excellence."
- **Change** — Connect to innovation: "I embrace change — from championing shift-left testing practices to pursuing my MSc in an emerging field like cloud-native computing."

11. Handling Your Weak Areas

For each known gap, here is the defensive strategy.

Gap 1: No Financial Domain Experience

If asked directly: Don't pretend. Bridge honestly:

> *"I haven't worked directly in financial markets, but I bring two relevant foundations: my Chartered Accountancy studies gave me fluency in financial data and business processes, and my experience testing SAP-Salesforce integrations — which involve complex data transformations, business rules, and data integrity requirements — is directly analogous to financial data pipeline testing. I'm a quick domain learner, and I'm already studying financial markets concepts to prepare for this transition."*

Gap 2: No Performance Testing Tools

If asked directly:

> *"I haven't used dedicated performance testing tools like JMeter or Gatling extensively. My performance validation experience has been at the API level — using REST Assured to assert response times and monitoring API throughput during integration testing. However, I understand the principles well: baseline measurement, load profiles, identifying bottlenecks, and the critical importance of performance in financial systems where latency directly impacts trading outcomes. This is an area I'm actively developing."*

Gap 3: No CMM/CMMI Experience

If asked directly:

> *"My direct experience has been in Agile/Scrum environments. I'm familiar with the CMM/CMMI concepts — the maturity levels, process areas, and the emphasis on process standardisation and continuous improvement. While I haven't worked in a formally CMM-assessed organisation, the principle of process maturity aligns with how I approach quality engineering: documenting processes, measuring effectiveness, and continuously improving. I'd be interested to learn how LSEG applies process maturity frameworks in practice."*

Gap 4: Limited Infrastructure/OS Experience

If asked directly:

> *"My daily work at Sysco LABS was primarily at the application and API testing level. However, my MSc in Cloud Native Computing gave me hands-on experience with Linux, Docker, Kubernetes, and cloud infrastructure. I'm comfortable navigating Linux environments, reading logs, and writing shell scripts. I recognise that LSEG's systems operate at a deeper infrastructure level than my previous work, and I'm prepared to deepen that knowledge — my cloud-native foundation gives me a strong starting point."*

Gap 5: Single Employer Experience

If asked about limited employer diversity:

> *"My 3.5 years at Sysco LABS were concentrated but not narrow. I worked across three distinct platforms — SAP, Salesforce, and Hybris E-Commerce — each with different architectures, testing challenges, and team dynamics. I went through a full progression from individual contributor to senior engineer leading QA across multiple teams. Since then, my MSc in Ireland exposed me to entirely different approaches to software design and cloud-native systems. I'm confident in my ability to adapt to new environments, and I'm excited about the learning curve that comes with joining a financial markets company."*

Gap 6: The Career Gap

See Section 2 for the full strategy. The short version:

> *"I made a strategic decision to invest in my education and certifications. The MSc deepened my technical knowledge; the ISTQB Advanced certification sharpened my testing expertise. I'm returning with stronger foundations than I had when I stepped away."*

---

12. Questions You Should Ask Them

Asking good questions demonstrates preparation, curiosity, and strategic thinking. Tailor questions to each interviewer.

Questions for Amy Lee (Director)

1. **"Your LinkedIn mentions AI-driven test automation — could you tell me more about LSEG's vision for AI in quality engineering? Where are you on that journey?"** *Why this works:* Shows you've researched her. Opens a conversation about her passion topic. Demonstrates forward-thinking mindset.
2. **"How does the quality engineering team at LSEG measure success? What metrics matter most?"** *Why this works:* Strategic question that shows you think beyond test execution.
3. **"What does the career progression path look like for an SDET at LSEG? Where do you see this role leading in 2-3 years?"** *Why this works:* Shows long-term commitment. Also helps you understand the actual scope of the role.
4. **"How does the QE team collaborate with development teams? Is it embedded, centralised, or a hybrid model?"** *Why this works:* Shows you understand different QA org models and care about how quality is integrated into the SDLC.
5. **"What's the biggest quality challenge your team is currently facing?"** *Why this works:* Shows problem-solving orientation. Also gives you valuable intel about the actual work.

Questions for Andy Purtell (QE Manager)

1. **"What does a typical day or sprint look like for an SDET on your team?"** *Why this works:* Practical question that helps you understand the actual work. Shows you care about the day-to-day reality, not just the title.
2. **"What tech stack does the team currently use for test automation? What are you considering adopting?"** *Why this works:* The JD doesn't mention the current stack. This fills a critical information gap and shows practical readiness.
3. **"You've been at LSEG (and its predecessor organisations) for a significant time — what keeps you here? What's kept the work interesting?"** *Why this works:* Personal question that builds rapport. Also gives you genuine insight into the culture.
4. **"How large is the QA team I'd be joining? What's the team structure?"** *Why this works:* Practical information you need. Shows you're thinking about how you fit in.

5. **"What would success look like in the first 90 days for someone in this role?"** *Why this works:* Shows you're already thinking about how to deliver value quickly.
6. **"The role mentions supporting production releases and internal customers — how does that work in practice?"** *Why this works:* Shows you read the JD carefully and are preparing for the full scope of the role.

Questions to Avoid

- Don't ask about salary, benefits, or time off in the first interview.
- Don't ask questions that are easily answered by the JD or LSEG's website.
- Don't ask "What does LSEG do?" — you should already know this.
- Don't ask more than 3-4 questions per interviewer — be respectful of time.

13. Practical Tips and Do/Don't Lists

Before the Interview

Do	Don't
Research LSEG's recent news (quarterly earnings, product launches, acquisitions)	Don't just skim the "About Us" page
Study both interviewers' LinkedIn profiles	Don't mention that you researched them unless it comes up naturally
Prepare 5-6 STAR stories covering: mentoring, conflict, critical bug, process improvement, technical challenge, cross-team collaboration	Don't memorise scripts — prepare frameworks and key points
Set up a clean, quiet interview space	Don't have distracting backgrounds or poor lighting
Test your camera, microphone, and internet connection	Don't scramble with tech issues at interview time
Prepare a 60-second "tell me about yourself" opener	Don't ramble for 5 minutes — be concise
Have a copy of your CV visible for reference	Don't read from your CV — use it as a prompt
Prepare a small coding environment in case of live coding	Don't be caught without an IDE if they ask you to code

During the Interview

Do	Don't
Use specific numbers and examples from your experience	Don't make vague claims ("I improved quality")
Be honest about gaps — then bridge to what you bring	Don't pretend you have experience you lack
Show enthusiasm for learning and the LSEG domain	Don't act like financial markets is just another domain
Listen carefully and answer the actual question asked	Don't give pre-rehearsed answers that don't fit the question
Take a moment to think before answering complex questions	Don't rush into answers — pausing is professional
Mirror the interviewer's communication style (strategic with Amy, practical with Andy)	Don't use the same approach for both interviewers
Reference your ISTQB Advanced certification when relevant	Don't oversell certifications as a substitute for experience
Ask clarifying questions if something is unclear	Don't guess at what they're asking
Show curiosity about AI in QA (especially with Amy)	Don't dismiss or be ignorant about AI's impact on testing
Demonstrate that you understand quality at the business level (MBA angle)	Don't position yourself as "just a tester"

Common Mistakes to Avoid

1. **Over-indexing on tools.** Saying "I know Selenium, Playwright, Appium, REST Assured, TestNG, JUnit, Cucumber, Pytest" is a list, not a skill. Instead, describe *how* you used a specific tool to solve a specific problem.
2. **Being defensive about the gap.** If you're defensive, it signals you think it's a problem too. Be matter-of-fact and forward-looking.
3. **Claiming "over four years" of experience.** If challenged, your credibility suffers. Be precise: "approximately three and a half years of hands-on QA experience, supplemented by my MSc and ISTQB Advanced certification."
4. **Not asking questions.** Having no questions signals disinterest. Prepare at least 3 per interviewer.
5. **Generic answers about quality.** "Quality is important" and "I'm passionate about testing" are empty statements. Ground everything in specifics.
6. **Ignoring the financial domain.** Even though the JD lists it as "nice to have," both interviewers live and breathe financial data. Showing zero interest or preparation for the domain will hurt you.
7. **Treating this as just another QA role.** LSEG is financial markets infrastructure. Show that you understand the gravity and responsibility of ensuring quality in systems that global markets depend on.

---

14. Pre-Interview Checklist

One Week Before

- [ ] Research LSEG: recent quarterly results, press releases, product updates.
- [ ] Study both interviewer LinkedIn profiles — note commonalities and talking points.
- [ ] Review Elektron/LSEG Real-Time basics: what it does, why it matters.
- [ ] Prepare and rehearse 6 STAR stories.
- [ ] Brush up on Java coding basics: write a few small programs.
- [ ] Review SQL fundamentals: JOINs, GROUP BY, HAVING, subqueries.
- [ ] Study basic Linux commands and be able to explain what each does.
- [ ] Prepare your "career gap" narrative and practice it until it sounds natural, not rehearsed.
- [ ] Read about AI in testing: know at least 3 specific applications and 2 tool names.
- [ ] Review ISTQB Advanced Level syllabus topics: test techniques, coverage criteria, white-box testing.
- [ ] Study financial markets basics: what market data is, why latency matters, what FIX protocol is.

One Day Before

- [ ] Confirm interview time, platform (Zoom/Teams/etc.), and any access credentials.
- [ ] Test camera, microphone, and internet connection.
- [ ] Prepare your workspace: clean background, good lighting, glass of water.
- [ ] Print (or have digitally ready) your CV, the JD, and your STAR stories.
- [ ] Review your prepared questions for each interviewer.
- [ ] Get a good night's sleep.

Day of Interview

- [ ] Log in 5 minutes early.
- [ ] Have your notes nearby but out of camera view.
- [ ] Take a deep breath. You have strong credentials. You belong in this interview.
- [ ] Remember: Amy wants to see a future quality leader. Andy wants to see a capable, honest engineer who can grow. Show them both.

Final Thought

You have genuine strengths that many candidates don't: ISTQB Advanced certification, an MSc in a highly relevant field, an MBA that gives you business acumen, quantifiable results from your QA work, and a published research paper. The career gap and single-employer history are real challenges, but they're manageable with honest, forward-looking framing.

The key to this interview is **bridging**: connecting what you have done to what LSEG needs. Every answer should follow the pattern: *"Here's what I've done → Here's how it connects to what you need → Here's how I'll grow into the rest."*

Go in prepared, be honest about gaps, be enthusiastic about the opportunity, and let your genuine strengths speak for themselves.

Question Bank Index

Role: Associate Lead, Software Development Engineer in Test (SDET) **Company:** London Stock Exchange Group (LSEG) **Requisition:** R0098555

Category Index

#	Category	File	Questions
1	Testing Fundamentals & Methodology	01_testing_fundamentals.md	Q1–Q10
2	Test Automation & Programming	02_test_automation_and_programming.md	Q11–Q20, Q88–Q89
3	Performance & Non-Functional Testing	03_performance_testing.md	Q21–Q26
4	Infrastructure, Cloud & Networking	04_infrastructure_cloud_networking.md	Q27–Q33
5	Financial Domain & LSEG Context	05_financial_domain.md	Q34–Q40
6	DevOps & SDLC Process	06_devops_and_sdlc.md	Q41–Q47
7	Defect Management & Production Support	07_defect_management.md	Q48–Q53
8	Leadership & Soft Skills	08_leadership_and_soft_skills.md	Q54–Q61
9	Scenario-Based / Problem-Solving	09_scenario_based.md	Q62–Q70
10	Personal & Career-Specific	10_personal_and_career.md	Q71–Q77
11	AI & Modern QA Practices	11_ai_and_modern_qa.md	Q78–Q82
12	Security Testing	12_security_testing.md	Q83–Q87
13	General SDET Questions	13_general_sdet_questions.md	—
14	AWS & Cloud Guide	14_aws_cloud_guide.md	—
15	DevOps Questions	15_devops_questions.md	—

Total: 89 numbered questions across 12 categories + 3 supplementary guides

General Interview Tips for This Role

1. **Use the STAR method** (Situation, Task, Action, Result) for all behavioral questions.
2. **Research LSEG** — Understand their products, recent news, and strategic direction. Mention specifics.
3. **Prepare code samples** — Be ready to write Python/Java/Shell scripts on a whiteboard or shared screen.
4. **Quantify your achievements** — “Reduced regression time by 70%” is stronger than “Improved regression testing.”
5. **Show domain interest** — Even if you haven’t worked in financial markets, demonstrate that you’ve done your homework.

6. **Ask interviewer-tailored questions** — Prepare different questions for each interviewer. For Amy (Director): ask about AI-driven QA strategy, quality metrics, career progression. For Andy (QE Manager, likely direct manager): ask about day-to-day team work, tech stack, first 90 days. See Q76 for specific prepared questions.
7. **Align with values** — Weave Integrity, Partnership, Excellence, and Change into your answers naturally.
8. **Be honest about gaps** — If you lack experience in an area (e.g., FIX protocol), say so, then explain how you'd ramp up quickly.

Interviewer Profiles

Understanding your two interviewers is critical for tailoring your communication style.

Amy Lee — Director, Quality Engineering

Attribute	Detail
Role	Director-level; strategic decision-maker
Background	Unix admin → Server management → QA leadership; Thomson Reuters → Refinitiv → LSEG
Current Focus	AI-driven test automation
Communication approach	Lead with *why* and outcomes, not just tools. Think strategically — quality at the organisational level.
Key topics to prepare	AI in QA, quality metrics, team scaling, career vision, process maturity

Andy Purtell — QE Manager (likely direct manager)

Attribute	Detail
Role	Likely your direct manager if hired
Background	QA Engineer → Senior QA → Tech Lead → QE Manager; deep Elektron/real-time data experience
Current Focus	Multi-platform testing at scale (Windows, Linux, Cloud), 500+ products
Communication approach	Lead with *specifics*. Real problems solved, real frameworks built, real bugs caught. Can detect superficial responses.
Key topics to prepare	Framework architecture, flaky tests, cross-platform testing, CI/CD details, mentoring

> ***See Q76 for prepared questions to ask each interviewer, and Q77 for detailed communication style guidance.***

How to Use This Guide

- **Review each category** in order, or focus on areas where you feel less confident.
- **Practice out loud** — Speaking your answers is very different from reading them.
- **Personalize** — Replace generic examples with your own real experiences.
- **Time yourself** — Aim for 2–3 minute answers for most questions.
- **Study the job description** — Many questions are directly tied to JD requirements. Keep the JD handy while preparing.

Testing Fundamentals

Q1: Walk us through how you approach test planning for a new feature — from requirements analysis to test execution.

What's being evaluated: Structured thinking, end-to-end understanding of the QA lifecycle, ability to work independently within defined parameters.

> ***Delivery note:*** *Pick 4–5 key steps and give depth rather than listing all 8. Aim for 2–3 minutes. The interviewer wants to see your thinking process, not a recitation of a textbook list.*

Answer guidance:

1. **Requirements Analysis** — Review functional and non-functional requirements. Clarify ambiguities with product owners and developers. Identify testable acceptance criteria.
2. **Risk Assessment** — Identify high-risk areas (e.g., data integrity in financial transactions, performance under load). Prioritize testing effort accordingly.
3. **Test Strategy & Plan** — Define scope, test types needed (functional, integration, performance, regression), environments, tools, entry/exit criteria, and timelines.
4. **Test Case Design** — Write test cases using techniques like equivalence partitioning, boundary value analysis, decision tables, and state transition diagrams. Trace each case back to a requirement.
5. **Automation Assessment** — Decide which cases are candidates for automation based on repeatability, stability, and ROI.
6. **Environment Setup** — Coordinate test environment provisioning (cloud or on-prem), test data preparation, and stub/mock services if needed.
7. **Execution & Reporting** — Execute tests (manual + automated), log defects with clear reproduction steps, track metrics (pass/fail rate, defect density), and report status to stakeholders.
8. **Closure** — Verify exit criteria are met, prepare test summary report, document lessons learned.

Tips: - Reference a real project where you followed this process. - Mention how you fit into a DevOps team and how testing integrates with CI/CD. - LSEG values process maturity (CMM/CMMI), so emphasize structured documentation.

Q2: What's the difference between system testing, integration testing, system/network integration testing, and acceptance testing? When would you use each?

What's being evaluated: Depth of testing knowledge, understanding of the test levels mentioned explicitly in the job description.

Answer guidance:

Test Level	Purpose	When
Integration Testing	Verify interactions between modules/ components (e.g., service A calling service B).	After unit testing, when individual components are combined.
System Testing	Validate the complete, integrated system against functional and non-functional requirements.	After integration testing, in an environment that mirrors production.
System/Network Integration Testing	Verify the system works correctly across network boundaries, firewalls, load balancers, and with external systems.	Critical in financial infrastructure — e.g., ensuring market data feeds traverse network segments correctly.
Acceptance Testing		Final stage before release.

Test Level	Purpose	When
	Confirm the system meets business requirements and is ready for production. Often involves UAT with stakeholders.	

Tips: - Give a concrete example: "In a trading platform, integration testing checks that the order management module correctly sends orders to the matching engine. System/network integration testing verifies this works across data centers with expected latency." - Mention that at LSEG, system/network integration testing is especially important given the distributed, multi-region financial infrastructure.

Q3: How do you decide what to automate versus what to test manually?

What's being evaluated: Pragmatic judgment, ROI thinking, automation maturity.

Answer guidance:

Automate when: - Tests are repetitive and run frequently (regression, smoke). - Tests are stable — the feature under test is unlikely to change drastically. - Tests involve large data sets or many combinations (data-driven testing). - Tests require precision (e.g., verifying exact financial calculations to decimal places). - Tests are part of CI/CD and need fast feedback.

Keep manual when: - Exploratory testing or usability evaluation. - One-off tests or features still in active flux. - Tests that require human judgment (e.g., visual validation, UX assessment). - The cost of building and maintaining automation exceeds the benefit.

Framework for decision: Use a simple scoring matrix — frequency of execution, stability of feature, complexity of automation, and business criticality. High scores across all four = automate.

Tips: - Avoid the "automate everything" trap — interviewers want to see you think about ROI. - Mention that in financial systems, automating calculations and data validation is high-priority because errors have monetary consequences.

Q4: Explain your understanding of CMM/CMMI. How has it influenced your testing process in previous roles?

What's being evaluated: Process maturity awareness, ability to work within formal SDLC frameworks — explicitly required in the job description.

Answer guidance:

CMMI (Capability Maturity Model Integration) is a process improvement framework with 5 maturity levels:

1. **Initial** — Processes are ad-hoc and chaotic.
2. **Managed** — Projects are planned, measured, and controlled.
3. **Defined** — Processes are standardized across the organization.
4. **Quantitatively Managed** — Processes are measured and controlled using statistical methods.
5. **Optimizing** — Continuous process improvement driven by data.

How it influences testing: - At Level 2+, testing has defined plans, reviews, and tracking. - At Level 3+, test processes are consistent across teams — e.g., standardized test case templates, defect lifecycle, and review gates. - At Level 4+, you track metrics like defect escape rate, test effectiveness, and use them to drive decisions.

Tips: - Be honest about your organization's maturity level and your role in improving it. - Tie it back to the JD: "I've recommended process improvements to my manager based on defect trend analysis" — this directly maps to the scope described.

Udani's bridging answer (Agile-only experience):

> *"I haven't worked in a formally CMMI-assessed organisation — my experience has been in Agile/Scrum environments. However, I've applied many of the same principles that CMMI formalises:*
>
> - *At Level 2 (Managed): I created standardised test plan templates and entry/exit criteria for every sprint, ensuring testing was planned, tracked, and controlled rather than ad-hoc.*
> - *At Level 3 (Defined): I worked to standardise our test automation approach across multiple teams at Sysco LABS — common page object patterns, shared utility libraries, consistent reporting formats — so that any team member could understand and contribute to any team's automation suite.*
> - *At Level 4 (Quantitatively Managed): I tracked defect metrics — escape rates, defect density by module, regression pass rates over time — and used these to drive decisions about where to invest automation effort.*
>
> *So while I haven't operated within a formal CMMI framework, I understand and have applied the underlying principles. I'd be interested to learn how LSEG applies process maturity frameworks in practice."*

Q5: How do you measure test coverage, and when is "enough" testing enough?

What's being evaluated: Analytical thinking, risk-based testing mindset.

Answer guidance:

Types of coverage: - **Requirements coverage** — % of requirements with at least one test case mapped. - **Code coverage** — Line, branch, and path coverage from automated tests (useful but not sufficient alone). - **Risk coverage** — Are high-risk areas tested more thoroughly? Use risk matrices.

"Enough" testing: - All critical/high-priority test cases pass. - Exit criteria defined in the test plan are met (e.g., 95% pass rate, zero critical defects open). - Risk-based assessment: remaining untested areas are low-risk and acceptable to stakeholders. - Defect discovery rate is trending toward zero (diminishing returns).

Tips: - Emphasize that 100% coverage is a myth — what matters is covering the right things. - In financial systems, regulatory and compliance-related tests are non-negotiable.

Q6: Describe a situation where you found a critical defect late in the cycle. How did you handle it?

What's being evaluated: Problem-solving under pressure, communication skills, ownership mentality.

Answer guidance (STAR format):

- **Situation:** Describe the project, timeline pressure, and the defect.
- **Task:** Your responsibility — assessing impact, communicating urgency.
- **Action:** Steps you took — reproduced the defect reliably, documented it with clear evidence, escalated to the dev lead and project manager, participated in root cause analysis, and re-tested the fix.
- **Result:** The outcome — was the release delayed? Was a hotfix deployed? What process change prevented this from happening again?

Tips: - Show ownership: "I took it upon myself to investigate further rather than just logging it." - Mention how you communicated the risk to stakeholders — this shows the customer service awareness LSEG values. - End with a process improvement you suggested to catch similar issues earlier.

Q7: What are the key test design techniques you use? When would you apply each one?

What's being evaluated: Depth of knowledge in systematic test design, ability to choose the right technique for the situation.

Answer guidance:

Technique	What It Does	When to Use
Equivalence Partitioning	Divides input data into partitions where all values in a partition are expected to	

Technique	What It Does	When to Use
	behave the same. Test one value per partition.	When inputs have clear ranges or categories (e.g., age groups, transaction amounts).
Boundary Value Analysis (BVA)	Tests at the exact boundaries of equivalence partitions (min, min+1, max-1, max).	When boundary-related bugs are likely (e.g., off-by-one errors in financial calculations).
Decision Tables	Maps combinations of conditions to expected actions/outcomes in a tabular format.	When multiple conditions interact to determine behavior (e.g., eligibility rules, pricing tiers).
State Transition Testing	Models the system as a finite state machine and tests transitions between states.	When the system has distinct states with defined transitions (e.g., order lifecycle: New → Filled → Settled → Cancelled).
Pairwise/Combinatorial Testing	Tests all possible pairs of parameter values rather than all combinations, dramatically reducing test count.	When there are many configurable parameters and full combinatorial testing is infeasible.

Tips: - Show you don't just use one technique — you pick the right one based on the feature. For example: "For a trade matching engine, I'd use state transition testing for the order lifecycle and BVA for price matching thresholds." - Mention tools like PICT or AllPairs for pairwise testing.

Q8: How do you write effective test cases? What makes a good test case?

What's being evaluated: Practical test design skills, attention to detail, communication through documentation.

Answer guidance:

Characteristics of a good test case: - **Clear title** — Describes what is being tested and the expected outcome (e.g., "Verify order rejection when account balance is insufficient"). - **Preconditions** — State what must be true before the test runs (environment, data, user state). - **Steps** — Numbered, specific, reproducible steps. Anyone should be able to execute them. - **Expected result** — Concrete, verifiable outcome for each step or the overall test. - **Test data** — Explicit values, not vague descriptions ("enter 10,000.50" not "enter a large amount"). - **Traceability** — Linked to a requirement, user story, or acceptance criterion. - **Independence** — Does not depend on other test cases having run first.

Common mistakes to avoid: - Vague steps ("test the login feature"). - Missing negative test cases — only testing the happy path. - Hardcoded environment-specific data that breaks in other environments.

Tips: - Mention that you use a standard template across the team for consistency (aligns with CMMI Level 3+ practices). - In financial systems, test cases for regulatory features need extra rigor — they may be audited.

> ***Elevation note for Associate Lead level:*** *Keep your answer concise and practical, then elevate to automated test design considerations: "Beyond manual test case quality, I also think about how test cases translate to automation — are they data-driven? Are assertions specific and deterministic? Can they run independently in any order? Good test case design at the planning stage directly reduces automation maintenance downstream."*

Q9: Explain the testing pyramid. How do you apply it in practice?

What's being evaluated: Understanding of test architecture, practical application of testing levels, strategic thinking about test ROI.

Answer guidance:

The testing pyramid (bottom to top):

1. **Unit Tests (base — largest number)** - Test individual functions/methods in isolation. - Fast, cheap, and reliable. Run in milliseconds. - Developers typically own these.

2. **Integration/Service Tests (middle)** - Test interactions between components, APIs, and databases. - Slower than unit tests but validate critical interfaces. - SDETs often focus here — API contract tests, database integration tests.
3. **End-to-End / UI Tests (top — fewest)** - Test complete user workflows through the UI. - Slowest, most brittle, most expensive to maintain. - Reserve for critical business flows only.

Applying it in practice: - Advocate for a high ratio of unit tests to catch bugs early and cheaply. - Push integration bugs down to unit tests when possible ("if this API test found the bug, could a unit test have caught it faster?"). - Limit E2E tests to 10–20 critical scenarios to avoid a top-heavy pyramid (the "ice cream cone anti-pattern"). - In CI/CD, run unit tests on every commit, integration tests on every merge, and E2E tests nightly or on release branches.

Tips: - Relate to the SDET role: "As an SDET, I focus most of my automation effort on the integration/API layer — it gives the best balance of coverage, speed, and stability." - In financial systems, the middle layer is critical — API and service tests validate data accuracy, message formats, and business logic without the fragility of UI tests.

Q10: What is risk-based testing and how do you implement it?

What's being evaluated: Strategic thinking, ability to prioritize testing effort based on business impact, practical risk assessment skills.

Answer guidance:

Risk-based testing prioritizes testing effort based on the likelihood and impact of failure for each feature or component.

Implementation steps:

1. **Identify risks:** - Review requirements, architecture, and past defect data. - Consult with developers, product owners, and operations. - Consider: What can go wrong? What's the impact if it does?
2. **Assess each risk on two dimensions:** - **Likelihood** — How probable is a defect? (New code = high, stable code = low; complex logic = high, simple CRUD = low) - **Impact** — What's the business consequence if it fails? (Financial loss, regulatory breach, data corruption, reputational damage)
3. **Create a risk matrix:**

	Low Impact	High Impact
High Likelihood	Medium priority	**Highest priority**
Low Likelihood	Lowest priority	Medium priority

1. **Allocate testing effort:** - High-risk areas get the most test cases, deepest coverage, and earliest testing. - Low-risk areas get basic smoke tests or are deferred.
2. **Review and adjust:** - Risks change as the project evolves. Reassess regularly (e.g., after each sprint).

Tips: - Give a concrete example: "In a payment processing module, I classified the transaction calculation logic as high-likelihood/high-impact and wrote 50+ test cases. The admin settings page was low-likelihood/low-impact and got 5 basic tests." - Mention that risk-based testing aligns with LSEG's need to protect financial data integrity — you test what matters most, not just what's easiest to test. - Show that you communicate risk assessments to stakeholders so they can make informed decisions about test coverage and release readiness.

Test Automation & Programming

Q11: Describe your test automation framework architecture. What design patterns do you use?

What's being evaluated: Technical depth, architectural thinking, hands-on automation experience (2+ years required).

Answer guidance:

Describe a framework you've built or contributed to. A strong answer covers:

- **Architecture layers:**
- **Test Layer** — Test cases and test suites.
- **Business Logic Layer** — Page Objects (for UI) or API client wrappers (for API testing).
- **Utility/Common Layer** — Logging, reporting, config management, test data helpers.
- **Driver/Infrastructure Layer** — WebDriver, REST clients, database connectors.
- **Design patterns:**
- **Page Object Model (POM)** — Encapsulates UI element locators and interactions. Reduces maintenance when UI changes.
- **Data-Driven Testing** — Externalize test data (CSV, JSON, database) to run the same test with different inputs.
- **Keyword-Driven** — Abstract test steps into keywords (common with Robot Framework).
- **Factory Pattern** — For creating test data objects.
- **Singleton** — For managing browser/driver instances.
- **Reporting:** Allure, ExtentReports, or built-in Robot Framework reports.
- **CI Integration:** Tests triggered by Jenkins/GitHub Actions on every commit or nightly.

Tips: - Be specific about tools: "I used Python + pytest + Selenium with POM for UI tests, and requests + pytest for API tests." - Mention how you handle test data, environment configuration, and parallel execution. - If you've used Robot Framework, highlight it — it's mentioned in the JD.

Q12: Write a small script (pseudocode is fine) that reads a CSV of test cases, executes them against an API, and reports pass/fail.

What's being evaluated: Practical coding ability, automation thinking.

Answer guidance (Python example):

```python
import csv
import json
import requests

def run_tests(csv_file, base_url):
    results = []
    with open(csv_file, 'r') as f:
        reader = csv.DictReader(f)
        for row in reader:
            endpoint = row['endpoint']
            method = row['method']
            expected_status = int(row['expected_status'])
            payload = json.loads(row['payload']) if row.get('payload') else None

            response = requests.request(
                method,
                f"{base_url}{endpoint}",
                json=payload
            )
```

```python
            status = "PASS" if response.status_code == expected_status else "FAIL"
            results.append({
                'test': row['test_name'],
                'status': status,
                'expected': expected_status,
                'actual': response.status_code
            })
            print(f"[{status}] {row['test_name']}: "
                  f"Expected {expected_status}, Got {response.status_code}")

    passed = sum(1 for r in results if r['status'] == 'PASS')
    print(f"\nResults: {passed}/{len(results)} passed")
    return results
```

Java equivalent (preferred — Udani's primary language, uses REST Assured):

```java
import io.restassured.RestAssured;
import io.restassured.response.Response;
import org.testng.annotations.DataProvider;
import org.testng.annotations.Test;
import static org.testng.Assert.assertEquals;

import java.io.BufferedReader;
import java.io.FileReader;

public class CsvApiTestRunner {

    @DataProvider(name = "csvTestData")
    public Object[][] readCsvData() throws Exception {
        List<Object[]> data = new ArrayList<>();
        try (BufferedReader br = new BufferedReader(new FileReader("test_cases.csv"))) {
            String line = br.readLine(); // skip header
            while ((line = br.readLine()) != null) {
                String[] values = line.split(",");
                data.add(new Object[]{
                    values[0], // testName
                    values[1], // endpoint
                    values[2], // method
                    Integer.parseInt(values[3]), // expectedStatus
                    values.length > 4 ? values[4] : "" // payload
                });
            }
        }
        return data.toArray(new Object[0][]);
    }

    @Test(dataProvider = "csvTestData")
    public void executeApiTest(String testName, String endpoint,
                               String method, int expectedStatus, String payload) {
        Response response = RestAssured.given()
            .baseUri("https://api.example.com")
            .header("Content-Type", "application/json")
            .body(payload.isEmpty() ? "" : payload)
            .request(method, endpoint);

        assertEquals(response.getStatusCode(), expectedStatus,
            String.format("[%s] Expected %d, got %d", testName,
                          expectedStatus, response.getStatusCode()));
    }
}
```

> ***Note:*** *Java + REST Assured is Udani's primary stack. The Python example above is equally valid — choose based on the interviewer's preference. REST Assured's fluent API makes test code highly readable and is widely used in enterprise test automation.*

Tips: - Show error handling awareness (timeouts, connection errors). - Mention how you'd extend it: parameterized headers, authentication, response body validation. - Keep it clean and readable — they're assessing code quality too.

Q13: How do you handle flaky tests in your automation suite?

What's being evaluated: Practical experience with real-world automation challenges, systematic debugging approach.

Answer guidance:

1. **Identify** — Track flaky tests with metrics (tests that pass/fail inconsistently without code changes). Tag them in your test management tool.
2. **Quarantine** — Move flaky tests to a separate suite so they don't block the CI pipeline, but don't delete them.
3. **Root Cause Analysis:** - **Timing issues** — Add explicit waits (not sleep), use polling/retry mechanisms for async operations. - **Test data dependencies** — Ensure tests create and clean up their own data (test isolation). - **Environment instability** — Shared environments with other teams causing interference. - **Order dependency** — Tests that rely on other tests running first.
4. **Fix** — Address the root cause. If it's an application bug causing inconsistency, log a defect.
5. **Prevent** — Code reviews for test code, guidelines for writing stable tests, mandatory wait strategies.

Tips: - Never say "just add retries" as the solution — that masks the problem. - Mention the business impact: "Flaky tests erode team trust in automation and slow down delivery."

Q14: What's your approach to maintaining test scripts as the application evolves rapidly?

What's being evaluated: Long-term thinking, sustainable automation practices.

Answer guidance:

- **Modular design** — Use Page Object Model or service layers so changes to the UI/API are isolated to one place.
- **Configuration over hardcoding** — Externalize URLs, credentials, test data, and environment settings.
- **Version control** — Test code lives in the same repo (or a linked repo) as application code. PRs for test changes reviewed alongside feature changes.
- **Tagging and categorization** — Tag tests by feature, priority, and type (smoke, regression) so you can selectively run subsets.
- **Regular maintenance sprints** — Dedicate time to update deprecated tests, remove dead code, and refactor.
- **Shift-left** — Developers write unit and integration tests; SDETs focus on higher-level automation. This distributes the maintenance burden.

Tips: - Mention that in a DevOps environment, test maintenance is a continuous activity, not a phase.

Q15: How do you integrate automated tests into a CI/CD pipeline? What tools have you used?

What's being evaluated: DevOps integration skills, CI/CD understanding — role explicitly sits within a DevOps team.

Answer guidance:

Pipeline stages: 1. **Commit stage** — Unit tests and static analysis run on every push. Fast feedback (< 5 min). 2. **Build stage** — Application is compiled/built, Docker images created. 3. **Integration test stage** — API and integration tests run against a deployed test environment. 4. **System test stage** — Full end-to-end and system tests (can be nightly if slow). 5. **Performance test stage** — Load/stress tests on a staging environment (scheduled, not on every commit). 6. **Gate** — Pipeline fails if tests don't meet defined thresholds (e.g., >95% pass rate, no critical defects).

Tools: - **CI/CD:** Jenkins, GitLab CI, GitHub Actions, Azure DevOps. - **Test execution:** pytest, JUnit, Robot Framework, TestNG. - **Containerization:** Docker for consistent test environments. - **Reporting:** Allure, JUnit XML reports integrated into CI dashboards. - **Notifications:** Slack/email alerts on failure.

Tips: - Show that you understand the trade-off between pipeline speed and test thoroughness. - Mention parallelizing tests to reduce pipeline time.

Q16: Explain the Robot Framework. When would you choose it over pytest or a JUnit-based approach?

What's being evaluated: Familiarity with Robot Framework (mentioned in JD), ability to choose the right tool for the context.

Answer guidance:

Robot Framework is a keyword-driven, open-source test automation framework written in Python. It uses a tabular syntax that makes tests readable by non-technical stakeholders.

Key features: - **Keyword-driven** — Tests are written as sequences of keywords (e.g., `Open Browser`, `Input Text`, `Click Button`). - **Extensible** — Libraries for web (SeleniumLibrary), API (RequestsLibrary), database, SSH, and more. - **Built-in reporting** — Generates HTML reports and logs out of the box. - **Tag-based execution** — Run subsets of tests by tags.

When to choose Robot Framework: - Tests need to be readable by non-technical team members (BAs, manual testers). - You want a unified framework for UI, API, and database testing. - Keyword reuse across teams is valuable.

When to choose pytest/JUnit instead: - You need maximum flexibility and complex test logic. - The team is highly technical and prefers code over keywords. - Performance or low-level unit testing where keyword abstraction adds overhead.

Tips: - If you have Robot Framework experience, describe a specific project. - If you don't, be honest but show willingness: "I've primarily used pytest, but I understand Robot Framework's value for cross-functional teams and I'm comfortable picking it up."

Q17: How do you handle test data management in automation?

What's being evaluated: Practical automation experience, understanding of test isolation and data lifecycle challenges.

Answer guidance:

Test data strategies:

1. **Create and tear down** — Each test creates its own data before execution and cleans it up after. Ensures test isolation. - Use API calls or database scripts to seed data. - Use `setup` and `teardown` fixtures (pytest) or `@BeforeEach` / `@AfterEach` (JUnit).
2. **Data factories** — Use the Factory pattern (e.g., FactoryBot, custom builders) to generate realistic test data programmatically. - Avoids hardcoded data that rots over time. - Can generate edge cases and random variations.
3. **Shared reference data** — Some data is static and shared across tests (e.g., country codes, instrument lists). Maintain this in version-controlled seed scripts.
4. **Database snapshots** — Restore a known database state before test suites run. Useful for complex legacy systems where creating data programmatically is difficult.
5. **Data masking/anonymization** — When using production-like data for testing, ensure sensitive information (PII, financial data) is masked to comply with regulations.

Challenges in financial systems: - Test data must be realistic — random strings won't expose bugs in price calculation or trade matching logic. - Regulatory constraints may limit what data can be used in test environments.

- Cross-system test data dependencies (e.g., an instrument must exist in the reference data system before it can be traded).

Tips: - Emphasize test isolation: "Each test should be able to run independently, in any order, without depending on or interfering with other tests." - Mention that poor test data management is one of the top causes of flaky tests.

Q18: How do you implement API test automation? Walk through your approach.

What's being evaluated: Hands-on API testing skills, systematic approach to API quality assurance.

Answer guidance:

Step-by-step approach:

1. **Understand the API:** - Review API documentation (Swagger/OpenAPI specs). - Identify endpoints, methods, request/response schemas, authentication, and rate limits.
2. **Design test categories:** - **Positive tests** — Valid inputs, expected responses, correct status codes. - **Negative tests** — Invalid inputs, missing required fields, unauthorized access, malformed payloads. - **Boundary tests** — Min/max values, empty strings, large payloads. - **Security tests** — Authentication/authorization checks, injection attempts, sensitive data exposure. - **Performance tests** — Response time under load, concurrent requests.
3. **Implement the framework:** - Use `requests` + `pytest` (Python) or `RestAssured` (Java). - Create a base API client class that handles authentication, headers, and base URL configuration. - Write assertion helpers for common checks (status code, response schema, specific field values).
4. **Schema validation:** - Use JSON Schema validation to verify response structure automatically. - This catches breaking changes even when specific field values are correct.
5. **Contract testing:** - Use tools like Pact to ensure consumer-provider API contracts are maintained. - Especially important in microservices architectures.
6. **CI integration:** - Run API tests on every PR merge — they're fast and stable. - Include response time assertions to catch performance regressions early.

Tips: - API tests give the best ROI in the testing pyramid — fast, reliable, and cover critical business logic. - In financial systems, validate decimal precision, currency handling, and timestamp formats rigorously.

Q19: What is BDD? How have you used Cucumber/Gherkin in test automation?

What's being evaluated: Understanding of Behavior-Driven Development, ability to bridge communication between technical and non-technical stakeholders.

Answer guidance:

BDD (Behavior-Driven Development) is a collaborative approach where developers, testers, and business stakeholders define application behavior using structured natural language before development begins.

Gherkin syntax:

```
Feature: Order Placement
  Scenario: Successful market order
    Given a client account with sufficient balance
    And the market is open for trading
    When the client places a market order for 100 shares of "LSEG.L"
    Then the order should be accepted
    And the order status should be "Filled"
    And the account balance should be reduced by the trade amount
```

How Cucumber works: - **Feature files** — Written in Gherkin by BAs/testers to describe business scenarios. - **Step definitions** — Code (Python/Java) that maps each Gherkin step to automation logic. - **Runners** — Execute scenarios and generate reports.

Benefits: - Living documentation — feature files serve as both specs and tests. - Shared understanding — reduces miscommunication between business and development. - Traceability — each scenario maps to a business requirement.

Challenges: - Overhead — writing and maintaining step definitions adds effort. - Anti-pattern risk — overly technical Gherkin defeats the purpose (e.g., "When I click the submit button on the /api/v2/orders endpoint"). - Step definition explosion — too many unique steps become unmaintainable.

Tips: - Be pragmatic: "BDD is most valuable when there's active collaboration between testers and business analysts. If the feature files are only read by developers, the overhead may not be justified." - Mention tools: Cucumber (Java), Behave (Python), or Robot Framework as a keyword-driven alternative.

Q20: How do you ensure your automation tests are reliable across different environments?

What's being evaluated: Environment management skills, understanding of configuration challenges, practical DevOps experience.

Answer guidance:

Key strategies:

1. **Externalize configuration:** - Use environment variables or config files for URLs, credentials, database connections, and feature flags. - Never hardcode environment-specific values in test code. - Use a config hierarchy: defaults → environment-specific overrides → command-line parameters.
2. **Containerized test environments:** - Use Docker Compose or Testcontainers to create consistent, reproducible environments. - Same database version, same service versions, same network configuration.
3. **Test data isolation:** - Each environment has its own test data. Don't rely on data that exists in one environment but not another. - Use data factories that create required data before each test run.
4. **Environment health checks:** - Before running tests, verify that the target environment is available and services are healthy. - Fail fast with a clear message rather than running tests against a broken environment.
5. **Abstraction layers:** - Abstract environment-specific behaviors behind interfaces (e.g., authentication may work differently in dev vs. staging). - Use environment-specific implementations behind a common API.
6. **Consistent test execution:** - Use the same test runner, same Python/Java version, same dependency versions across environments. - Lock dependency versions in requirements.txt or pom.xml.

Tips: - Mention a real scenario: "We had tests passing in dev but failing in staging because the database schema was different. I introduced Flyway migrations for test databases to ensure schema consistency." - In financial systems, environment parity is critical — a test that passes in a non-production environment must behave the same way with production-like data volumes and configurations.

Q88: How do you automate testing for Salesforce Lightning applications with Selenium?

What's being evaluated: Hands-on experience with a notoriously difficult automation target. This is a key differentiator — most SDET candidates haven't automated Salesforce Lightning.

Answer guidance:

Salesforce Lightning presents unique automation challenges that standard Selenium approaches can't handle out of the box:

Challenge 1: Shadow DOM

Lightning Web Components use shadow DOM, which standard Selenium locators cannot penetrate.

Solution: Use JavaScriptExecutor to pierce shadow DOM boundaries:

```
// Piercing shadow DOM to access elements inside Lightning components
WebElement shadowHost = driver.findElement(By.cssSelector("lightning-input"));
SearchContext shadowRoot = (SearchContext) ((JavascriptExecutor) driver)
    .executeScript("return arguments[0].shadowRoot", shadowHost);
```

```
WebElement inputField = shadowRoot.findElement(By.cssSelector("input"));
inputField.sendKeys("test value");
```

Challenge 2: Dynamic Element IDs

Salesforce generates dynamic IDs that change across sessions and deployments (e.g., `input-47;a`).

Solution: Use stable locator strategies: - **Relative CSS selectors** based on component hierarchy: `lightning-input[field-label='Account Name']` - **Custom `data-*` attributes** (if the development team cooperates): `[data-testid='account-name-input']` - **XPath with text matching** as a last resort: `//lightning-button[.//span[text()='Save']]` - **Component-level selectors** based on Salesforce component names rather than rendered HTML

Challenge 3: Iframes

Salesforce uses iframes extensively — Visualforce pages, embedded components, and classic pages within Lightning Experience.

```
// Switch to Visualforce iframe embedded in Lightning
driver.switchTo().frame(driver.findElement(
    By.cssSelector("iframe[title='accessibility title']")));
// Interact with elements inside iframe
driver.findElement(By.id("page:form:field")).sendKeys("value");
// Switch back to main content
driver.switchTo().defaultContent();
```

Challenge 4: Asynchronous rendering

Lightning components render asynchronously. Standard waits may not work.

Solution: Custom wait conditions for Lightning's loading states:

```
// Wait for Lightning components to finish loading
WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(30));
wait.until(driver -> {
    return (Boolean) ((JavascriptExecutor) driver).executeScript(
        "return document.querySelector('.slds-spinner') === null " +
        "&& document.readyState === 'complete'"
    );
});
```

Challenge 5: Sandbox management and test data

- Salesforce sandboxes refresh periodically, resetting test data.
- **Solution:** API-first data setup using Salesforce REST API or Bulk API to create test data before UI test execution, rather than relying on pre-existing data.

```
// Create test data via Salesforce REST API before UI test
RestAssured.given()
    .baseUri(sfBaseUrl)
    .header("Authorization", "Bearer " + accessToken)
    .body("{\"Name\": \"Test Account\", \"Industry\": \"Technology\"}")
    .post("/services/data/v58.0/sobjects/Account")
    .then().statusCode(201);
```

Tips: - This is your strongest differentiator. Most candidates automate standard web apps — Salesforce Lightning automation requires specialised knowledge. - Structure your answer as: "Here are the specific challenges I faced, and here's how I solved each one." - Mention BrowserStack for cross-browser Salesforce testing (it's on your CV). - For Andy: this demonstrates exactly the kind of hands-on problem-solving he values.

Q89: How do you approach database testing and SQL validation in your automation?

What's being evaluated: Ability to validate data integrity across systems, SQL proficiency, understanding of data-driven testing in enterprise environments.

Answer guidance:

Context from Udani's experience: At Sysco LABS, testing SAP → Salesforce integrations required validating that data transformations were correct at the database level — an invoice created in SAP should appear with correct values in Salesforce, and vice versa.

Database testing approach:

1. Cross-system data validation:

The most critical database testing pattern in enterprise integration: verify that data flowing between systems arrives correctly.

```sql
-- Validate invoice data between SAP and Salesforce
SELECT
    sap.invoice_id,
    sap.amount AS sap_amount,
    sf.amount AS sf_amount,
    sap.currency,
    sap.vendor_id
FROM sap_invoices sap
JOIN sf_invoices sf ON sap.invoice_id = sf.external_reference_id
WHERE sap.amount != sf.amount
   OR sap.currency != sf.currency;
-- Any rows returned = data transformation bug
```

2. Common SQL patterns for testing:

Pattern	SQL	Use Case
Duplicate detection	`SELECT column, COUNT(*) FROM table GROUP BY column HAVING COUNT(*) > 1`	Find duplicate records that shouldn't exist
NULL/missing data	`SELECT * FROM orders WHERE customer_id IS NULL OR order_date IS NULL`	Validate mandatory fields are populated
Referential integrity	`SELECT o.id FROM orders o LEFT JOIN customers c ON o.customer_id = c.id WHERE c.id IS NULL`	Find orphaned records
Data range validation	`SELECT * FROM prices WHERE unit_price <= 0 OR unit_price > 999999.99`	Validate business rules on data values
Aggregation verification	`SELECT order_id, SUM(line_amount) AS calc_total, order_total FROM orders JOIN order_lines... HAVING calc_total != order_total`	Verify calculated totals match stored totals
Temporal consistency	`SELECT * FROM audit_log WHERE created_date > modified_date`	Catch timestamp anomalies

3. JDBC in automation framework:

Integrate database validation into your test automation:

```java
// Database validation in TestNG test using JDBC
@Test
public void verifyInvoiceDataIntegrity() throws SQLException {
    Connection conn = DriverManager.getConnection(dbUrl, dbUser, dbPass);

    String query = "SELECT sap.invoice_id, sap.amount, sf.amount " +
```

```
                    "FROM sap_invoices sap " +
                    "JOIN sf_invoices sf ON sap.invoice_id = sf.external_ref " +
                    "WHERE ABS(sap.amount - sf.amount) > 0.01";

    ResultSet rs = conn.createStatement().executeQuery(query);

    List<String> mismatches = new ArrayList<>();
    while (rs.next()) {
        mismatches.add(String.format("Invoice %s: SAP=%.2f, SF=%.2f",
            rs.getString("invoice_id"),
            rs.getDouble(2), rs.getDouble(3)));
    }

    assertTrue(mismatches.isEmpty(),
        "Data mismatches found: " + String.join("; ", mismatches));

    conn.close();
}
```

4. Financial context — why this matters:

- In financial systems, data integrity is non-negotiable. A price discrepancy between systems can cause incorrect trading decisions.
- Reconciliation testing (comparing data across systems) is a daily operation in financial infrastructure.
- Decimal precision matters — use `ABS(a - b) > 0.01` rather than `a != b` for floating-point comparison, but understand the required precision for the business context.

Tips: - Connect to your real experience: "In the Vendor Invoice Management project, I automated SQL-based validation of invoice data flowing from SAP to Salesforce — verifying amounts, currencies, vendor mappings, and line item totals." - Mention test data cleanup: "After each test, I ran cleanup queries to remove test data — maintaining database hygiene is critical for test independence." - For financial systems: "At LSEG, I'd expect database testing to include market data validation — ensuring prices, volumes, and timestamps in the database match what was received from exchange feeds."

Performance Testing

> ***Honest framing:*** *This is a gap area. Udani's direct experience with dedicated performance testing tools (JMeter, Gatling, Locust) is limited. Her performance validation experience has been at the API level — using REST Assured to assert response times and monitoring API throughput during integration testing.* ***Strategy:*** *Acknowledge the gap honestly, demonstrate strong conceptual understanding, describe API-level performance validation you have done with REST Assured, and show readiness to learn dedicated tools quickly.*

Q21: How would you design a performance test for a high-throughput financial data feed?

What's being evaluated: Performance testing knowledge applied to LSEG's core domain — market data.

Answer guidance:

1. **Define objectives:** - Throughput: How many messages/second must the system handle? (e.g., 100,000 ticks/second) - Latency: What's the acceptable end-to-end latency? (e.g., < 5ms for market data) - Sustained load: Can it maintain performance over hours?
2. **Test types:** - **Load test** — Simulate expected peak load (e.g., market open, high-volatility events). - **Stress test** — Push beyond expected capacity to find the breaking point. - **Endurance/soak test** — Run at normal load for extended periods to detect memory leaks or resource exhaustion. - **Spike test** — Sudden bursts of activity (e.g., flash crash scenario).
3. **Test design:** - Use realistic data patterns (not just random payloads). - Simulate multiple data consumers. - Measure at multiple points: producer, network, consumer.
4. **Tools:** JMeter, Gatling, Locust, or custom tools for low-latency protocols.
5. **Metrics to capture:** Throughput (msg/sec), latency (p50, p95, p99), CPU/memory utilization, GC pauses, network I/O, error rate.
6. **Analysis:** Compare against SLAs, identify bottlenecks, profile hotspots.

Tips: - Mention that financial data feeds often use specialized protocols (e.g., FIX, ITCH, proprietary binary) — generic HTTP load testing tools may not suffice. - Show awareness that latency outliers (p99) matter more than averages in financial systems.

Q22: What tools have you used for performance/load testing? Walk us through a test you've designed.

What's being evaluated: Hands-on experience, ability to articulate a complete performance test scenario.

Answer guidance:

Pick a real example and walk through it:

- **Context:** "We had an API serving real-time pricing data. Before a major release, we needed to validate it could handle 10x current traffic."
- **Tool choice:** "I used Locust (Python-based) because the team was already proficient in Python and we needed custom load shapes."
- **Script design:** "Created user classes simulating different client behaviors — some polling every second, some subscribing to WebSocket feeds."
- **Environment:** "Deployed the test target on AWS using production-equivalent infrastructure (same instance types, same database size)."
- **Execution:** "Ramped up from 100 to 5,000 concurrent users over 30 minutes, then held steady for 2 hours."
- **Findings:** "Discovered that database connection pooling was exhausted at 3,000 users. Response times degraded from 50ms to 2s. The fix was increasing the pool size and adding read replicas."
- **Outcome:** "After the fix, the system handled 7,000 concurrent users with p99 latency under 200ms."

Tips: - Be specific about numbers and outcomes. - Show that you don't just run the test — you analyze, report, and drive the fix.

Udani's honest framing:

> *"I want to be upfront: my direct experience with dedicated performance testing tools like JMeter or Gatling is limited. My performance validation has been at the API level — using REST Assured to assert response times and throughput during integration testing. For example, in the Vendor Invoice Management project, I added response time assertions to our API tests to catch performance regressions when invoice processing time exceeded acceptable thresholds.*
> *However, I understand the fundamentals well: load testing to validate capacity, stress testing to find breaking points, endurance testing for memory leaks and degradation over time, and spike testing for sudden load changes. I know the key metrics — p50, p95, p99 latency, throughput, error rates, and resource utilisation.*
> *To reduce this gap, I'm preparing to deepen my hands-on skills with JMeter before starting the role — familiarising myself with thread groups, samplers, listeners, and assertions. I understand that performance is mission-critical in financial markets infrastructure, and I'm committed to developing this capability quickly."*

Pre-interview preparation checklist: - [ ] Install Apache JMeter and run a basic load test against a public API. - [ ] Learn key JMeter concepts: thread groups (virtual users), samplers (HTTP requests), listeners (results visualisation), assertions, and timers. - [ ] Create a simple test plan: ramp 50 users over 2 minutes, hold steady for 5 minutes, measure response times and error rates. - [ ] Understand how to read a JMeter results summary: mean/median/p95/p99 response times, throughput (requests/sec), error percentage. - [ ] Be able to explain the difference between JMeter, Gatling, and Locust at a high level.

Q23: How do you identify performance bottlenecks — at the application layer vs. network layer vs. database layer?

What's being evaluated: In-depth analysis capability (explicitly required in JD), systematic debugging approach.

Answer guidance:

Systematic approach:

1. **Establish baseline metrics** across all layers before testing.
2. **Application layer:** - Profile the application (e.g., Java Flight Recorder, Python cProfile, .NET Profiler). - Check for CPU-bound operations, memory leaks, inefficient algorithms, excessive garbage collection. - Monitor thread/connection pool utilization.
3. **Network layer:** - Check latency between services (ping, traceroute). - Monitor bandwidth utilization, packet loss, TCP retransmissions. - Look for DNS resolution delays, firewall/proxy overhead. - Tools: Wireshark, tcpdump, netstat.
4. **Database layer:** - Identify slow queries (slow query log, EXPLAIN plans). - Monitor connection pool usage, lock contention, deadlocks. - Check I/O wait times, index utilization, table scan frequency. - Tools: Database-native monitoring, APM tools.
5. **Infrastructure:** - CPU, memory, disk I/O, network I/O on each host. - Cloud-specific: check if you're hitting service limits (e.g., AWS throttling).

Tips: - Describe a real scenario where you narrowed down a bottleneck: "Initially it looked like the API was slow, but profiling showed the application was waiting on database queries. The root cause was a missing index on a frequently-joined column."

Q24: What's the difference between load, stress, soak, and spike testing?

What's being evaluated: Clarity of performance testing concepts, ability to choose the right test type for the situation.

Answer guidance:

Test Type	Purpose	Load Profile	Duration	What It Reveals
Load Testing	Validate system behavior under expected peak load.	Ramp up to expected max concurrent users/ transactions.	Minutes to hours	Whether SLAs are met under normal peak conditions.
Stress Testing	Find the system's breaking point by exceeding expected capacity.	Increase load progressively beyond expected maximum until failure.	Until failure occurs	Maximum capacity, failure mode, and recovery behavior.
Soak (Endurance) Testing	Detect issues that emerge over extended periods — memory leaks, resource exhaustion, connection pool depletion.	Steady, moderate load (e.g., 70% of peak).	Hours to days (8+ hours typical)	Memory leaks, GC degradation, disk space exhaustion, log file growth.
Spike Testing	Validate behavior under sudden, dramatic load changes.	Sudden jump from low to very high load, then back.	Short bursts	Auto-scaling behavior, queue overflow handling, error rates during surges.

How to choose: - **Before release:** Load test (meets SLAs?), stress test (what's the ceiling?). - **Before production deployment:** Soak test (stable over a trading day?). - **For event readiness:** Spike test (IPO day, index rebalancing, earnings announcements).

Tips: - In financial systems, soak testing is critical — the system must remain stable throughout a full trading day (6.5+ hours for equities markets). - Mention that stress testing should also verify graceful degradation: "When the system reaches capacity, does it reject new connections cleanly, or does it crash?"

Q25: How do you set performance baselines and SLAs?

What's being evaluated: Strategic thinking about performance, ability to define measurable quality targets, stakeholder communication.

Answer guidance:

Setting baselines:

1. **Measure current performance:** - Run performance tests against the existing system to establish current metrics. - Capture: response time (p50, p95, p99), throughput, error rate, resource utilization. - Run during realistic conditions (production-like data volume, concurrent users).
2. **Document baselines:** - Record metrics with environment details, data volume, and test date. - These become the reference point for detecting regressions.
3. **Trend tracking:** - Run the same performance tests regularly (weekly or per release). - Plot metrics over time to detect gradual degradation.

Defining SLAs:

1. **Collaborate with stakeholders:** - Business owners define acceptable user experience (e.g., "market data must update within 10ms"). - Operations defines infrastructure constraints (e.g., "system must handle 50,000 concurrent connections").
2. **Use percentiles, not averages:** - "p99 response time < 200ms" is more meaningful than "average response time < 100ms" because averages hide outliers.
3. **Include error budgets:** - "99.9% of requests return successfully" — this allows 0.1% error rate, which is realistic for distributed systems.
4. **Make SLAs testable:** - Every SLA should be automatically verifiable in the performance test suite. - Pipeline gates should enforce SLAs: fail the build if p99 latency exceeds the threshold.

Tips: - In financial markets, SLAs are often driven by regulatory requirements and competitive pressure — a data feed that's 1ms slower than a competitor's loses customers. - Show that you understand the difference between SLIs (what you measure), SLOs (what you target), and SLAs (what you promise).

Q26: Describe how you'd performance-test a database-heavy application.

What's being evaluated: Practical performance testing skills, database performance knowledge, ability to design targeted tests.

Answer guidance:

Approach:

1. **Identify database-intensive operations:** - Review the application to find operations with heavy database interaction: reports, searches, batch processing, transaction histories. - Check for N+1 query patterns, large result sets, complex joins.
2. **Prepare realistic test data:** - Load the database with production-scale data volumes (not just 100 rows). - Include realistic data distribution (e.g., some accounts with 10 trades, others with 10 million). - Ensure indexes are built and statistics are up to date.
3. **Design targeted test scenarios:** - **Query performance:** Benchmark individual critical queries with EXPLAIN/ANALYZE. - **Concurrent access:** Simulate multiple users performing reads and writes simultaneously. - **Lock contention:** Test scenarios where multiple transactions update the same rows. - **Connection pooling:** Gradually increase concurrent connections to find the pool saturation point. - **Batch operations:** Test ETL jobs, end-of-day processing, bulk inserts under load.
4. **Monitor at all levels:** - **Application:** Response times, queue depths, thread pool usage. - **Database:** Slow query log, lock waits, buffer pool hit ratio, disk I/O, active connections. - **Infrastructure:** CPU, memory, disk IOPS, network throughput.
5. **Optimize and retest:** - Add missing indexes, optimize queries, tune connection pool sizes. - Consider read replicas, caching layers, or query result pagination. - Retest to verify improvements and ensure no regressions.

Tips: - In financial systems, end-of-day batch processing (settlement, reconciliation, reporting) often puts extreme load on databases. Test these scenarios specifically. - Mention that you'd test with production-scale data: "Testing with 1,000 rows won't reveal issues that appear with 100 million rows — query plans change, indexes behave differently, and memory requirements shift." - Show awareness of database-specific tuning: PostgreSQL's `work_mem`, Oracle's SGA, SQL Server's max degree of parallelism.

Infrastructure, Cloud & Networking

Q27: Compare how you'd set up a test environment on AWS vs. Azure. What services would you use?

What's being evaluated: Cloud familiarity (AWS/Azure mentioned in JD), practical infrastructure knowledge.

Answer guidance:

Concern	AWS	Azure
Compute	EC2 instances or ECS/EKS for containers	Virtual Machines or AKS for containers
Database	RDS, DynamoDB	Azure SQL, Cosmos DB
Networking	VPC, Security Groups, ALB	VNet, NSG, Application Gateway
CI/CD Integration	CodePipeline, or Jenkins on EC2	Azure DevOps Pipelines
Infrastructure as Code	CloudFormation, Terraform	ARM Templates, Terraform
Test Environment Provisioning	Use Terraform/CDK to spin up ephemeral environments per feature branch	Same with Terraform or Bicep
Cost Management	Use spot instances for test environments, auto-shutdown schedules	Use B-series VMs, auto-shutdown

Tips: - Emphasize ephemeral environments: "I spin up test environments on demand and tear them down after tests complete to save costs." - Mention security: "Test environments should mirror production security controls (VPCs, encryption, IAM roles) to catch security-related issues early."

Q28: Explain the basics of TCP/IP. How would you troubleshoot a connectivity issue between two services in a test environment?

What's being evaluated: Networking fundamentals (mentioned as nice-to-have), systematic troubleshooting.

Answer guidance:

TCP/IP basics: - **IP (Internet Protocol)** — Handles addressing and routing packets between hosts. - **TCP (Transmission Control Protocol)** — Provides reliable, ordered, error-checked delivery of data. Uses a 3-way handshake (SYN, SYN-ACK, ACK) to establish connections. - **Ports** — Identify specific services on a host (e.g., 80 for HTTP, 443 for HTTPS, 5432 for PostgreSQL).

Troubleshooting steps:

1. **Verify DNS resolution:** `nslookup` or `dig` — can the client resolve the server's hostname?
2. **Check network reachability:** `ping` — is the host reachable at the IP level?
3. **Check port connectivity:** `telnet host port` or `nc -zv host port` — is the service port open and accepting connections?
4. **Check firewall/security groups:** Are the right ports allowed in the security group or firewall rules?
5. **Check the service:** Is the target service actually running? (`systemctl status` , `docker ps` , check logs)
6. **Check routing:** `traceroute` — are packets taking the expected path? Any hops timing out?
7. **Check TLS/certificates:** If HTTPS, are certificates valid and trusted?
8. **Capture traffic:** Use `tcpdump` or Wireshark to inspect actual packets if all else looks correct.

Tips: - Walk through a real example: "Service A couldn't reach Service B. Ping worked but telnet to port 8080 timed out. Turned out the security group only allowed traffic from the old subnet."

Q29: You need to test a distributed system across Windows Server and Linux. How do you manage environment consistency?

What's being evaluated: Cross-platform proficiency (both mentioned in JD), practical environment management.

Answer guidance:

- **Containerization** — Use Docker to standardize the application runtime regardless of host OS. This eliminates "works on my machine" issues.
- **Infrastructure as Code** — Define environments using Terraform or Ansible. Ansible is particularly good for cross-platform provisioning (Windows and Linux modules).
- **Configuration management** — Centralize configuration (e.g., environment variables, config files) and use tools like Ansible, Puppet, or Chef to ensure consistency.
- **CI/CD matrix builds** — Run the same test suite on both platforms in the pipeline (e.g., GitHub Actions matrix strategy with `windows-latest` and `ubuntu-latest`).
- **Shared test scripts** — Write tests in cross-platform languages (Python, Java) rather than platform-specific scripts (PowerShell vs. Bash).
- **Platform-specific tests** — Some tests may need to be platform-specific (e.g., Windows service management vs. systemd). Clearly tag and separate these.

Tips: - Mention any experience managing Windows Server environments — it's less common among SDETs and stands out.

Q30: How do you use containers (Docker/Kubernetes) in your testing workflow?

What's being evaluated: Modern DevOps practices, environment management skills.

Answer guidance:

- **Test environment provisioning** — `docker-compose up` to spin up the entire application stack (app + database + message queue) locally or in CI.
- **Test isolation** — Each test run gets a fresh set of containers — no leftover state from previous runs.
- **Service virtualization** — Run mock/stub services in containers (e.g., WireMock for API mocks).
- **Parallel test execution** — Spin up multiple containers to run tests in parallel against isolated instances.
- **Kubernetes for scale testing** — Deploy the application to a K8s cluster to test auto-scaling, pod failures, and rolling updates.
- **Testcontainers library** — Programmatically spin up containers within test code (e.g., spin up a PostgreSQL container before integration tests, tear it down after).

Tips: - If you've used Testcontainers, mention it — it shows modern testing practices. - Relate to the JD: "In a DevOps team, being able to spin up and tear down test environments quickly is essential for fast feedback loops."

Q31: How do you manage test data in cloud environments?

What's being evaluated: Cloud-native testing practices, data management in distributed systems, security awareness.

Answer guidance:

Challenges of test data in the cloud: - Data residency and compliance requirements (GDPR, data sovereignty). - Cost of maintaining large test datasets in cloud storage/databases. - Data consistency across multiple services and regions. - Security of sensitive test data (especially in financial systems).

Strategies:

1. **Ephemeral test databases:** - Spin up a fresh database instance for each test run using IaC (Terraform) or Testcontainers. - Populate with seed data, run tests, tear down. No persistent test data to manage.
2. **Data subsetting:** - Extract a representative subset of production data (anonymized/masked) for testing. - Tools like Delphix, Tonic, or custom scripts can create realistic subsets.

3. **Synthetic data generation:** - Generate test data programmatically that matches production patterns. - Use libraries like Faker (Python) or custom generators for domain-specific data (e.g., realistic trade records, price feeds).
4. **Shared reference data services:** - Centralize reference data (instrument lists, currency codes, exchange calendars) in a read-only service accessible by all test environments.
5. **Data versioning:** - Version test data alongside application code so the right test data is always paired with the right code version. - Use database migration tools (Flyway, Liquibase) for schema evolution.
6. **Security controls:** - Encrypt test data at rest and in transit. - Use IAM roles to restrict access to test data stores. - Never use real customer data in test environments — always mask or synthesize.

Tips: - In financial systems, test data often needs to include realistic market data scenarios (volatile markets, market halts, corporate actions). Random data won't exercise the right code paths. - Mention cost optimization: "We tag all test resources and use automated cleanup to avoid runaway cloud costs from forgotten test environments."

Q32: Explain the difference between horizontal and vertical scaling. How does this affect your test strategy?

What's being evaluated: Infrastructure knowledge, ability to design tests that validate scaling behavior.

Answer guidance:

Definitions:

	Vertical Scaling (Scale Up)	Horizontal Scaling (Scale Out)
What	Add more resources (CPU, RAM) to a single server.	Add more servers/instances to distribute the load.
Limit	Bounded by maximum hardware capacity.	Theoretically unlimited (add more nodes).
Complexity	Simple — no application changes needed.	Requires stateless design, load balancing, distributed data management.
Downtime	May require restart to resize.	Can scale without downtime (auto-scaling).
Cost	Exponentially more expensive at higher tiers.	Linear cost scaling.

Impact on test strategy:

1. **Vertical scaling testing:** - Test on different instance sizes to find the optimal cost/performance ratio. - Verify the application takes advantage of additional resources (e.g., multi-threaded). - Test the upper bound — what happens when you can't scale up further?
2. **Horizontal scaling testing:** - **Auto-scaling validation:** Verify that new instances spin up under load and scale down when load decreases. - **State management:** Test that sessions/data are not lost when requests are routed to different instances. - **Load balancer behavior:** Verify even distribution, health check accuracy, and sticky session behavior. - **Data consistency:** Test that distributed caches, databases, and message queues remain consistent across nodes. - **Failover testing:** Kill individual nodes during load testing to verify the system recovers without user impact.
3. **Combined approach (most common):** - Test both strategies: scale up the individual nodes, then scale out the number of nodes. - Find the "right-sizing" sweet spot: optimal instance size × number of instances.

Tips: - Financial systems often use horizontal scaling for stateless services (API gateways, data distribution) and vertical scaling for stateful components (databases, matching engines). - Mention auto-scaling metrics: "I configure scale-out triggers based on CPU utilization, request queue depth, or custom metrics like message processing lag."

Q33: How do you test applications deployed in a microservices architecture?

What's being evaluated: Modern architecture understanding, testing strategy for distributed systems, ability to handle complexity.

Answer guidance:

Challenges of testing microservices: - Many independently deployable services with complex interactions. - Service version compatibility — Service A v2 may not work with Service B v1. - Distributed data management — no single database to verify. - Network-related failures (latency, timeouts, partial failures).

Testing strategy layers:

1. **Unit tests (per service):** - Each service has its own unit tests covering business logic. - Developers own these; fast feedback on every commit.
2. **Contract tests (between services):** - Use Pact or Spring Cloud Contract to verify that API contracts between consumer and provider are maintained. - Run independently for each service — no need to deploy all services together. - Catches breaking changes early without full integration environments.
3. **Integration tests (per service):** - Test each service with its real dependencies (database, message queue) using Testcontainers. - Verify database interactions, message production/consumption.
4. **End-to-end tests (full system):** - Deploy all services together in a staging environment. - Test critical business flows that span multiple services. - Keep these minimal — they're slow and brittle.
5. **Chaos/resilience testing:** - Use tools like Chaos Monkey, Gremlin, or Litmus to inject failures. - Test: What happens when a service is down? When the network is slow? When a database is unavailable? - Verify circuit breakers, retries, and fallback behaviors.
6. **Observability testing:** - Verify that distributed tracing (Jaeger, Zipkin) correctly tracks requests across services. - Test that logging is consistent and correlated across services. - Verify monitoring alerts fire correctly.

Tips: - Emphasize contract testing: "In a microservices architecture, contract tests are the most valuable investment because they catch integration issues without the cost of full system deployment." - In financial systems, data consistency across microservices is critical — mention eventual consistency patterns and how you test for them (e.g., verifying that a trade recorded in the order service eventually appears in the settlement service).

Financial Domain

> ***Known gap area.*** *Both interviewers have spent their entire careers in financial data — Amy at Thomson Reuters/ Refinitiv/LSEG, Andy deeply embedded in Elektron/real-time data systems. You don't have direct financial markets experience.* ***Use the bridging pattern throughout this section:*** *"Here's what I know → Here's what connects from my experience → Here's how I'll close the gap." Your Chartered Accountancy background is an untapped differentiator — most SDET candidates have zero financial knowledge.*

Q34: What do you know about LSEG and its products?

What's being evaluated: Research effort, genuine interest in the company and domain.

Answer guidance:

LSEG (London Stock Exchange Group) is a leading global financial markets infrastructure and data provider. Key divisions:

- **Capital Markets** — Operates the London Stock Exchange, Borsa Italiana, and Turquoise (equities trading venues).
- **Data & Analytics (Refinitiv)** — Provides financial data, analytics, and trading platforms (Eikon/Workspace, Elektron data feeds). Acquired Refinitiv in 2021.
- **FTSE Russell** — Global index provider (FTSE 100, Russell 2000, etc.). Used for benchmarking, ETFs, and derivatives.
- **Post Trade** — Clearing (LCH) and settlement services.

Purpose: Driving financial stability, empowering economies, enabling sustainable growth.

Values: Integrity, Partnership, Excellence, Change.

Scale: 25,000 people across 65 countries.

Tips: - Don't just recite facts — connect them to the role: "As an SDET testing financial data systems, I understand that data accuracy and system reliability directly impact market participants and, by extension, financial stability." - Mention recent news or initiatives (e.g., LSEG's partnership with Microsoft, cloud migration efforts).

Udani's bridging answer:

> *"While I haven't worked directly in financial markets, I've done my research on LSEG. I know LSEG is more than the London Stock Exchange — it includes Refinitiv for data and analytics, FTSE Russell for indexing, and LCH for clearing. I'm particularly interested in the Data & Analytics division and LSEG Real-Time (formerly Elektron), which delivers real-time market data to financial institutions globally.*
> *What connects to my background: my Chartered Accountancy studies at CA Sri Lanka gave me a working understanding of financial data, reporting, and the importance of data integrity in financial contexts. And my experience testing complex enterprise integrations — SAP to Salesforce data flows, cross-system invoice reconciliation — is directly analogous to the data pipeline testing that LSEG's infrastructure requires. The stakes are different — market data errors can cause incorrect trading decisions worth millions — but the testing principles of data fidelity, transformation validation, and end-to-end integrity are the same. I'm eager to deepen this into financial markets specifically."*

Q35: How does testing financial markets software differ from testing a typical web application?

What's being evaluated: Domain awareness, understanding of the unique challenges in fintech/capital markets.

Answer guidance:

Aspect	Financial Software	Typical Web App
Data accuracy		

Aspect	Financial Software	Typical Web App
	Must be exact to decimal precision. A rounding error can mean millions in losses.	Minor display inconsistencies may be tolerable.
Latency	Microsecond-to-millisecond latency requirements for market data and trading.	Sub-second response times are usually acceptable.
Reliability	Near-zero downtime. Market hours are critical — an outage during trading means regulatory scrutiny and financial loss.	Some downtime may be acceptable with proper communication.
Regulatory compliance	Must comply with regulations (MiFID II, SEC rules, etc.). Audit trails are mandatory.	Compliance varies by industry but is typically less strict.
Concurrency	Massive concurrent transactions during market events (e.g., market open, index rebalancing).	Concurrent users, but typically less extreme.
Data volume	Millions of price updates per second across thousands of instruments.	Typically lower data throughput.
Security	Financial data is highly sensitive. Encryption, access controls, and penetration testing are critical.	Important but consequences of breach may be less severe.
Rollback complexity	Financial transactions may be irreversible. Testing must ensure correctness before production.	Easier to roll back or fix in production.

Tips: - Show respect for the domain: "I understand that in financial systems, a bug isn't just a user inconvenience — it can have regulatory, financial, and reputational consequences."

Udani's bridging answer:

> *"I can see the parallels between my experience and financial systems testing, even though the domains differ. In my E-Commerce testing, I dealt with:*
>
> - ***Data accuracy at scale** — pricing calculations across thousands of products, discount rules, VAT calculations. A rounding error in an invoice affects customer trust and revenue. In financial markets, that same principle applies at a much higher magnitude — a data error in a market data feed can cause incorrect trading decisions.*
> - ***Latency sensitivity** — our E-Commerce platform needed real-time inventory updates. If stock levels lagged behind orders, customers could purchase out-of-stock items. Financial systems take this to another level — microsecond latency matters for market data.*
> - ***Regulatory and audit requirements** — our Salesforce and SAP integrations had to maintain audit trails for invoicing and customer data changes. Financial systems extend this with regulations like MiFID II and SOX.*
>
> *My Chartered Accountancy studies also give me a foundation that most SDET candidates lack — I understand financial statements, data integrity in accounting contexts, and the business processes behind financial reporting. I'm committed to deepening this into capital markets specifics — I've already been studying market data flows, FIX protocol basics, and LSEG's product suite."*

Q36: What considerations are important when testing systems that handle market data or trade execution?

What's being evaluated: Domain-specific testing knowledge, understanding of financial system criticality.

Answer guidance:

- **Data integrity** — Verify that prices, volumes, and timestamps are accurate end-to-end. Compare against source data feeds.

- **Latency testing** — Measure and validate that processing latency meets SLAs (often single-digit milliseconds).
- **Sequencing** — Market data must arrive in the correct order. Test for out-of-order message handling.
- **Completeness** — Ensure all subscribed instruments are being received. No missing symbols.
- **Edge cases** — Market halt/resume messages, corporate actions (splits, dividends), zero-volume instruments, negative price adjustments.
- **Failover and resilience** — Test what happens when a feed goes down. Does the system switch to backup feeds? Are stale prices flagged?
- **Market replay** — Use recorded market data to replay historical scenarios (e.g., flash crash, high-volatility events) and validate system behavior.
- **Regulatory compliance** — Ensure timestamps meet regulatory precision requirements (e.g., MiFID II requires microsecond-level timestamps). Verify audit trails are complete.
- **Security** — Ensure unauthorized users cannot access sensitive trading data or execute unauthorized trades.

Tips: - If you have financial markets experience, this is your moment to shine. - If you don't, show that you've researched the domain and understand the stakes.

Q37: Have you worked with FIX protocol or any financial messaging standards?

What's being evaluated: Domain-specific technical knowledge (nice-to-have per JD).

Answer guidance:

FIX (Financial Information eXchange) is the standard messaging protocol for electronic trading. It defines message formats for orders, executions, market data, and more.

Key concepts: - **Sessions** — A FIX connection between two parties (initiator and acceptor) with sequence number management. - **Message types** — NewOrderSingle (D), ExecutionReport (8), MarketDataRequest (V), Heartbeat (0), etc. - **Tags** — Each field has a numeric tag (e.g., Tag 35 = MsgType, Tag 55 = Symbol, Tag 44 = Price). - **Testing FIX:** Use tools like QuickFIX (open-source FIX engine), FIX analyzers, or custom simulators to send/receive FIX messages and validate behavior.

If you haven't worked with FIX: "I haven't worked directly with FIX protocol, but I understand it's the industry standard for trading communication. I've worked with similar message-based protocols and I'm confident I can learn FIX quickly. I've already reviewed the FIX specification to understand the message structure."

Tips: - Even surface-level knowledge of FIX protocol terminology will impress. - Mention related experience: message queues (Kafka, RabbitMQ), binary protocols, or any financial messaging.

Q38: What is market data and how does it flow through a trading system?

What's being evaluated: Understanding of LSEG's core domain, ability to articulate data flows in financial systems.

Answer guidance:

What is market data? Market data is real-time and historical information about financial instruments — prices, volumes, order book depth, trade executions, and corporate actions. It originates from exchanges and is distributed to market participants (traders, portfolio managers, risk systems).

Data flow in a typical trading system:

1. **Source/Exchange** → Generates raw market data (trades, quotes, order book updates).
2. **Feed handler** → Receives raw data via exchange-specific protocols (e.g., ITCH, FIX, proprietary binary), normalizes it into a common format.
3. **Ticker plant / Data platform** → Aggregates, enriches, and distributes data. Applies entitlements (who can see what).
4. **Distribution layer** → Publishes data via APIs, message buses (Kafka, Solace), or proprietary platforms (Refinitiv Elektron).
5. **Consumer applications** → Trading desks (Eikon/Workspace), algorithmic trading engines, risk management systems, compliance systems.

6. **Persistence** → Time-series databases store historical data for analytics, backtesting, and regulatory reporting.

Key testing considerations: - **Latency at each hop** — Where is latency introduced? Is each component meeting its SLA? - **Data fidelity** — Is the data identical at the consumer as it was at the source? - **Throughput** — Can each component handle peak data rates (e.g., market open, high-volatility events)? - **Failover** — If one feed handler fails, does the system seamlessly switch to a backup? - **Entitlements** — Are data access controls enforced correctly? (e.g., a client entitled to delayed data shouldn't receive real-time data)

Tips: - Connect to the role: "As an SDET at LSEG, I'd expect to test the data platform and distribution layers — ensuring data accuracy, latency, and entitlements work correctly across the pipeline." - Show you understand the business impact: "Inaccurate market data can lead to wrong trading decisions, regulatory fines, and loss of client trust."

Q39: How do you test for regulatory compliance (e.g., MiFID II, SOX)?

What's being evaluated: Awareness of regulatory requirements in financial systems, ability to translate compliance rules into testable criteria.

Answer guidance:

Key regulations and their testing implications:

Regulation	Key Requirement	What to Test
MiFID II (EU)	Transaction reporting, best execution, timestamp precision (microsecond).	Verify timestamps are microsecond-precise. Test transaction reporting completeness and accuracy. Validate best execution logic.
SOX (US)	Financial reporting accuracy, internal controls, audit trails.	Verify audit logs capture all data modifications. Test access controls (separation of duties). Validate data integrity in financial reports.
GDPR (EU)	Data privacy, right to erasure, consent management.	Test data masking/anonymization. Verify deletion workflows actually remove data. Test consent tracking.
MAR (EU)	Market abuse detection, suspicious transaction reporting.	Test surveillance system alerts. Verify detection rules for insider trading patterns, market manipulation.

Testing approach:

1. **Translate regulations into test requirements:** - Work with compliance teams to understand the specific rules. - Create a traceability matrix: regulation clause → system requirement → test case.
2. **Audit trail testing:** - Verify every user action, data modification, and system event is logged. - Test that logs are tamper-proof and include who, what, when, and from where. - Validate log retention periods meet regulatory requirements.
3. **Access control testing:** - Test role-based access controls (RBAC) — users should only access functions appropriate to their role. - Verify separation of duties (e.g., the person who creates a trade cannot also approve it).
4. **Data accuracy testing:** - Financial calculations must be precise — test with known inputs and expected outputs. - Verify rounding behavior matches regulatory standards.
5. **Reporting testing:** - Validate regulatory reports (content, format, timeliness). - Test report generation under high-volume conditions.

Tips: - You don't need to be a compliance expert — show that you know how to collaborate with compliance teams and translate their requirements into testable criteria. - Mention that compliance tests should be part of the automated regression suite — they run on every release, not just during audits.

Q40: What is reconciliation testing in the context of financial systems?

What's being evaluated: Understanding of a critical financial testing practice, attention to data integrity.

Answer guidance:

What is reconciliation? Reconciliation is the process of comparing two or more sets of data to verify they agree. In financial systems, it ensures that records across different systems are consistent — for example, that the trades recorded in the order management system match the trades recorded in the settlement system.

Types of reconciliation in financial systems:

1. **Trade reconciliation** — Verify that trades captured in the front-office system match those in the back-office/settlement system.
2. **Position reconciliation** — Confirm that the positions (holdings) calculated by the trading system match those in the custody/accounting system.
3. **Cash reconciliation** — Ensure that cash balances across accounts, banks, and internal systems agree.
4. **Market data reconciliation** — Compare market data received by your system against the official exchange data to detect discrepancies.
5. **Regulatory reconciliation** — Verify that data reported to regulators matches internal records.

How to test reconciliation:

1. **Set up test scenarios with known discrepancies:** - Missing records, duplicate records, mismatched amounts, timing differences. - Verify the reconciliation process detects and flags each type of discrepancy.
2. **Test break resolution workflow:** - When a reconciliation "break" (mismatch) is found, test the workflow for investigating and resolving it. - Verify that breaks are properly escalated based on severity and age.
3. **Volume testing:** - Reconciliation processes often handle millions of records. Test with production-scale volumes. - Verify performance doesn't degrade as data volume grows.
4. **Timing tests:** - Reconciliation is often time-sensitive (e.g., must complete before market open). - Test that the process finishes within the required window.

Tips: - Reconciliation failures in financial systems can indicate fraud, system errors, or data corruption — testing these processes is critical. - Show understanding of the T+1/T+2 settlement cycle: "Trade reconciliation must account for settlement timing — a trade executed today may not settle for 1-2 business days." - Mention that reconciliation testing is particularly important during system migrations: "When migrating to a new platform, we'd run parallel reconciliation between old and new systems to validate data integrity."

DevOps & SDLC

Q41: How do you fit into a DevOps team as an SDET? Describe your day-to-day collaboration with developers and ops.

What's being evaluated: Team collaboration, DevOps mindset — the JD explicitly places this role in a DevOps team.

Answer guidance:

Daily activities: - **Standup** — Share testing progress, blockers, and what's coming next. Align with developers on which features are ready for testing. - **Code reviews** — Review developer PRs with a testing lens: "Is this testable? Are edge cases handled? Are there unit tests?" - **Automation development** — Write and maintain automated tests alongside feature development (not after). - **Pipeline monitoring** — Watch CI/CD pipeline results, investigate failures, fix broken tests. - **Shift-left collaboration** — Participate in design reviews and sprint planning to identify testability concerns early. - **Bug triage** — Work with developers to reproduce, investigate, and prioritize defects.

Collaboration with Ops: - Help define health checks, monitoring alerts, and acceptance criteria for deployments. - Participate in incident postmortems — provide testing perspective on what was missed. - Support production validation after releases.

Tips: - Emphasize that you see yourself as a quality advocate embedded in the team, not a gatekeeper. - Mention the JD's point about supporting the technical operations group post-release.

Q42: Describe a process improvement you recommended and implemented. What was the measurable impact?

What's being evaluated: Initiative, analytical thinking, process improvement mindset — explicitly mentioned in the JD scope.

Answer guidance (STAR format):

Example: - **Situation:** "Our regression test suite took 4 hours to run, which meant we could only run it nightly. Developers didn't get feedback until the next morning." - **Task:** "I was asked to find ways to speed up the feedback loop." - **Action:** "I analyzed test execution times and found that 60% of the time was spent on UI tests that could be replaced with API tests. I also parallelized the remaining UI tests across 4 browser instances and implemented test prioritization based on code change analysis." - **Result:** "Regression suite dropped from 4 hours to 45 minutes. We moved it from nightly to every PR merge. Defect escape rate to production decreased by 30%."

Tips: - Always quantify the impact — hours saved, defects reduced, pipeline time cut. - Show that you didn't just suggest it — you implemented it and measured the results.

Q43: How do you approach workflow analysis to identify quality bottlenecks?

What's being evaluated: Analytical capability, process thinking — JD mentions "workflow analysis and recommends quality improvements."

Answer guidance:

1. **Map the current workflow** — Document the end-to-end process from requirement to production: who does what, how long each step takes, what the handoffs are.
2. **Collect data** — Measure cycle times for each stage, defect injection/detection points, wait times, and rework rates.
3. **Identify bottlenecks:** - Where do stories/tickets pile up? (e.g., waiting for test environments) - Where are defects most frequently found? (e.g., mostly during system testing means earlier testing is insufficient) - Where is rework happening? (e.g., frequent test failures due to unclear requirements)
4. **Root cause analysis** — Use techniques like "5 Whys" or fishbone diagrams to understand why bottlenecks exist.

5. **Recommend improvements** — Propose targeted changes (e.g., "Automate environment provisioning to eliminate the 2-day wait" or "Add API contract tests to catch integration issues earlier").
6. **Measure again** — Track whether the improvement had the desired effect.

Tips: - Use a specific example from your experience. - Show that you approach process improvement with data, not just opinions.

Q44: What does "shift-left testing" mean to you, and how have you practiced it?

What's being evaluated: Modern testing philosophy, proactive quality mindset.

Answer guidance:

Shift-left testing means moving testing activities earlier in the software development lifecycle, rather than treating testing as a phase that happens after development.

How to practice it: - **Requirements review** — Participate in grooming sessions. Ask "How will we test this?" before development starts. Identify ambiguities and missing acceptance criteria. - **Test-driven development (TDD)** — Write tests before code (when applicable). - **Developer testing** — Encourage and support developers in writing unit and integration tests. Provide testing expertise without gatekeeping. - **API contract testing** — Test API contracts early (e.g., using Pact) before full integration environments are available. - **Static analysis** — Integrate linters, SAST tools, and code quality checks in the IDE and commit hooks. - **Early automation** — Start writing automated tests as soon as user stories are defined, not after development is "complete."

Tips: - Shift-left doesn't mean "QA does everything earlier" — it means the whole team takes responsibility for quality earlier. - Relate to LSEG: "In financial systems where defect cost is high, catching issues early in the cycle is especially valuable."

Q45: How do you implement test environment management in a DevOps workflow?

What's being evaluated: Practical DevOps skills, ability to manage complex environment requirements, infrastructure automation.

Answer guidance:

Key principles:

1. **Environments as code:** - Define all environments using Infrastructure as Code (Terraform, CloudFormation, Ansible). - Store environment definitions in version control alongside application code. - Any team member can reproduce any environment from the code.
2. **Ephemeral environments:** - Spin up short-lived environments for feature branches or PR testing. - Automatically tear down after tests pass or after a time limit. - Benefits: no contention, no stale data, cost-effective.
3. **Environment parity:** - Dev, staging, and production should be as similar as possible. - Same OS, same database version, same network topology, same security controls. - Use containers to enforce consistency.
4. **Self-service provisioning:** - Developers and testers can request environments through CI/CD pipelines or chatbots. - No waiting for ops to manually provision — reduces cycle time.
5. **Environment monitoring:** - Monitor environment health (service availability, resource usage, data freshness). - Alert on environment issues before they waste tester time.
6. **Data management:** - Each environment has its own data lifecycle. - Seed scripts run automatically during provisioning. - Sensitive data is masked or synthetic.

Tips: - Give a concrete example: "I set up a GitHub Actions workflow that creates a complete test environment for each PR, runs the test suite, and tears it down — all within 20 minutes." - Mention cost governance: "We tag all test resources with team and purpose, and run automated cleanup of idle environments nightly."

Q46: What is your experience with infrastructure as code for test environments?

What's being evaluated: Hands-on IaC experience, ability to automate environment provisioning, modern DevOps practices.

Answer guidance:

Tools and experience:

Tool	Use Case	Experience
Terraform	Multi-cloud infrastructure provisioning. Define VPCs, compute, databases, networking.	"I use Terraform to provision complete test environments on AWS — VPC, RDS, ECS cluster, and load balancer — in a single `terraform apply`."
Ansible	Configuration management, software installation, cross-platform (Windows + Linux).	"I use Ansible playbooks to configure test servers — install dependencies, deploy the application, and seed test data."
Docker Compose	Local and CI test environments with multiple services.	"Our `docker-compose.yml` defines the full application stack — 5 services, 2 databases, a message broker — for local development and CI testing."
Helm/Kubernetes	Deploying test environments in Kubernetes clusters.	"I create Helm charts for test deployments with configurable replicas, resource limits, and test-specific environment variables."

Benefits for testing: - **Reproducibility** — Same environment every time. No "it worked on my machine." - **Version control** — Environment changes are reviewed, tracked, and reversible. - **Speed** — Spin up a complete environment in minutes, not days. - **Cost** — Ephemeral environments only exist during test execution.

Best practices: - Keep test environment IaC in the same repository as the application (or a closely linked repo). - Use modules/templates for common patterns (e.g., a "standard test database" module). - Include smoke tests in the provisioning pipeline to verify the environment is healthy before running the test suite.

Tips: - Be specific about what you've done: "I reduced our test environment provisioning time from 3 days (manual request to ops) to 15 minutes (automated pipeline)." - Mention drift detection: "I run `terraform plan` regularly to detect configuration drift between what's defined in code and what's actually deployed."

Q47: How do you measure and report on quality metrics to stakeholders?

What's being evaluated: Communication skills, analytical thinking, ability to translate technical metrics into business value.

Answer guidance:

Key quality metrics:

Metric	What It Measures	Why It Matters
Defect escape rate	% of defects found in production vs. total defects.	Measures testing effectiveness — lower is better.
Test coverage	% of requirements/code covered by tests.	Indicates untested risk areas.
Automation rate	% of test cases that are automated.	Indicates regression testing maturity.
Test execution time	Time to run the full test suite.	Impacts CI/CD pipeline speed and feedback loops.
Defect density	Defects per unit of code (e.g., per KLOC or per feature).	Identifies high-risk modules.
Mean time to detect (MTTD)		Measures shift-left effectiveness.

Metric	What It Measures	Why It Matters
	Average time between defect introduction and discovery.	
Mean time to resolve (MTTR)	Average time to fix a defect once discovered.	Measures development responsiveness.

Reporting approach:

1. **Dashboards** — Create real-time dashboards (Grafana, Jira, Azure DevOps) showing key metrics. Make them visible to the team.
2. **Sprint reports** — Include quality metrics in sprint reviews: tests run, pass rate, new defects, automation progress.
3. **Trend analysis** — Show metrics over time, not just snapshots. "Our defect escape rate improved from 15% to 5% over 3 quarters."
4. **Risk reports** — Highlight areas with low coverage or high defect density. Recommend action items.
5. **Stakeholder-appropriate language** — Translate technical metrics into business impact. "5% defect escape rate means 95% of issues are caught before customers see them."

Tips: - Show that you report proactively, not just when asked: "I send a weekly quality summary to the project manager and flag any concerning trends immediately." - Relate to the JD: "I've recommended process improvements based on metric analysis — for example, when defect density spiked in a specific module, I proposed additional code review and targeted testing, which reduced defects by 40%." - In financial systems, quality metrics may need to be shared with compliance and audit teams — show awareness of this.

Defect Management

Q48: After a release goes to production, how do you support the technical operations team and service management?

What's being evaluated: Post-release awareness, customer service mindset — both explicitly mentioned in the JD.

Answer guidance:

- **Production validation** — Run a smoke test suite against production immediately after deployment to confirm critical functionality works.
- **Monitoring** — Watch dashboards and alerts for anomalies in error rates, latency, throughput, or business metrics.
- **On-call support** — Be available (or on rotation) to help investigate issues that arise after release. Provide test environment context to ops teams.
- **Defect triage** — When production issues are reported, help reproduce them, identify root cause, and verify fixes in lower environments before hotfix deployment.
- **Knowledge transfer** — Document known issues, workarounds, and test coverage gaps for the ops team.
- **Postmortem participation** — Contribute to incident reviews with testing perspective: "What could we have tested differently to catch this?"

Tips: - Show that you see your responsibility extending beyond "test and throw over the wall." - The JD mentions supporting "the technical operation group as well as service management" — address both explicitly.

Q49: Describe your defect triage process. How do you prioritize and communicate bug severity?

What's being evaluated: Structured approach to defect management, communication skills.

Answer guidance:

Severity classification:

Severity	Definition	Example
Critical (S1)	System is down or data corruption. No workaround.	Trading system cannot process orders.
High (S2)	Major feature is broken but workaround exists.	Price feed is delayed but manual entry is possible.
Medium (S3)	Feature works but with minor issues.	Report formatting is incorrect but data is accurate.
Low (S4)	Cosmetic or minor inconvenience.	Alignment issue on a rarely-used admin page.

Triage process: 1. **Log with evidence** — Clear title, steps to reproduce, expected vs. actual, environment, screenshots/logs. 2. **Assign severity and priority** — Severity is the technical impact; priority is the business urgency (they don't always align). 3. **Triage meeting** — Regular meetings with dev lead, PM, and QA lead to review new defects, agree on priority, and assign owners. 4. **Communication** — High-severity defects communicated immediately via Slack/email. Status updates until resolution. 5. **Verification** — Once fixed, re-test in the relevant environment and close the defect.

Tips: - Emphasize that you write defect reports that developers can act on immediately — no back-and-forth needed. - Mention that in financial systems, S1/S2 defects may require regulatory notification, adding urgency.

Q50: Tell us about a time a production incident was traced back to a testing gap. What did you learn?

What's being evaluated: Self-awareness, accountability, learning from failure, process improvement.

Answer guidance (STAR format):

- **Situation:** Describe a production incident and how it was traced to insufficient testing.
- **Task:** Your role in investigating and addressing the gap.
- **Action:** What you did — e.g., "I analyzed our test coverage and found we had no tests for [specific scenario]. I created test cases to cover the gap, added them to the regression suite, and proposed a checklist for future features to ensure similar scenarios are covered."
- **Result:** "We haven't had a recurrence. I also shared the learning with the team in a postmortem, and we updated our test design guidelines."

Tips: - Be honest — everyone has testing gaps. The interviewer wants to see how you responded and improved. - Don't blame others. Show ownership: "While the code was written by the development team, I took responsibility for improving the test coverage in that area." - Tie it back to process: "This incident led us to add [specific check] to our test planning process."

Q51: How do you perform root cause analysis on recurring production issues?

What's being evaluated: Analytical depth, systematic problem-solving, ability to drive permanent fixes rather than Band-Aid solutions.

Answer guidance:

Structured RCA approach:

1. **Gather data:** - Collect all related incidents: timestamps, error logs, stack traces, affected systems, user reports. - Identify patterns: Does it happen at specific times? Under specific load? After specific operations?
2. **Timeline reconstruction:** - Build a chronological timeline of events leading to and during each incident. - Identify the trigger (what changed?) and the propagation path (how did it spread?).
3. **Root cause techniques:** - **5 Whys** — Ask "why" iteratively until you reach the fundamental cause.
   - Example: "Why did the trade fail? → Because the price was null. → Why was the price null? → Because the feed handler crashed. → Why did it crash? → Because it couldn't handle a malformed message. → Why couldn't it handle it? → Because there was no input validation. → **Root cause: Missing input validation in the feed handler.**"
   - **Fishbone (Ishikawa) diagram** — Categorize potential causes: People, Process, Technology, Environment.
   - **Fault tree analysis** — Map out all possible paths that could lead to the failure.
4. **Corrective actions:** - **Immediate fix** — Address the symptom to restore service. - **Permanent fix** — Address the root cause to prevent recurrence. - **Detection improvement** — Add monitoring/alerting to catch it earlier next time. - **Test gap closure** — Add test cases that would have caught this issue.
5. **Track and verify:** - Document the RCA in the incident management system. - Assign action items with owners and deadlines. - Verify that the permanent fix actually prevents recurrence.

Tips: - Emphasize that you don't stop at the first "why": "The immediate cause is rarely the root cause. If a production incident was caused by missing validation, the root cause might be that our test planning process doesn't include negative test cases for external data feeds." - Show that RCA leads to systemic improvements, not just one-off fixes.

Q52: Describe your approach to regression testing after a hotfix.

What's being evaluated: Risk management, practical judgment under pressure, understanding of hotfix risks.

Answer guidance:

The challenge: Hotfixes are deployed under time pressure, but inadequate regression testing can introduce new issues. The key is balancing speed with confidence.

Approach:

1. **Targeted regression scope:** - Identify the blast radius of the change: What modules, services, and data flows are affected? - Run the full test suite for affected components, not just the specific fix. - Include upstream and downstream integration tests — a fix in Service A may affect Service B.
2. **Risk-based prioritization:** - **Must run:** Tests directly related to the fix + smoke tests for critical business flows. - **Should run:** Tests for related features and integration points. - **Can skip (with documented risk):** Tests for completely unrelated features.
3. **Automated regression:** - If you have a well-maintained automation suite, run it. This is where automation ROI pays off. - If the suite takes too long, run a prioritized subset (smoke + affected areas).
4. **Manual verification:** - Manually verify the hotfix resolves the reported issue. - Perform brief exploratory testing around the changed area.
5. **Production validation:** - After hotfix deployment, run production smoke tests immediately. - Monitor error rates, latency, and business metrics for anomalies. - Keep the team on standby for rapid rollback if issues emerge.
6. **Post-hotfix activities:** - If regression tests were skipped under pressure, schedule them for the next business day. - Add new test cases to prevent the original issue from recurring. - Update the regression suite if the hotfix changed any behavior.

Tips: - Show that you've handled this in practice: "During a production hotfix for a pricing calculation error, I ran the targeted regression suite (45 minutes) plus manual verification of the specific fix. We caught a secondary issue in the margin calculation that the fix had inadvertently affected." - In financial systems, hotfix regression is especially critical — a fix for one calculation error shouldn't introduce another.

Q53: How do you manage technical debt in test automation?

What's being evaluated: Long-term thinking, practical maintenance awareness, ability to balance speed with sustainability.

Answer guidance:

Common forms of test automation technical debt: - **Flaky tests** that are ignored or disabled rather than fixed. - **Duplicate tests** covering the same functionality with slight variations. - **Hard-coded test data** that breaks when environments change. - **Outdated tests** validating features that have changed or been removed. - **Poor abstractions** — test code copied-and-pasted rather than using shared utilities. - **Missing tests** — areas where automation was planned but never implemented. - **Slow tests** that nobody wants to run because they take too long.

Management strategies:

1. **Make it visible:** - Track automation debt in the backlog like any other technical debt. - Use metrics: number of disabled tests, flaky test rate, test execution time trends.
2. **Allocate regular maintenance time:** - Dedicate a percentage of each sprint (e.g., 10-20%) to automation maintenance. - Alternatively, schedule periodic "test health sprints" focused entirely on cleanup.
3. **Prevent accumulation:** - Code reviews for test code (same standards as production code). - Definition of done includes test maintenance: "No new flaky tests introduced." - Enforce test naming conventions, structure, and abstraction patterns.
4. **Prioritize by impact:** - Fix flaky tests first — they erode team trust in automation. - Remove dead tests — they add confusion and slow execution. - Refactor duplicate tests — they multiply maintenance effort.
5. **Measure improvement:** - Track suite health over time: execution time, pass rate, flaky rate. - Celebrate improvements: "We reduced our flaky test rate from 8% to 1% this quarter."

Tips: - Be honest about the challenge: "Every automation suite accumulates debt over time. The question isn't whether you have debt, but whether you manage it actively." - Relate to the business: "Automation debt directly impacts delivery speed — a flaky suite that takes 2 hours to run and fails 20% of the time costs the team hours of investigation every week." - Show ownership: "I schedule quarterly 'automation health reviews' where I analyze test suite metrics and propose targeted cleanup work."

Leadership & Soft Skills

Q54: This role involves limited coordination of others' work. How do you mentor junior testers or coordinate test efforts?

What's being evaluated: Leadership potential, mentoring ability — the JD mentions "limited co-ordination of the work of others."

Answer guidance:

- **Task delegation** — Break down test efforts into clear, manageable tasks. Assign based on skill level and development goals.
- **Knowledge sharing** — Pair with junior team members on automation tasks. Conduct internal workshops on tools and techniques.
- **Code reviews** — Review their test scripts, provide constructive feedback on test design, coding style, and coverage.
- **Standards and templates** — Create and maintain team test standards (test case templates, naming conventions, automation guidelines) so juniors have a clear framework to follow.
- **Regular check-ins** — Brief daily sync to unblock issues and adjust priorities.
- **Progressive responsibility** — Gradually increase the complexity of tasks assigned as they grow.

Tips: - Give a specific example of mentoring someone and the outcome. - Show that you lead through enablement, not micromanagement.

Q55: Describe a time you had to push back on a developer or product manager regarding quality standards.

What's being evaluated: Professional courage, communication skills, ability to advocate for quality.

Answer guidance (STAR format):

- **Situation:** "A product manager wanted to skip regression testing for a 'minor' database migration to meet a deadline."
- **Task:** "I needed to explain the risk without being confrontational."
- **Action:** "I pulled data from a previous incident where a 'minor' migration caused data inconsistencies in production. I presented the risk in business terms: 'If this migration introduces data errors, we'll spend more time on hotfixes than we saved by skipping regression. We can run a targeted subset in 2 hours instead of the full 6-hour suite.' The PM agreed to the compromise."
- **Result:** "We ran the targeted regression and caught two issues that would have caused incorrect calculations in production."

Tips: - Always frame pushback in terms of business risk, not just "best practices say so." - Show that you propose alternatives rather than just saying "no." - Relate to LSEG's value of **Integrity** — "I believe in being transparent about quality risks, even when it's not what people want to hear."

Q56: How do you handle competing priorities — e.g., a release deadline vs. unresolved defects?

What's being evaluated: Decision-making under pressure, risk assessment, communication.

Answer guidance:

1. **Assess risk** — Classify the unresolved defects by severity and business impact. Are any critical? Are there workarounds?

2. **Communicate transparently** — Present the situation to stakeholders with clear options: - Option A: Delay release to fix all defects. - Option B: Release with known issues (document them), commit to fixing in the next sprint. - Option C: Release with a reduced scope (exclude the affected feature).
3. **Provide a recommendation** — Based on risk assessment: "I recommend Option B because the remaining defects are low-severity cosmetic issues with no functional impact. Here's the known issues document."
4. **Document the decision** — Regardless of the outcome, document who decided what and why. This is crucial in regulated environments.
5. **Follow through** — If defects are deferred, ensure they're tracked and not forgotten.

Tips: - Show that you don't just escalate problems — you come with options and a recommendation. - In financial systems, regulatory and data integrity defects are never acceptable to defer — show you understand this distinction.

Q57: LSEG values Integrity, Partnership, Excellence, and Change. Give an example of how you've demonstrated one of these in your career.

What's being evaluated: Cultural fit, alignment with LSEG values.

Answer guidance:

Prepare one example for each value, then choose the strongest one during the interview:

- **Integrity:** "I discovered that our test results were being manually adjusted to show higher pass rates in management reports. I raised this with my manager, even though it was uncomfortable, because I believe that accurate reporting is essential for making good decisions."
- **Partnership:** "When a new developer joined the team and was struggling with our test framework, I spent time pairing with them rather than just pointing them to documentation. Their ramp-up time was cut in half, and they started contributing test automation within two weeks."
- **Excellence:** "I wasn't satisfied with our 70% automation coverage, so I set a personal goal to reach 90% within two quarters. I created a plan, tracked progress weekly, and involved the team. We hit 92%."
- **Change:** "When our team was resistant to adopting CI/CD, I set up a proof-of-concept pipeline on a small project. The team saw the benefits firsthand — faster feedback, fewer integration issues — and we adopted it for all projects."

Tips: - Choose the value that resonates most with your genuine experience. - Be authentic — interviewers can tell when answers are fabricated.

Q58: How do you handle a disagreement with a team member about test approach or prioritization?

What's being evaluated: Conflict resolution, teamwork, communication skills.

Answer guidance:

1. **Listen first** — Understand their perspective fully before responding. Ask clarifying questions.
2. **Find common ground** — "We both want to ensure quality. Let's discuss the trade-offs of each approach."
3. **Use data** — Support your position with evidence: test metrics, defect history, risk assessment. Avoid opinion-based arguments.
4. **Propose an experiment** — "Let's try your approach on this sprint and measure the outcome. If it works better, we'll adopt it."
5. **Escalate constructively** — If you can't agree, involve the team lead or manager with both perspectives presented fairly.
6. **Accept and commit** — Once a decision is made, support it fully regardless of who "won."

Tips: - Show maturity: "I've learned that my first instinct isn't always right, and some of the best process improvements in my career came from ideas I initially disagreed with."

Q59: How do you onboard a new team member to your test automation framework?

What's being evaluated: Knowledge transfer ability, documentation skills, mentoring approach, team enablement.

Answer guidance:

Structured onboarding plan:

1. **Day 1–2: Context and orientation:** - Walk through the architecture: system under test, test framework structure, CI/CD pipeline. - Share key documentation: README, coding standards, test naming conventions, contribution guidelines. - Set up their local development environment together (pair programming).
2. **Day 3–5: Guided practice:** - Assign a simple "starter task" — e.g., add a new test case to an existing test suite. - Review their work together, explaining the "why" behind conventions and patterns. - Walk through an end-to-end workflow: write test → run locally → push → see CI results.
3. **Week 2: Increasing independence:** - Assign progressively complex tasks. - Have them fix a known flaky test (teaches framework internals and debugging skills). - Introduce them to the defect triage process and team ceremonies.
4. **Week 3–4: Full contribution:** - Assign a real feature's test coverage. - Code reviews with detailed feedback. - Check in on blockers daily, then reduce frequency as confidence builds.

Supporting materials: - **Quick-start guide** — "Clone repo → install dependencies → run first test" in 10 minutes. - **Architecture diagram** — Visual overview of framework layers and how they connect. - **Glossary** — Team-specific terminology, acronyms, and conventions. - **Example tests** — Annotated examples of well-written tests as templates.

Tips: - Show empathy: "I remember being the new person. Having someone who invests time in your onboarding makes a huge difference." - Mention the ROI: "A 2-week structured onboarding saves months of confusion and bad habits. The new team member becomes productive faster, and the code quality stays high."

Q60: Describe how you manage stakeholder expectations when quality is at risk.

What's being evaluated: Communication skills, professional courage, ability to manage up, risk communication.

Answer guidance:

When to raise the flag: - Test coverage is insufficient for a release. - Critical defects are unresolved with the release date approaching. - Test environments are unstable, limiting test confidence. - Timeline pressure is causing shortcuts in testing.

How to communicate:

1. **Be proactive, not reactive:** - Don't wait until the release date to raise concerns. Flag risks as soon as you identify them. - "I'd rather have an uncomfortable conversation early than a crisis later."
2. **Use data, not emotions:** - Present facts: test coverage percentages, defect counts by severity, untested risk areas. - Avoid: "I feel like we're not ready." Use: "We have 12 untested user stories, including 3 that touch payment processing."
3. **Present options with trade-offs:** - Option A: "Delay release by 3 days, complete all testing, high confidence." - Option B: "Release on time with known gaps, accept risk in [specific area], plan hotfix coverage." - Option C: "Reduce scope — release without [feature X], ship it next sprint with full coverage." - Include your recommendation and the reasoning.
4. **Document everything:** - Record the risk assessment, the options presented, and the decision made. - In regulated environments (like financial services), this documentation may be required for audits.
5. **Follow through:** - Whatever is decided, track the deferred items and ensure they're addressed. - After release, report on whether the accepted risks materialized.

Tips: - Frame quality as a business concern, not a QA concern: "This isn't about testing's comfort level — it's about the risk of [specific business impact] if [scenario] occurs." - Show that you manage up without being adversarial: "I've found that stakeholders appreciate transparency. Even if they choose to accept the risk, they want to make that choice with full information."

Q61: How do you stay current with testing tools, methodologies, and industry trends?

What's being evaluated: Growth mindset, continuous learning, professional development.

Answer guidance:

Active learning strategies:

1. **Community engagement:** - Follow testing thought leaders on LinkedIn/Twitter (e.g., Ministry of Testing community, Test Guild). - Participate in local or virtual meetups and conferences (SeleniumConf, TestBash, Agile Testing Days). - Contribute to open-source testing tools or libraries.
2. **Hands-on experimentation:** - Set up personal projects to try new tools (e.g., Playwright vs. Cypress, k6 vs. Gatling). - Build proof-of-concepts before recommending tools to the team. - Maintain a personal GitHub repo with automation experiments.
3. **Structured learning:** - Online courses (Udemy, Coursera, Test Automation University). - Certifications where relevant (ISTQB, AWS, cloud certifications). - Read books: "Agile Testing" by Lisa Crispin, "Continuous Delivery" by Jez Humble.
4. **Knowledge sharing:** - Present learnings to the team in lunch-and-learn sessions. - Write internal tech notes or blog posts about new tools/techniques. - Mentor others — teaching deepens your own understanding.
5. **Industry awareness:** - Follow trends in AI-assisted testing, shift-left practices, and observability. - Understand how emerging technologies (cloud-native, microservices, event-driven architecture) affect testing strategies.

Tips: - Be specific: "Recently I evaluated Playwright as a potential replacement for our Selenium-based UI tests. I built a PoC, compared execution speed and reliability, and presented findings to the team. We're now migrating our critical UI tests." - Show that learning isn't passive: "I don't just read about tools — I build things with them to understand their strengths and limitations." - Relate to the role: "For this LSEG role, I've been researching financial testing practices, FIX protocol, and market data testing approaches."

Scenario-Based Questions

Q62: You're given a legacy system with no existing tests. How do you build a test strategy from scratch?

What's being evaluated: Strategic thinking, prioritization, ability to work independently within defined parameters.

Answer guidance:

1. **Understand the system:** - Review documentation (if any), architecture diagrams, and codebase. - Interview developers, ops, and business users to understand critical workflows and known pain points. - Review production incident history to identify fragile areas.
2. **Prioritize by risk:** - Identify the most critical business flows (e.g., order processing, data feeds, reporting). - Focus first on areas with highest risk of failure and highest business impact.
3. **Start with characterization tests:** - Write tests that document the current behavior (not expected behavior). These act as a safety net before making changes.
4. **Build incrementally:** - Add smoke tests for critical paths (can be run in < 15 minutes). - Add regression tests for areas being actively modified. - Introduce API tests before UI tests (faster to write, more stable, better ROI).
5. **Establish infrastructure:** - Set up a CI pipeline, even a basic one, to run tests automatically. - Set up test environments and test data management.
6. **Set targets:** - Define coverage goals per quarter. Track and report progress.

Tips: - Emphasize pragmatism: "I wouldn't try to achieve 90% coverage in a month. I'd focus on the top 10 critical workflows first." - Mention that you'd make recommendations to your immediate manager (per the JD scope).

Q63: A system integration test is failing intermittently in the CI pipeline but passes locally. Walk us through your debugging approach.

What's being evaluated: Systematic debugging, analytical capability, understanding of CI/CD environments.

Answer guidance:

1. **Gather data:** - How often does it fail? (20% of the time? Only on certain days?) - Is it always the same assertion that fails? - Check CI logs for the exact error message and stack trace.
2. **Identify differences between local and CI:** - **Environment:** OS, installed dependencies, database version, service versions. - **Timing:** CI machines may be slower or faster, causing race conditions. - **State:** Is there leftover data from previous test runs in CI? Locally you might have a clean slate. - **Resources:** CI runners may have limited CPU/memory, causing timeouts. - **Parallelism:** Are other tests running concurrently in CI that share resources?
3. **Reproduce:** - Try running the test multiple times locally in a loop: `for i in {1..50}; do pytest test_x.py; done` - Run in a Docker container matching the CI environment. - Run with the same parallelism settings as CI.
4. **Common culprits:** - **Race conditions** — Async operations completing in different orders. - **Shared test data** — Tests not properly isolating their data. - **External dependencies** — Third-party service being flaky. - **Clock/timezone** — CI server in a different timezone than local.
5. **Fix and verify:** - Address the root cause (not just add a retry). - Run the test 100+ times in CI to confirm stability.

Tips: - Show systematic thinking — don't jump to conclusions. - Mention that you'd add better logging/screenshots for future debugging.

Q64: You discover that a third-party API your system depends on has changed its response format with no notice. What do you do?

What's being evaluated: Incident response, communication, proactive testing practices.

Answer guidance:

Immediate response: 1. **Assess impact** — What functionality is broken? How many users/systems are affected? 2. **Communicate** — Alert the team, product owner, and service management immediately. If production is affected, trigger the incident process. 3. **Investigate** — Confirm the change by comparing current responses to documented API specs. Capture evidence. 4. **Workaround** — If possible, implement a temporary fix (e.g., adapter layer to transform the new format to the expected format).

Resolution: 5. **Contact the vendor** — Report the undocumented change. Ask for release notes, a migration guide, or a rollback. 6. **Fix the code** — Update the application to handle the new format. Update tests accordingly. 7. **Test thoroughly** — Run regression tests with both old and new response formats if backward compatibility is needed.

Prevention: 8. **Contract tests** — Implement API contract tests (e.g., Pact) that run regularly against the third-party API to detect changes early. 9. **Version pinning** — If the API supports versioning, pin to a specific version. 10. **Monitoring** — Add health checks that validate expected response structure from third-party APIs.

Tips: - Show that you think beyond the immediate fix to prevention. - In financial systems, a broken third-party API (e.g., market data provider) can have severe consequences — convey that urgency.

Q65: How would you test a real-time market data streaming service for correctness and performance simultaneously?

What's being evaluated: Ability to design complex test scenarios for LSEG's core domain.

Answer guidance:

Correctness testing: - **Data validation** — Compare received data against a trusted reference source (e.g., exchange direct feed vs. consolidated feed). Verify symbol, price, volume, timestamp accuracy. - **Sequencing** — Verify messages arrive in the correct order. Detect and flag gaps or duplicates. - **Completeness** — Ensure all subscribed instruments are being received. No missing symbols. - **Edge cases** — Market halt/resume messages, corporate actions (splits, dividends), zero-volume instruments, negative price adjustments.

Performance testing: - **Throughput** — Measure messages processed per second under various load levels. - **Latency** — Measure end-to-end latency from source to consumer at p50, p95, p99, and max. - **Sustained load** — Run for extended periods (8+ hours simulating a full trading day) to detect degradation. - **Burst handling** — Simulate sudden spikes (e.g., market open, major news events).

Simultaneous approach: - Run correctness checks as consumers of the streaming data alongside performance monitoring. - Use a "canary consumer" that validates every Nth message for correctness while the performance harness measures throughput and latency. - Compare timestamps across the pipeline to identify where latency is introduced.

Tools: Custom consumers in Java/C++ for low-latency measurement, Kafka consumer lag monitoring, Prometheus/Grafana for dashboards.

Tips: - Show understanding that correctness and performance can be at odds — e.g., adding validation logic introduces latency. - Mention market replay testing using historical data.

Q66: You have one week before release and the automation suite covers only 40% of the new features. What's your plan?

What's being evaluated: Prioritization under pressure, risk-based thinking, practical decision-making.

Answer guidance:

1. **Risk-based prioritization:** - List all untested features. Classify by business criticality and risk. - Focus automation on the highest-risk, most critical paths.

2. **Smart testing strategy:** - **Automate the top 20%** — The critical happy-path scenarios that must work. - **Manual testing for the rest** — Experienced testers do exploratory and edge-case testing on medium-risk features. - **Skip low-risk, low-impact features** — Document them as known coverage gaps with a plan to cover post-release.
3. **Parallel work:** - Split the team: some writing automation, others doing targeted manual testing. - If possible, pair SDETs with manual testers — SDETs automate while testers execute manually.
4. **Communicate:** - Report the coverage gap to the project manager and product owner with a clear risk assessment. - Get agreement on what's acceptable and document the decision.
5. **Post-release plan:** - Commit to increasing automation coverage to 80%+ in the next sprint. - Add the remaining tests to the backlog with estimated effort.

Tips: - Don't promise 100% coverage in a week — show realistic judgment. - Emphasize risk-based decisions: "I'd rather have 100% coverage of the 5 most critical features than 40% coverage of everything." - Mention that in financial systems, you might recommend delaying the release if critical regulatory or data-integrity features are uncovered — show you know when to escalate.

Q67: The team wants to migrate from monolith to microservices. How do you adapt the test strategy?

What's being evaluated: Architectural awareness, ability to evolve testing practices, strategic thinking about long-term quality.

Answer guidance:

Phase 1: Before migration (testing the monolith): - Establish a comprehensive regression test suite for the monolith — this becomes your safety net. - Write characterization tests that document current behavior. - Identify integration points that will become service boundaries.

Phase 2: During migration (strangler fig pattern): - **Contract tests** — As each service is extracted, implement contract tests (Pact) between the new service and remaining monolith. - **Parallel running** — Run both old and new implementations simultaneously. Compare outputs for consistency. - **Feature flags** — Toggle between monolith and microservice implementations. Test both paths. - **Integration tests** — Focus on service boundaries. Does the extracted service behave identically to the monolith component it replaced?

Phase 3: After migration (microservices architecture): - **Per-service testing** — Each service has its own unit and integration tests. - **Contract testing** — Mandatory for all service-to-service interactions. - **End-to-end tests** — Minimal, covering only critical business flows. - **Chaos testing** — Test resilience: service failures, network partitions, latency injection. - **Observability** — Verify distributed tracing, correlated logging, and monitoring.

Key changes in testing approach:

Aspect	Monolith	Microservices
Unit tests	Test within a single codebase.	Test within each service independently.
Integration tests	Database and internal modules.	Service-to-service communication, message queues.
Deployment testing	One deployment to test.	Multiple independent deployments — test compatibility.
Data testing	Single database.	Distributed data — eventual consistency, data duplication.
Failure modes	Process crash, memory leak.	Network failures, partial outages, cascading failures.

Tips: - Emphasize that testing strategy must evolve with the architecture: "You can't test microservices the same way you test a monolith." - Contract tests become the most important investment — highlight this. - Mention the risk of a "distributed monolith" — tightly coupled microservices that are worse than the original monolith.

Q68: You're asked to test a system with no documentation. What do you do?

What's being evaluated: Resourcefulness, exploratory testing skills, ability to work with ambiguity.

Answer guidance:

Step-by-step approach:

1. **Gather context from people:** - Interview developers — ask about architecture, critical flows, known issues, and recent changes. - Talk to operations — what breaks most? What does monitoring show? - Talk to business users — what are the critical workflows? What matters most? - Review support tickets and incident history for patterns.
2. **Explore the system:** - **Exploratory testing** — Use the system as a user would. Map out features, workflows, and data flows. - **Code review** — Read the source code to understand business logic, edge cases, and error handling. - **API exploration** — Use tools like Swagger UI, Postman, or curl to discover and test APIs. - **Database inspection** — Review schema, relationships, and stored procedures to understand data models.
3. **Create documentation as you go:** - Map out system architecture and data flows. - Document discovered features and their expected behavior. - This becomes both test documentation and system documentation.
4. **Build tests incrementally:** - Start with characterization tests — capture current behavior as the baseline. - Add smoke tests for discovered critical paths. - Use exploratory testing sessions with session-based test management (SBTM) to systematically cover the system.
5. **Validate your understanding:** - Share your documentation with developers and business users: "Is this how it works?" - Correct misunderstandings early before they become incorrect tests.

Tips: - Frame this as an opportunity, not a problem: "An undocumented system is a chance to add significant value — the documentation I create as part of testing becomes a lasting asset for the team." - Mention that you'd use tools to accelerate understanding: log analysis, network traffic capture, database query logs. - Show that you don't wait for perfect information to start testing: "I begin exploratory testing on day one. Formal test cases follow as my understanding deepens."

Q69: How would you test a disaster recovery failover for a trading system?

What's being evaluated: Understanding of high-availability requirements in financial systems, ability to test critical infrastructure scenarios.

Answer guidance:

Why DR testing matters in trading systems: - Trading systems must meet strict uptime SLAs (often 99.99%+). - An outage during market hours means lost trades, regulatory scrutiny, and reputational damage. - Regulators (e.g., SEC, FCA) may require evidence of DR testing.

DR test plan:

1. **Define recovery objectives:** - **RTO (Recovery Time Objective)** — How quickly must the system recover? (e.g., < 30 seconds for trading systems) - **RPO (Recovery Point Objective)** — How much data loss is acceptable? (e.g., zero for trade data)
2. **Test scenarios:** - **Primary data center failure** — Simulate complete DC outage. Verify automatic failover to DR site. - **Database failover** — Primary DB goes down. Verify replica promotion and data integrity. - **Network partition** — Connectivity between primary and DR site is lost. Verify split-brain prevention. - **Partial failure** — Single service/component failure. Verify circuit breakers and fallback behavior. - **Failback** — After recovery, switch back to the primary site. Verify no data loss or duplication.
3. **Validation criteria:** - **Recovery time** — Measure actual RTO against the target. - **Data integrity** — Verify no trades were lost, duplicated, or corrupted during failover. - **Functionality** — Run smoke tests against the DR system to confirm all critical functions work. - **Performance** — Verify the DR system can handle production load (not just a "warm" backup). - **Monitoring** — Verify that alerts fire correctly during the failover and that the operations team is notified.

4. **Execution approach:** - Schedule DR tests during non-market hours initially (weekends/holidays). - Progress to "game day" exercises during market hours (with proper risk controls). - Involve operations, development, and business teams. - Document everything — results, issues found, timing measurements.
5. **Post-test activities:** - Review results against RTO/RPO targets. - Document any issues and track remediation. - Update runbooks based on lessons learned.

Tips: - Show awareness of the business impact: "A failed DR failover during a real outage is catastrophic for a trading system. That's why we test it regularly, not just once." - Mention regulatory requirements: "Regulators expect financial institutions to demonstrate DR capability. Test results may be reviewed during audits." - Relate to LSEG: "For a market infrastructure provider like LSEG, DR is existential — the exchange going down affects entire markets."

Q70: A critical automated test has been disabled for months because "nobody has time to fix it." How do you handle it?

What's being evaluated: Ownership mentality, ability to address systemic issues, practical prioritization.

Answer guidance:

Step 1: Assess the situation - What does the disabled test cover? What's the risk of not having it? - Why was it disabled? What's the root cause — flaky test, environment issue, or actual application bug? - Is there any manual testing covering the same scenario? If not, it's an unguarded risk.

Step 2: Quantify the risk - "This test validates trade settlement calculations. Without it, a regression in settlement could reach production undetected." - Frame in business terms: cost of a potential production incident vs. cost of fixing the test.

Step 3: Make a case for fixing it - Don't ask for permission — if it's within your scope, just fix it. - If it requires broader effort, present the risk to your manager with a clear recommendation. - Propose a time-boxed investigation: "Give me 4 hours to diagnose and fix this. If it's bigger than that, I'll come back with an estimate."

Step 4: Fix or retire - **Fix** — Investigate the root cause, fix it, verify the test is stable (run 50+ times), re-enable it. - **Refactor** — If the test is poorly written or testing the wrong thing, rewrite it properly. - **Retire** — If the test is genuinely obsolete (feature was removed, test is redundant), delete it. A disabled test that nobody needs is worse than no test — it creates the illusion of coverage.

Step 5: Prevent recurrence - Establish a team policy: disabled tests must have an associated ticket with an owner and a deadline. - Track disabled tests as a metric — report on the count in sprint reviews. - Add a CI check: if a test has been disabled for > 2 weeks, flag it automatically.

Tips: - Show ownership: "I wouldn't wait to be asked. If I notice a disabled test covering a critical area, I'd investigate it proactively." - Address the cultural issue: "A culture where tests stay disabled for months indicates that test maintenance isn't prioritized. Part of the fix is changing the team's attitude toward test health." - Relate to the LSEG value of **Excellence**: "Accepting disabled tests as normal is accepting mediocrity. Excellence means maintaining the tools we depend on."

Personal & Career

> ***Why this section matters:*** *These are the questions that set the tone for the entire interview. Your opening narrative, career gap explanation, and "Why LSEG" answer will be evaluated within the first 10 minutes. Nail these and the rest of the interview flows from a position of strength.*

Q71: "Tell me about yourself."

What's being evaluated: Communication skills, self-awareness, ability to frame your professional identity concisely. Both interviewers will form a first impression from this answer.

Answer guidance:

Aim for 60–90 seconds. Follow the **Present** → **Past** → **Future** arc:

> *"I'm a quality engineering professional who combines hands-on test automation expertise with strategic thinking. I've spent three and a half years building and leading test automation across complex enterprise platforms — SAP, Salesforce, and E-Commerce systems at Sysco LABS — where I designed Selenium and REST Assured frameworks, integrated them into CI/CD pipelines, and mentored junior engineers.*
> *After that intensive period, I made a deliberate investment in my education — completing a First Class MSc in Cloud Native Computing at TUS in Ireland and earning my ISTQB Advanced Level Technical Test Analyst certification. I also hold an MBA, which gives me a business perspective on quality — not just 'did the test pass,' but 'does this quality strategy serve the business.'*
> *Now I'm looking to bring that deeper foundation back to a challenging technical environment, and LSEG represents exactly that — global financial infrastructure where quality engineering has the highest stakes and greatest impact."*

Tips: - Practice this until it sounds natural, not rehearsed. Record yourself and listen back. - Tailor emphasis per interviewer: with Amy, lean into the strategic/MBA/cloud-native angle; with Andy, emphasise the hands-on Selenium/REST Assured/CI/CD specifics. - Do NOT list tools — that's a laundry list, not a narrative. Mention 2–3 tools max, embedded in context. - Keep under 90 seconds. If they want more detail, they'll ask.

Q72: "Walk us through the gap since October 2023."

What's being evaluated: Honesty, self-awareness, evidence of continued professional development. This is the single biggest risk area — how you handle it determines whether the interview continues on a positive trajectory.

Answer guidance:

Frame the gap as a **deliberate investment**, not an absence. Use the three-part structure: **what you did** → **why you did it** → **what it gave you**.

> *"After three and a half years of intensive QA work at Sysco LABS, I made a deliberate decision to invest in my education. I moved to Ireland to complete my MSc in Cloud Native Computing at TUS, which I finished with First Class Honours in October 2024. This wasn't a step away from QA — it was a step deeper. The MSc covered Docker, Kubernetes, microservices architectures, and cloud-native design patterns — all directly relevant to how modern test infrastructure is built and deployed.*
> *During my time in Ireland, I also served as a Global Student Experience Assistant, which developed my cross-cultural communication skills — relevant for working across LSEG's global offices. I then completed my ISTQB Advanced Level Technical Test Analyst certification in April 2025, which deepened my expertise in test design techniques, white-box testing, and coverage analysis beyond what my work experience had covered.*
> *So while the timeline shows a gap in employment, the reality is continuous professional development — I returned with stronger technical foundations and broader perspectives than when I stepped away."*

What they're really evaluating (the subtext):

Their concern	Your counter
Are your skills current?	MSc in cloud-native computing (2024), ISTQB Advanced (2025)
Are you committed to QA?	Advanced certification + "Gen AI driven QA" as stated interest
Can you be productive quickly?	Cloud-native + advanced testing knowledge = faster ramp-up

Tips: - Be matter-of-fact, not defensive. If you sound apologetic, it signals you think it's a problem too. - Never say "I was looking for the right opportunity" — this sounds passive. - Don't pretend the Student Experience Assistant role was technical — it was a cross-cultural communication role. Own it for what it is. - If they push further, mention specific coursework or projects from the MSc that relate to SDET work.

Q73: "Why LSEG? Why this role?"

What's being evaluated: Research depth, genuine motivation, understanding of the company and role beyond the job description.

Answer guidance:

Structure around three pillars: **Scale & Impact** → **Technical Challenge** → **Growth & Culture**.

> *"Three things draw me to LSEG specifically.*
> *First, **scale and impact**. LSEG operates critical global financial infrastructure — the quality bar isn't just 'no bugs,' it's 'billions of dollars of transactions depend on this working perfectly.' That level of stakes drives the kind of rigorous quality engineering I want to practice.*
> *Second, **technical challenge**. Testing real-time data delivery systems like LSEG Real-Time, across multiple platforms and regulatory environments, is a fundamentally different challenge from testing web applications. I want to grow into that complexity — and my MSc in Cloud Native Computing gives me the architectural foundation to contribute meaningfully to those systems.*
> *Third, **growth and culture**. LSEG's investment in AI-driven quality engineering, the global presence, and the engineering culture offer the kind of environment where I can both contribute and continue developing as a quality leader. I'm particularly drawn to the values — Integrity, Partnership, Excellence, and Change — because they align with how I approach quality: with rigour, collaboration, high standards, and a willingness to evolve."*

Tips: - Reference LSEG products by name: "LSEG Real-Time," "Workspace," "FTSE Russell indexes." This proves you've done your research. - Connect your CA background: "My Chartered Accountancy studies give me a foundation in financial data that most SDET candidates lack." - Show awareness of the Microsoft partnership and cloud migration efforts if you can reference recent news. - Avoid generic answers like "It's a great company" — be specific about what excites you.

Q74: "Where do you see yourself in 3–5 years?"

What's being evaluated: Ambition balanced with realism, commitment to the role and company, leadership trajectory (important for an "Associate Lead" title).

Answer guidance:

> *"In 3 years, I see myself as a senior quality engineering practitioner — someone who shapes QA strategy for a product area, not just individual test suites. I'd like to be leading a QA stream, driving test automation architecture decisions, and mentoring the next generation of SDETs.*
> *In 5 years, I'd like to be influencing quality practices at an organisational level — perhaps driving initiatives like AI-augmented testing strategies or building quality metrics that connect directly to business outcomes. LSEG's scale and the complexity of financial markets infrastructure provides the right environment to develop those capabilities.*

> *The combination of my MBA, MSc, and hands-on QA experience gives me the foundation for that trajectory — but I know the financial domain expertise and the deeper platform knowledge come from doing the work, which is why this role is the right next step."*

Tips: - Balance ambition with humility — show you want to grow but recognise you have learning to do. - Align your trajectory with LSEG's structure — don't describe a path that doesn't exist at the company. - For Andy: emphasise the hands-on technical growth path (deeper framework design, cross-platform testing at scale). - For Amy: emphasise the strategic leadership path (QA strategy, AI adoption, quality metrics).

Q75: "Why are you leaving your current role? / What have you been doing recently?"

What's being evaluated: Motivation, honesty, whether you're running from something or toward something.

Answer guidance:

This is a softer variant of Q72 (the gap question). The key difference is tone — this question is about motivation, not explanation.

> *"I'm not leaving a role — I'm completing a deliberate investment in my professional development. After my MSc and ISTQB Advanced certification, I'm now ready to return to a challenging technical environment where I can apply my deeper foundations.*
> *I chose to take time for education rather than staying in a comfort zone, because I saw the industry moving toward cloud-native architectures and AI-driven quality practices — and I wanted to be prepared for that shift, not catching up after it happened. Now I'm looking for the right environment to bring that investment to bear, and LSEG's combination of technical complexity, financial domain impact, and quality engineering maturity makes it exactly the right place."*

Tips: - Frame as "moving toward LSEG" not "moving away from a gap." - If they mention the Student Experience Assistant role, be honest: "It supported me financially during my time in Ireland. My professional focus during that period was my MSc and certification studies." - Keep this answer short — 60 seconds max. They'll probe further if needed.

Q76: Questions to ask the interviewers

What's being evaluated: Preparation, genuine curiosity, strategic thinking, whether you understand the role and company at a deeper level.

Answer guidance:

Prepare 3–4 questions per interviewer. Choose based on conversation flow — don't robotically ask all of them.

Questions for Amy Lee (Director, Quality Engineering)

#	Question	Why it works
1	"Your LinkedIn mentions AI-driven test automation — could you tell me more about LSEG's vision for AI in quality engineering? Where are you on that journey?"	Shows you've researched her. Opens her passion topic. Demonstrates forward-thinking mindset.
2	"How does the quality engineering team at LSEG measure success? What metrics matter most?"	Strategic question that shows you think beyond test execution.
3	"What does the career progression path look like for an SDET at LSEG? Where do you see this role leading in 2–3 years?"	Shows long-term commitment. Helps you understand actual role scope.
4	"How does the QE team collaborate with development teams? Is it embedded, centralised, or a hybrid model?"	Shows you understand different QA org models.

#	Question	Why it works
5	"What's the biggest quality challenge your team is currently facing?"	Problem-solving orientation. Gives you valuable intel.

Questions for Andy Purtell (QE Manager — likely direct manager)

#	Question	Why it works
1	"What does a typical day or sprint look like for an SDET on your team?"	Practical. Shows you care about day-to-day reality, not just the title.
2	"What tech stack does the team currently use for test automation? What are you considering adopting?"	Fills a critical info gap (JD doesn't specify). Shows practical readiness.
3	"You've been at LSEG and its predecessor organisations for a significant time — what keeps you here? What keeps the work interesting?"	Builds rapport. Gives genuine cultural insight.
4	"How large is the QA team I'd be joining? What's the team structure?"	Practical information. Shows you're thinking about how you fit in.
5	"What would success look like in the first 90 days for someone in this role?"	Shows you're already thinking about delivering value quickly.
6	"The role mentions supporting production releases and internal customers — how does that work in practice?"	Shows you read the JD carefully. Prepares you for full role scope.

Tips: - Pick 3–4 per interviewer based on conversation flow. Don't ask all of them. - Don't ask about salary, benefits, or time off in the first interview. - Don't ask questions easily answered by the JD or LSEG's website. - Listen to their answers — ask follow-ups. The best interview questions lead to genuine conversation. - If time is short, prioritise Q1 for Amy (AI vision) and Q5 for Andy (first 90 days).

Q77: Tailoring your communication style to each interviewer

What's being evaluated: Emotional intelligence, adaptability, ability to communicate effectively with different stakeholders — a critical skill for an Associate Lead who works across teams.

Answer guidance:

This isn't a question they'll ask directly. It's a meta-strategy that should inform every answer you give.

Amy Lee — Director, Quality Engineering

Dimension	Approach
Level of detail	Strategic. Lead with outcomes and business impact, not implementation details.
Language	"Quality strategy," "business value," "organisational capability," "risk reduction."
What to emphasise	MSc + MBA combination, leadership trajectory, AI/modern QA interest, global team experience.
What to avoid	Tool lists, overly tactical answers, "I tested X" without connecting to business impact.
Her background	Unix admin → Server management → QA leadership. Deep technical roots but now operates strategically.
Key topics	AI-driven test automation, quality metrics, team scaling, process maturity, career vision.

Example adaptation for Amy:

Instead of: *"I used Selenium with TestNG and Page Object Model to automate 200 test cases."*

Say: *"I designed an automation strategy that reduced regression cycles from 5 days to 2 days, which directly enabled the team to release more frequently. The architecture used a layered approach — page objects for maintainability, data-driven patterns for coverage breadth — which I documented so the team could scale it independently."*

Andy Purtell — QE Manager (likely direct manager)

Dimension	Approach
Level of detail	Specific. He wants to hear about real problems solved, real frameworks built, real bugs caught.
Language	"I built," "the specific challenge was," "the root cause turned out to be," "here's what I'd do differently."
What to emphasise	Hands-on coding, Selenium/REST Assured specifics, CI/CD integration details, debugging approach, mentoring specifics.
What to avoid	Vague or buzzword-heavy answers. He can detect superficial responses.
His background	QA Engineer → Senior QA → Tech Lead → QE Manager. Deep Elektron/real-time data experience.
Key topics	Framework architecture, flaky tests, cross-platform testing, test data management, team onboarding, production support.

Example adaptation for Andy:

Instead of: *"I have experience with CI/CD integration."*

Say: *"At Sysco LABS, I set up Jenkins pipelines with parameterised builds — the commit stage ran smoke tests in under 5 minutes, and the full regression suite triggered nightly on the staging environment. We used SonarQube quality gates to block merges with code quality violations. One specific challenge was managing test data across environments — I solved that by building API-based data factories that created and cleaned up test data per suite execution."*

Tips: - If both interviewers are in the room simultaneously, default to Andy's practical style but elevate key points to Amy's strategic level. - Watch for cues — if Amy asks a follow-up, she wants more strategic depth. If Andy asks, he wants more technical detail. - The dynamic between them: Amy screens for "Will this person grow into a quality leader?" Andy screens for "Can this person actually do the work?" You need to pass both filters.

AI & Modern QA

> ***Why this section matters:*** *Amy Lee's current focus is AI-driven test automation — this is her passion topic and likely evaluation lens. Demonstrating knowledge here will resonate strongly with her. Your CV lists "Gen AI driven QA and test automation" as an interest — be prepared to back that up with substance.*

Q78: How is AI changing the SDET role? What's your view on AI in test automation?

What's being evaluated: Awareness of industry trends, ability to think critically about emerging technology (not just hype), strategic thinking about the future of quality engineering.

Answer guidance:

Structure your answer around **three layers of AI impact**:

1. AI as an accelerator (today — already practical): - **Code generation assistance** — GitHub Copilot, Amazon CodeWhisperer generate boilerplate test code, assertion patterns, and data factories. Saves time on repetitive scaffolding. - **Test case generation from requirements** — Tools like CodiumAI (now Qodo) analyse source code and generate unit/integration tests with edge cases humans might miss. - **Log analysis and failure triage** — AI models identify patterns in test failure logs, cluster similar failures, and suggest root causes. Reduces time spent manually triaging CI failures.

2. AI as a quality multiplier (emerging — early adoption): - **Intelligent test selection** — ML models predict which tests are most likely to fail based on code changes, reducing suite execution time by 60–80% while maintaining defect detection. - **Self-healing test frameworks** — AI automatically adapts element locators and test steps when the application UI changes, dramatically reducing maintenance overhead. - **Visual AI testing** — Computer vision detects visual regressions that pixel-comparison misses — layout shifts, font changes, colour inconsistencies across browsers.

3. AI as a paradigm shift (future — strategic): - **Autonomous test design** — AI that understands application behaviour and designs test strategies, not just individual test cases. - **Predictive quality** — ML models that predict defect-prone areas before testing begins, based on code complexity, change velocity, and historical defect patterns. - **Shift from "test more" to "test smarter"** — The SDET role evolves from writing test scripts to curating AI-generated test strategies, building quality feedback loops, and designing the systems that AI uses to learn.

Financial domain connection:

> *"In financial systems, AI in testing is particularly valuable because the combinatorial explosion of test scenarios — instruments × markets × currencies × regulatory rules — makes exhaustive manual test design impossible. AI-driven test generation and intelligent selection can cover scenarios that humans would never enumerate."*

Tips: - Don't be purely theoretical — mention specific tools by name (see table below). - Acknowledge limitations: AI-generated tests still need human review for business logic correctness. AI hallucinates test scenarios that don't match real requirements. - For Amy: position yourself as an eager adopter, not a skeptic. Show you've been following this space and are ready to contribute to LSEG's AI-driven QA initiatives. - For Andy: ground it in practical value — less maintenance, faster feedback, better coverage.

Q79: What is intelligent test selection? How does ML-based test prioritisation work?

What's being evaluated: Depth of understanding of a specific AI/ML application in testing, ability to explain technical concepts clearly.

Answer guidance:

The problem: Running the full regression suite on every code change is slow and wasteful. In a large system (like LSEG's 500+ products), full regression might take hours or days. Most of those tests aren't relevant to the specific change.

The solution — intelligent test selection:

ML models learn the relationship between code changes and test outcomes. When new code is committed, the model predicts which tests are most likely to fail and runs only those first.

How it works:

1. **Training data:** Historical records of code changes (which files changed, which functions were modified) paired with test results (which tests passed/failed).
2. **Feature engineering:** Code change features (files changed, lines modified, author, time of day, component) + test features (test history, flakiness score, last failure date).
3. **Model:** Typically gradient-boosted trees or neural networks that predict failure probability for each test given the current change.
4. **Selection:** Tests ranked by predicted failure probability. Run the top N% first (e.g., top 20% likely covers 95% of actual failures).
5. **Feedback loop:** Actual results feed back into the model, continuously improving predictions.

Key tools:

Tool	Approach	Notable Feature
Launchable	ML-based test selection as a service	Integrates with JUnit, pytest, TestNG; claims 80% suite reduction
Predictive Test Selection (Azure DevOps)	Built into Azure Pipelines	Analyses code changes to select relevant tests
Codecov Test Analytics	Coverage-based impact analysis	Maps code changes to tests via coverage data
Custom/in-house	Build your own using test history + code change data	Full control, requires ML expertise

Limitations: - Cold start problem — needs historical data to be effective. - Can miss tests affected by indirect dependencies (e.g., config changes, shared libraries). - Must still run the full suite periodically (nightly/weekly) to catch missed regressions.

Tips: - Connect to LSEG's scale: "With 500+ products across multiple platforms, intelligent test selection could dramatically reduce CI pipeline time while maintaining confidence in releases." - Mention that this complements, not replaces, a solid test architecture — you still need well-designed, independent, well-maintained tests.

Q80: What are self-healing test frameworks? How do they work?

What's being evaluated: Understanding of a practical AI application that solves a real SDET pain point — test maintenance.

Answer guidance:

The problem: UI test automation is fragile. When developers change element IDs, class names, or page structure, Selenium locators break. In large suites, locator maintenance consumes 30–40% of SDET effort.

The solution — self-healing:

Self-healing frameworks use AI to automatically find and update element locators when the original locator fails, without human intervention.

How it works:

1. **Multi-locator strategy:** Each element is identified by multiple attributes (ID, CSS, XPath, text, position, visual appearance) during test creation.
2. **Failure detection:** When the primary locator fails, the framework doesn't immediately fail the test.
3. **Alternative search:** AI algorithms try alternative locators based on: - Attribute similarity (e.g., ID changed from `btn-submit` to `submit-btn`). - DOM structure (element is still the 3rd button in the form). - Visual position (element is still in the same screen location). - Text content (button still says "Submit").
4. **Healing:** If an alternative locator finds the element, the test continues. The framework logs the healed locator for review.

5. **Learning:** Over time, the framework learns which locator strategies are most stable for each element type.

Key tools:

Tool	Type	How It Works
Healenium	Open-source	Selenium extension; uses ML to find elements when locators break. Stores locator history in a database.
Testim	Commercial	AI-powered locators that adapt to UI changes. Learns from test execution history.
mabl	Commercial SaaS	Auto-healing with visual + DOM analysis. Includes AI-driven test creation.
Selenium IDE (newer versions)	Open-source	Basic fallback locator strategies built in.

Practical considerations: - Self-healing is valuable for **reducing false negatives** (tests failing due to locator changes, not real bugs). - It should NOT silently heal — changes must be reviewed. A healed locator might indicate a legitimate UI change that needs new test assertions. - Works best for stable applications with incremental UI changes. Major redesigns still require manual test updates.

Connection to Udani's experience:

> *"In my Salesforce Lightning testing experience, dynamic element IDs were one of the biggest challenges. Self-healing frameworks would have reduced the locator maintenance burden significantly — Salesforce generates dynamic IDs that change across deployments, so traditional locator strategies break frequently. I used JavaScript executor and custom wait strategies as a manual equivalent, but AI-driven self-healing would be a step-change improvement."*

Tips: - For Andy: frame this as a practical solution to the maintenance challenge he faces with 500+ products. - For Amy: frame this as a strategic investment that frees SDET capacity from maintenance toward higher-value test design work.

Q81: How can generative AI (Copilot, CodiumAI) be used in day-to-day SDET work?

What's being evaluated: Practical awareness of GenAI tools, ability to distinguish hype from genuine productivity gains, judgment about when AI helps vs. when it gets in the way.

Answer guidance:

Practical use cases for GenAI in SDET work:

Use Case	Tool(s)	What It Does	Limitation
Boilerplate generation	GitHub Copilot, Amazon CodeWhisperer	Generates test method scaffolding, assertion patterns, page object structures	May not follow project conventions; needs review
Test case generation from code	Qodo (formerly CodiumAI), Diffblue Cover	Analyses source code and generates unit/integration tests with edge cases	Generated tests may not reflect real business requirements
Test data generation	ChatGPT, Copilot	Creates realistic test data sets, boundary values, edge case inputs	Financial data requires domain-specific validity (e.g., valid ISINs, correct currency codes)
Regex and XPath generation	Copilot, ChatGPT	Generates complex locators, validation patterns, SQL queries from natural language	Must verify correctness — AI-generated regex can have subtle errors

Use Case	Tool(s)	What It Does	Limitation
Debugging assistance	Copilot Chat, Claude	Analyses error logs, suggests root causes, explains stack traces	Lacks project-specific context; suggestions are starting points
Documentation	Copilot, ChatGPT	Generates test plan templates, test case descriptions, framework documentation	May produce generic content that doesn't match project specifics
Code review assistance	Qodo, SonarQube AI	Reviews test code for quality, suggests improvements, identifies anti-patterns	Cannot assess business logic correctness

A realistic perspective:

> *"I see GenAI as a productivity multiplier, not a replacement for SDET judgment. It excels at generating boilerplate, suggesting edge cases I might miss, and accelerating routine tasks. But it doesn't understand business context — it can generate a test for a function, but it can't tell you whether that function's behaviour is correct for the business requirement. The SDET's role shifts from writing every line of test code to curating, reviewing, and directing AI-generated output."*

What NOT to say: - Don't claim AI will replace testers — this sounds naive. - Don't dismiss AI as "just hype" — this sounds resistant to change, especially to Amy. - Don't claim extensive hands-on experience if you haven't used these tools — say you've explored them and are ready to integrate them into your workflow.

Tips: - If asked whether you've used these tools, be honest: "I've used Copilot for test code generation and explored CodiumAI for automated test generation. I'm actively building my fluency with these tools because I believe they'll be fundamental to SDET productivity in the next 2–3 years." - For Amy: connect to the strategic shift — "AI doesn't reduce the need for quality engineers; it elevates the role from test execution to test strategy and AI oversight."

Q82: What is visual AI testing? How does it differ from traditional visual regression testing?

What's being evaluated: Understanding of a specific AI-driven testing technique, ability to compare approaches and articulate trade-offs.

Answer guidance:

Traditional visual regression testing: - Takes screenshot of baseline (expected) and compares pixel-by-pixel to current screenshot. - Any pixel difference = failure. This creates massive false positive rates: - Font rendering differences across OS/browser. - Anti-aliasing variations. - Dynamic content (timestamps, ads, user-specific data). - Sub-pixel rendering differences. - Requires constant baseline updates and manual triage.

Visual AI testing: - Uses computer vision and ML models trained on how humans perceive visual differences. - Understands page structure, layout, and visual hierarchy — not just pixels. - Can distinguish meaningful visual changes (broken layout, wrong colour, missing element) from noise (font rendering, sub-pixel differences).

Comparison:

Aspect	Traditional (Pixel Comparison)	Visual AI
False positive rate	High (20–40% of flagged changes are noise)	Low (AI filters noise like anti-aliasing)
Cross-browser accuracy	Poor (rendering differences flagged as bugs)	Good (understands acceptable rendering variance)
Dynamic content handling	Manual ignore regions required	AI can identify and handle dynamic areas automatically
Layout validation	Limited to pixel match	Understands layout structure, spacing, alignment

Aspect	Traditional (Pixel Comparison)	Visual AI
Maintenance	High (baseline updates, ignore regions)	Lower (AI learns acceptable variance)
Cost	Free/open-source tools available	Typically commercial (Applitools: ~$500/user/month)

Key tools:

Tool	Approach	Best For
Applitools Eyes	Visual AI with Ultrafast Grid for cross-browser	Enterprise visual testing at scale
Percy (BrowserStack)	Snapshot comparison with smart diff	CI-integrated visual regression
Chromatic	Visual testing for Storybook components	Component-level visual testing
BackstopJS	Open-source screenshot comparison	Budget-conscious teams

Financial domain connection:

> *"In financial applications, visual accuracy matters — a misaligned decimal point in a trading dashboard or an incorrectly rendered chart could lead to wrong trading decisions. Visual AI testing could validate that financial data is displayed correctly across different screen sizes, browsers, and user contexts without the false positive noise that makes pixel comparison impractical at scale."*

Tips: - If you haven't used visual AI testing, be honest but informed: "I haven't implemented visual AI testing in a production environment, but I understand the technology and its advantages over pixel comparison. In my Salesforce testing, we relied on manual visual verification supplemented by basic screenshot comparison — visual AI would have significantly improved that workflow." - For Amy: connect to the broader AI-in-QA strategy. Visual AI is one piece of the puzzle alongside intelligent test selection and self-healing frameworks. - For Andy: frame in terms of practical value — reduced maintenance, fewer false positives, faster triage.

Security Testing

> ***Why this section matters:*** *The job description mentions security testing awareness, and financial systems demand it. Udani lists Veracode on her CV, and financial data handling raises the security stakes significantly. Both interviewers will appreciate security consciousness — it signals maturity as a quality engineer.*

Q83: What is the OWASP Top 10 and how does it influence your testing?

What's being evaluated: Awareness of common security vulnerabilities, ability to translate security knowledge into practical testing activities.

Answer guidance:

OWASP (Open Worldwide Application Security Project) Top 10 is the most widely recognised list of critical web application security risks, updated periodically based on real-world data.

Current OWASP Top 10 (2021 edition) with testing implications:

#	Vulnerability	What It Is	How to Test
A01	**Broken Access Control**	Users can act outside their intended permissions.	Test role-based access: can a viewer modify data? Can user A access user B's records? Test direct URL manipulation, API endpoint access without auth.
A02	**Cryptographic Failures**	Sensitive data exposed due to weak or missing encryption.	Verify HTTPS everywhere, check for sensitive data in URLs/logs, validate encryption of data at rest, check password hashing algorithms.
A03	**Injection**	Untrusted data sent to an interpreter (SQL, OS command, LDAP).	Send SQL injection payloads (`' OR 1=1 --`), test input fields with special characters, verify parameterised queries are used.
A04	**Insecure Design**	Flaws in the design/ architecture, not just implementation.	Review business logic for abuse cases (e.g., can a user bypass payment by manipulating order flow?). Threat modelling during design phase.
A05	**Security Misconfiguration**	Default configurations, open cloud storage, verbose error messages.	Check default credentials, verify error pages don't leak stack traces, scan for unnecessary open ports, review HTTP security headers.
A06	**Vulnerable Components**	Using libraries/frameworks with known vulnerabilities.	Scan dependencies with SCA tools (Snyk, OWASP Dependency-Check), monitor CVE databases, keep dependencies updated.
A07	**Authentication Failures**	Broken authentication, weak passwords, session management issues.	Test brute force protection, session timeout, token invalidation on logout, password complexity rules, MFA enforcement.

#	Vulnerability	What It Is	How to Test
A08	**Data Integrity Failures**	Code and infrastructure without integrity verification (insecure CI/CD, unsigned updates).	Verify CI/CD pipeline security, check for unsigned packages, validate data integrity checksums.
A09	**Logging & Monitoring Failures**	Insufficient logging to detect breaches.	Verify security events are logged (failed logins, access denials), test that logs don't contain sensitive data, validate alerting thresholds.
A10	**Server-Side Request Forgery (SSRF)**	Application fetches remote resources without validating user-supplied URLs.	Test URL input fields with internal addresses (`localhost` , `169.254.169.254` for cloud metadata), verify allowlists are enforced.

How to use OWASP Top 10 in practice:

1. **Test planning:** Include OWASP Top 10 checks in the test plan for any web-facing application.
2. **Test case design:** Create specific negative test cases for each relevant vulnerability category.
3. **Automation:** Integrate basic security checks into automated regression suites (e.g., verify auth headers, test access control boundaries).
4. **Shift-left:** Include security test scenarios in code reviews and design reviews.

Tips: - You don't need to be a penetration tester — show that you understand common vulnerabilities and include security considerations in your testing approach. - In financial systems, A01 (Broken Access Control) and A02 (Cryptographic Failures) are especially critical — unauthorized access to trading data or leaked financial information has regulatory consequences. - Mention that OWASP also provides testing guides (OWASP Testing Guide) and tools (ZAP) — showing awareness of the broader ecosystem.

Q84: Explain SAST, DAST, and IAST — when would you use each?

What's being evaluated: Understanding of security testing approaches, ability to choose the right tool/approach for the context.

Answer guidance:

Aspect	SAST (Static)	DAST (Dynamic)	IAST (Interactive)
Full name	Static Application Security Testing	Dynamic Application Security Testing	Interactive Application Security Testing
When it runs	During development (analyses source code)	Against running application	During test execution (agent inside app)
What it analyses	Source code, bytecode, or binary	Application behaviour via HTTP requests	Real-time code execution paths
Finds	Code-level vulnerabilities (SQL injection patterns, hardcoded secrets, insecure functions)	Runtime vulnerabilities (XSS, auth bypass, misconfigurations)	Both code-level and runtime vulnerabilities with exact code location
False positive rate	Higher (analyses code paths that may not be reachable)	Lower (tests actual application behaviour)	Lowest (combines static and dynamic analysis)
Speed	Fast (no running application needed)	Slower (requires deployed application)	Medium (runs during test execution)
CI/CD integration	Early in pipeline (commit/build stage)	Later in pipeline (test/staging stage)	During functional test execution

Aspect	SAST (Static)	DAST (Dynamic)	IAST (Interactive)
Language dependent?	Yes (needs parser for each language)	No (language-agnostic, tests via HTTP)	Yes (agent runs inside application runtime)
Example tools	SonarQube, Veracode Static, Checkmarx, Semgrep	OWASP ZAP, Burp Suite, Veracode Dynamic	Contrast Security, Synopsys Seeker

When to use each:

1. **SAST — Use early, use often:** - Integrate into IDE and CI pipeline so developers get feedback on every commit. - Catches insecure coding patterns before code is even built. - Best for: finding hardcoded credentials, injection vulnerabilities, insecure crypto usage.
2. **DAST — Use in testing/staging:** - Run against deployed applications in test environments. - Simulates real attacks without needing source code access. - Best for: finding authentication issues, XSS, CSRF, server misconfigurations.
3. **IAST — Use during functional testing:** - Runs alongside your existing functional tests — no separate security test suite needed. - Provides the most accurate results (lowest false positives) because it sees both code and runtime behaviour. - Best for: comprehensive security validation during QA cycles.

Recommended pipeline integration:

```
Commit → [SAST] → Build → [SCA] → Deploy to Test → [DAST + IAST during functional tests] →
Staging → [Pen Test]
```

Tips: - Connect to your experience: "At Sysco LABS, we used SonarQube for static analysis as part of our quality gates, and Veracode for security scanning. SonarQube caught code quality and basic security issues on every commit, while Veracode provided deeper security analysis." - Emphasise the "shift-left" principle: catching security issues in SAST is 10x cheaper than finding them in DAST, and 100x cheaper than finding them in production. - For financial systems: "In a financial environment, I'd advocate for all three layers — SAST in the IDE and CI pipeline, DAST against staging environments, and periodic penetration testing by specialists."

Q85: How do you integrate security testing into CI/CD pipelines (DevSecOps)?

What's being evaluated: Understanding of DevSecOps principles, ability to embed security into automated workflows without slowing delivery.

Answer guidance:

DevSecOps embeds security testing into every stage of the CI/CD pipeline, making security a continuous activity rather than a gate at the end.

Pipeline integration model:

Pipeline Stage	Security Activity	Tools	Blocking?
Pre-commit	Secret scanning (prevent credential leaks)	git-secrets, TruffleHog, pre-commit hooks	Yes — block commit if secrets detected
Commit/Build	SAST — static code analysis	SonarQube, Veracode, Semgrep	Yes — fail build on critical/high severity
Dependency resolution	SCA — software composition analysis	Snyk, OWASP Dependency-Check, Dependabot	Yes — block if known CVEs in dependencies
Test environment	DAST — dynamic scanning of deployed app	OWASP ZAP, Burp Suite Enterprise	Advisory — alert on findings, don't block
Functional testing	IAST — security analysis during test execution	Contrast Security	Advisory for medium, block for critical
Pre-production		Trivy, Checkov, tfsec	Yes — block deployment with vulnerable images

Pipeline Stage	Security Activity	Tools	Blocking?
	Container scanning, infrastructure-as-code scanning		
Production	Runtime protection, WAF monitoring	RASP tools, cloud WAFs	Monitor and alert

Key principles:

1. **Automate everything possible:** Manual security reviews don't scale. Integrate scanning tools that run automatically on every pipeline execution.
2. **Gate on severity:** Not every finding should block the pipeline. Define severity thresholds — block on critical/high, alert on medium, track low.
3. **Fast feedback:** SAST and SCA run in minutes. DAST may take longer — run it in parallel with functional tests.
4. **Developer ownership:** Security findings should surface in the developer's PR, not in a separate security report weeks later. Shift-left means shift responsibility left too.
5. **Metrics:** Track mean time to remediate (MTTR) for security findings. Target: critical findings fixed before merge; high findings fixed within one sprint.

Tips: - Connect to your CI/CD experience: "At Sysco LABS, I integrated SonarQube quality gates into our Jenkins pipeline — this included basic security checks alongside code quality. I'd extend this approach with dedicated SAST/DAST tools for a financial systems context." - For financial systems: "In a regulated environment like LSEG, security testing in the pipeline isn't optional — it's a compliance requirement. Regulatory bodies expect evidence that security is tested continuously, not just at release boundaries." - Don't position security as "someone else's job" — an SDET who understands DevSecOps is significantly more valuable.

Q86: What security considerations are specific to financial systems?

What's being evaluated: Domain awareness, understanding of the heightened security requirements in financial infrastructure.

Answer guidance:

Financial systems have security requirements that go significantly beyond typical web applications:

1. Data classification and protection:

Data Type	Sensitivity	Protection Required
Market data (prices, trades)	High	Encryption in transit, access controls, entitlement enforcement
Customer PII	Critical	Encryption at rest + in transit, data masking in non-prod, GDPR compliance
Trading credentials / API keys	Critical	Hardware security modules (HSMs), secrets management, rotation policies
Transaction records	High	Tamper-proof audit trails, immutable logging
Regulatory reports	High	Access controls, integrity verification, retention policies

2. Access control and entitlements: - **Granular entitlements:** Users see only the data they're entitled to. A client with delayed data entitlement must not receive real-time data. - **Separation of duties:** The person who creates a trade cannot approve it. Testing must verify these controls. - **Multi-tenancy isolation:** One client's data must never leak to another client's view.

3. Regulatory requirements: - **MiFID II** (EU): Transaction reporting, best execution, audit trails, timestamp precision. - **SOX** (US): Internal controls, financial reporting accuracy, change management controls. - **PCI DSS** (if

handling payment data): Network segmentation, encryption, access logging. - **Cyber resilience regulations:** Financial regulators increasingly require proof of cyber resilience testing (e.g., EU DORA regulation).

4. Specific attack vectors in finance: - **Market manipulation via data injection:** What if someone injects false price data into the feed? - **Insider trading enablement:** Can system access be abused to gain trading advantage? - **DDoS against trading infrastructure:** Testing resilience against availability attacks during market hours. - **API abuse:** Rate limiting, authentication bypass, data scraping via APIs.

5. Testing implications: - Security testing in financial systems must be **continuous, automated, and auditable**. - Test environments must handle data masking — you cannot use real customer data or real trading data in test environments. - Penetration testing must be carefully scheduled to avoid impacting live systems. - All security testing activities must be documented for regulatory audits.

Tips: - Connect to your experience: "While I haven't worked in financial markets specifically, the data integrity and access control challenges are analogous to what I tested in SAP-Salesforce integrations — ensuring the right data reaches the right system with the right permissions." - Show awareness of the business impact: "A security breach at a financial infrastructure company like LSEG doesn't just affect one organisation — it can undermine trust in the entire market." - For Amy: position security testing as part of the overall quality strategy, not a separate activity.

Q87: Tell us about your experience with Veracode.

What's being evaluated: Hands-on experience with a specific security testing tool (listed on your CV), ability to describe practical application in STAR format.

Answer guidance (STAR format):

> ***Situation:*** *"At Sysco LABS, during the Hybris E-Commerce project, we were responsible for ensuring our application met the client's (Brakes UK) security compliance requirements before each production release."*
> ***Task:*** *"As the QA engineer on the team, I was involved in conducting security testing using Veracode to identify vulnerabilities in our codebase and ensure compliance with the client's security standards."*
> ***Action:*** *"I worked with Veracode's static analysis (SAST) scanning capabilities. We integrated Veracode scans into our release process — before each major release, we submitted the application build for scanning. My specific responsibilities included: - Reviewing the scan results and triaging findings by severity (critical, high, medium, low). - Collaborating with developers to understand and remediate findings — explaining the vulnerability, its potential impact, and suggesting fixes. - Verifying that remediated issues were resolved in subsequent scans. - Documenting the security testing outcomes in the release report for stakeholder sign-off.*
> *One specific example: Veracode flagged a potential SQL injection vulnerability in a search function that used string concatenation instead of parameterised queries. I worked with the developer to refactor the code to use prepared statements, verified the fix resolved the Veracode finding, and added a targeted test case to our automation suite to prevent regression."*
> ***Result:*** *"Our application consistently passed Veracode security scans with zero critical or high findings at release time. The process also improved the team's security awareness — developers started proactively checking for common vulnerability patterns during code reviews, which reduced the number of findings in each subsequent scan."*

Broader Veracode knowledge to demonstrate:

Veracode Feature	What It Does
Static Analysis (SAST)	Scans source code or compiled binaries for security vulnerabilities
Dynamic Analysis (DAST)	Tests running applications for runtime vulnerabilities
Software Composition Analysis (SCA)	Identifies vulnerabilities in third-party libraries and dependencies
Manual Penetration Testing	Veracode also offers human-led pen testing services
Veracode Fix	AI-assisted remediation suggestions for identified vulnerabilities

Veracode Feature	What It Does
Policy compliance	Define security policies and track compliance across applications

Tips: - Be honest about your depth of experience — don't claim to be a Veracode expert if you primarily reviewed results rather than configured scans. - Connect to the SDET role: "Security scanning is another dimension of quality. As an SDET, I advocate for integrating security checks into the CI/CD pipeline alongside functional quality gates — Veracode's CI integration capabilities support exactly this approach." - For financial systems: "In a financial infrastructure company like LSEG, security scanning isn't a nice-to-have — it's a regulatory requirement. Veracode or similar tools should be part of every pipeline."

General SDET Questions

A comprehensive collection of Software Development Engineer in Test (SDET) interview questions organized by category, with detailed answers.

Table of Contents

1. Testing Fundamentals
2. Test Automation Frameworks
3. Programming & Data Structures
4. API Testing
5. CI/CD & DevOps
6. Performance Testing
7. Security Testing
8. Database Testing
9. Design Patterns & Architecture
10. Behavioral & Process

1. Testing Fundamentals

Q1.1: What is the difference between Verification and Validation?

Verification checks whether the product is being built correctly — it evaluates work products (requirements, design, code) against specifications through reviews, walkthroughs, and inspections. It answers: *"Are we building the product right?"*

Validation checks whether the correct product is being built — it evaluates the final product against user needs and requirements through actual testing (functional, UAT, etc.). It answers: *"Are we building the right product?"*

Aspect	Verification	Validation
Focus	Process	Product
Techniques	Reviews, inspections, walkthroughs	Testing, demos, prototyping
When	During development	After development
Example	Code review confirms logic matches design	UAT confirms feature meets user expectation

Q1.2: Explain the Testing Pyramid and why it matters.

The Testing Pyramid is a strategy that guides the proportion of tests at each level:

```
        /  UI/E2E  \        ← Few, slow, expensive
       /————————————\
      / Integration  \      ← Moderate number
     /————————————————\
    /    Unit Tests    \    ← Many, fast, cheap
   /————————————————————\
```

- **Unit tests (base):** Test individual functions/methods in isolation. They are fast, cheap, and should form ~70% of the test suite.
- **Integration tests (middle):** Test interactions between components, services, or modules. ~20% of the suite.

- **UI/E2E tests (top):** Test complete user workflows through the UI. Slow, brittle, and expensive — ~10% of the suite.

Why it matters: Teams that invert the pyramid (heavy E2E, few unit tests) suffer from slow feedback loops, flaky test suites, and high maintenance costs. The pyramid ensures fast feedback at the base and confidence at the top.

Q1.3: What is the difference between black-box, white-box, and grey-box testing?

- **Black-box testing:** The tester has no knowledge of internal code structure. Tests are based purely on requirements and specifications. Techniques include equivalence partitioning, boundary value analysis, and decision tables.
- **White-box testing:** The tester has full knowledge of the internal code. Tests are designed to exercise specific code paths, branches, and conditions. Techniques include statement coverage, branch coverage, and path coverage.
- **Grey-box testing:** The tester has partial knowledge of internals — typically understands architecture, data flows, or database schemas but doesn't inspect code line-by-line. Common in integration testing and API testing where you understand the system's structure but test through external interfaces.

Q1.4: What are flaky tests and how do you handle them?

A **flaky test** is one that produces inconsistent results (passes and fails) on the same code without any changes. Common causes include:

- **Timing issues:** Race conditions, insufficient waits, async operations not properly handled.
- **Test order dependency:** Tests that rely on state left by previous tests.
- **Environment dependency:** Hardcoded ports, paths, or assumptions about external services.
- **Shared mutable state:** Global variables or shared databases modified across tests.

Handling strategies:

1. **Quarantine:** Move flaky tests into a separate suite that doesn't block CI. Track and fix them within a time window.
2. **Retry with analysis:** Allow a single retry but log every retry. If a test needs retries frequently, it needs fixing.
3. **Root cause fixes:** - Replace `sleep()` with explicit waits/polling. - Isolate test data — each test creates and tears down its own state. - Use dependency injection or mocks to remove external dependencies. - Ensure deterministic ordering or make tests fully independent.
4. **Prevention:** Enforce test isolation in code reviews. Run tests in randomized order during CI.

Q1.5: Explain equivalence partitioning and boundary value analysis with an example.

Consider a field that accepts ages between 18 and 65.

Equivalence Partitioning (EP) divides inputs into groups (partitions) that the system should handle identically:

Partition	Range	Expected Result
Invalid (low)	< 18	Rejected
Valid	18–65	Accepted
Invalid (high)	> 65	Rejected

You pick one representative value from each partition (e.g., 10, 30, 80).

Boundary Value Analysis (BVA) focuses on values at the edges of partitions, where defects are most likely:

- Test values: **17** (just below), **18** (lower boundary), **19** (just above lower), **64** (just below upper), **65** (upper boundary), **66** (just above upper).

BVA is used alongside EP because bugs cluster at boundaries — off-by-one errors, incorrect comparison operators (`<` vs `<=`), and type overflow issues.

Q1.6: What is the difference between regression testing and retesting?

Retesting verifies that a specific defect that was reported has been fixed. You re-run the exact test case that originally found the bug.

Regression testing verifies that new code changes (bug fixes, features, refactors) have not broken existing functionality. You run a broader suite of tests — not just the ones related to the change.

Aspect	Retesting	Regression Testing
Purpose	Confirm a specific fix	Ensure nothing else broke
Scope	Narrow — specific defect	Broad — affected areas and beyond
Automation	Usually manual	Highly suitable for automation
When	After a fix is deployed	After any code change

Q1.7: What is risk-based testing and when would you use it?

Risk-based testing prioritizes test efforts based on the **likelihood** and **impact** of failures. It's used when you cannot test everything — which is almost always.

Process: 1. **Identify risks:** List features, modules, and scenarios. For each, assess the probability of failure and the business impact if it fails. 2. **Prioritize:** High-probability + high-impact items get the most testing effort. Low-probability + low-impact items may get minimal or no testing. 3. **Allocate:** Design more tests, use more techniques, and automate critical paths for high-risk areas. Lower-risk areas may only get smoke testing.

When to use: Time-constrained releases, legacy systems with unknown code quality, safety-critical applications, or when deciding which tests to include in a regression suite.

---

2. Test Automation Frameworks

Q2.1: What is the Page Object Model (POM) and why is it used?

The **Page Object Model** is a design pattern where each web page (or significant component) is represented by a class. The class encapsulates the page's elements and actions, separating test logic from page interaction logic.

```
PageObject (LoginPage)          Test (LoginTest)
├── usernameField               ├── testValidLogin()
├── passwordField               │     loginPage.login("user", "pass")
├── loginButton                 │     assert dashboardPage.isVisible()
├── login(user, pass)           ├── testInvalidLogin()
└── getErrorMessage()           │     loginPage.login("bad", "wrong")
                                │     assert loginPage.getErrorMessage()
```

Benefits: - **Maintainability:** If the UI changes (e.g., a locator changes), you update one class, not dozens of tests. - **Readability:** Tests read like business workflows, not DOM manipulation. - **Reusability:** Multiple tests share the same page object. - **Reduced duplication:** Element locators and interactions are defined once.

Q2.2: How do you decide what to automate and what to keep manual?

Automate when: - Tests are repetitive and run frequently (regression suites). - Tests require precise data comparison (large datasets, pixel comparison). - Tests run across many configurations (cross-browser, cross-device). - Tests are stable — the feature under test rarely changes. - Tests require setup that is tedious manually (seeding databases, creating test users).

Keep manual when: - Exploratory testing — human intuition finds unexpected bugs. - Usability / UX testing — automation can't judge "does this feel right?" - Tests that run rarely (one-off verifications). - Features that are changing rapidly — automation cost outweighs benefit until the feature stabilizes. - Ad hoc testing for newly reported issues.

Framework for deciding: Calculate the break-even point. If `(manual_execution_time × expected_runs) > (automation_development_time + maintenance_cost)`, automate it.

Q2.3: Explain the difference between Selenium, Cypress, and Playwright.

Feature	Selenium	Cypress	Playwright
Language support	Java, Python, C#, JS, Ruby	JavaScript/TypeScript only	JS/TS, Python, Java, C#
Browser support	Chrome, Firefox, Safari, Edge, IE	Chrome, Firefox, Edge, WebKit (limited)	Chromium, Firefox, WebKit
Architecture	Communicates via WebDriver protocol (out-of-process)	Runs inside the browser (in-process)	Uses CDP/browser-specific protocols (out-of-process)
Auto-waiting	Manual waits needed	Built-in auto-waiting	Built-in auto-waiting
Parallel execution	Via Selenium Grid or third-party	Limited native parallelism	Native parallel contexts
Network interception	Limited (proxy-based)	Built-in `cy.intercept()`	Built-in `route()` API
iframes / tabs	Supported but cumbersome	Limited iframe support	First-class multi-tab/iframe support
Speed	Slower due to HTTP-based protocol	Fast (in-browser)	Fast (direct protocol)

When to choose each: - **Selenium:** Legacy projects, need IE support, or team is locked into Java/C#. - **Cypress:** JavaScript-heavy team, single-browser focus, strong developer experience needed. - **Playwright:** Modern projects needing cross-browser coverage, multi-tab/iframe testing, or non-JS language support.

Q2.4: What is a data-driven testing framework?

A **data-driven framework** separates test logic from test data. The same test script runs multiple times with different input data sets, typically stored in external sources (CSV, Excel, JSON, database, or parameterized annotations).

How it works: 1. Test logic is written once as a template. 2. Data is stored externally or in parameterized fixtures. 3. The framework iterates over each data row, injecting values into the test.

Example concept:

```
Test: loginTest(username, password, expectedResult)
Data:
  | "validUser"   | "validPass"   | "success" |
  | "invalidUser" | "validPass"   | "error"   |
  | "validUser"   | "wrongPass"   | "error"   |
  | ""            | ""            | "error"   |
```

Benefits: High coverage with minimal code duplication. Adding new test scenarios means adding data rows, not new test methods.

Tools: TestNG `@DataProvider`, JUnit `@ParameterizedTest`, pytest `@pytest.mark.parametrize`, Cucumber Scenario Outlines.

Q2.5: How do you handle dynamic elements in UI automation?

Dynamic elements change their attributes (IDs, classes, positions) on each page load. Strategies:

1. **Relative locators:** Use stable parent-child relationships rather than absolute paths. - Bad: `//div[3]/span[2]/button` - Good: `//button[contains(text(), 'Submit')]`
2. **Custom data attributes:** Advocate for `data-testid` attributes added by developers specifically for testing. These are stable and semantic. - `[data-testid="submit-button"]`
3. **CSS/XPath with partial matching:** - `[id*='login']` matches `login-btn-12345` - `//div[contains(@class, 'modal')]`
4. **Explicit waits:** Wait for elements to appear/stabilize before interacting. - `WebDriverWait(driver, 10).until(EC.element_to_be_clickable(locator))`
5. **Retry mechanisms:** For elements that update asynchronously, retry the interaction with a short polling interval.
6. **Shadow DOM handling:** Use framework-specific APIs (Playwright's `locator()` pierces shadow DOM by default; Selenium requires `shadowRoot` access).

Q2.6: What is BDD and how does Cucumber fit in?

Behavior-Driven Development (BDD) is a practice where tests are written in natural language (Gherkin syntax) to describe system behavior from the user's perspective. It bridges communication between developers, testers, and business stakeholders.

Gherkin syntax:

```
Feature: User Login
  Scenario: Successful login with valid credentials
    Given the user is on the login page
    When the user enters "admin" and "password123"
    And clicks the login button
    Then the user should be redirected to the dashboard
```

Cucumber is a tool that executes Gherkin scenarios by mapping each step to code (step definitions):

```
@Given("the user is on the login page")
public void navigateToLogin() {
    driver.get("https://app.example.com/login");
}
```

Benefits: Living documentation, shared understanding, readable by non-technical stakeholders.

Pitfalls: Over-engineering step definitions, writing imperative steps instead of declarative ones, maintaining a large Gherkin layer that adds overhead without stakeholder engagement.

Q2.7: How do you manage test environments and test data?

Test environments: - Use **infrastructure-as-code** (Terraform, Docker Compose) to spin up consistent environments. - Maintain environment parity — test environments should mirror production as closely as possible. - Use **containerized services** (Docker) for databases, message queues, and mock services. - Implement environment configuration via environment variables, not hardcoded values.

Test data: - **Factory pattern:** Generate test data programmatically (e.g., `UserFactory.create(role: "admin")`). - **Database seeding:** Load known data sets before test execution; truncate/reset afterward. - **API-based setup:** Use APIs to create test data rather than UI interactions — faster and more reliable. - **Data isolation:** Each test should create its own data and not depend on data from other tests. Use unique identifiers (UUIDs, timestamps) to avoid collisions. - **Sensitive data:** Never use production data in tests. Use synthetic data generators for PII.

---

3. Programming & Data Structures

Q3.1: Explain time complexity. What is Big O notation?

Big O notation describes the upper bound of an algorithm's time (or space) growth rate as input size increases. It focuses on the dominant term and ignores constants.

Notation	Name	Example
O(1)	Constant	HashMap lookup
O(log n)	Logarithmic	Binary search
O(n)	Linear	Iterating an array
O(n log n)	Linearithmic	Merge sort, quicksort (average)
$O(n^2)$	Quadratic	Nested loops (bubble sort)
$O(2^n)$	Exponential	Recursive Fibonacci (naive)

Why SDETs need this: When writing test utilities that process large data sets (log parsers, data generators, report aggregators), poor algorithmic choices can make test suites unacceptably slow. An SDET should be able to evaluate whether a helper function will scale.

Q3.2: What are the SOLID principles?

Principle	Meaning	Testing Relevance
S — Single Responsibility	A class should have one reason to change	Each page object handles one page; each test class covers one feature
O — Open/Closed	Open for extension, closed for modification	Add new test data providers without modifying existing test logic
L — Liskov Substitution	Subtypes should be substitutable for their base types	A `MobileLoginPage` extending `LoginPage` should work wherever `LoginPage` is used
I — Interface Segregation	Clients shouldn't depend on interfaces they don't use	Don't force all page objects to implement a `search()` method if only some pages have search
D — Dependency Inversion	Depend on abstractions, not concretions	Tests depend on a `Browser` interface, not directly on `ChromeDriver` — enables swapping drivers

---

Q3.3: Explain the difference between a Stack and a Queue. Where would you use each in testing?

Stack (LIFO — Last In, First Out): - Push adds to the top, pop removes from the top. - Use case in testing: Tracking navigation history for "back button" testing, undo/redo verification, managing nested contexts (e.g., modal on top of modal).

Queue (FIFO — First In, First Out): - Enqueue adds to the back, dequeue removes from the front. - Use case in testing: Processing test execution results in order, managing message queue verification (messages should be consumed in order), simulating user request queues for load testing.

Priority Queue: A variant where elements are dequeued based on priority, not insertion order. Useful for risk-based test execution — run highest-priority tests first.

Q3.4: What is the difference between HashMap and TreeMap?

Feature	HashMap	TreeMap
Ordering	No guaranteed order	Sorted by keys (natural or custom comparator)
Time complexity	O(1) average for get/put	O(log n) for get/put
Null keys	Allows one null key	Does not allow null keys
Implementation	Hash table	Red-black tree

Testing relevance: - **HashMap:** Fast lookup for test data mappings (test case ID → test data), caching page elements. - **TreeMap:** When you need sorted test results (e.g., generating reports sorted by test name or timestamp), or when verifying that a sorted data structure returns elements in order.

Q3.5: What is the difference between concurrency and parallelism? Why does it matter for test execution?

Concurrency: Multiple tasks make progress during overlapping time periods by interleaving execution (may be on a single core). Think of a single chef switching between multiple dishes.

Parallelism: Multiple tasks execute simultaneously on multiple cores/machines. Think of multiple chefs each cooking a different dish at the same time.

Testing relevance: - **Parallel test execution** (e.g., TestNG parallel suites, pytest-xdist, Playwright workers) runs tests simultaneously on multiple threads/processes/machines. This reduces total execution time linearly with available resources. - **Concurrency bugs** in the application (race conditions, deadlocks) can only be caught by tests that simulate concurrent access — multiple threads hitting the same endpoint, multiple users modifying the same record. - **Thread safety in test frameworks:** Shared state in test utilities (e.g., a singleton WebDriver) can cause test failures when tests run in parallel. Use thread-local storage or dependency injection to isolate state.

Q3.6: What is mocking and when should you use it?

Mocking replaces real dependencies with controlled substitutes during testing. Types of test doubles:

Type	Purpose
Mock	Verifies that specific interactions occurred (method called with expected args)
Stub	Returns predefined responses without verifying interactions
Fake	A working implementation that is simplified (e.g., in-memory database)
Spy	Wraps a real object, recording interactions while delegating to the real implementation

When to mock: - External services (payment gateways, email, third-party APIs) — unreliable and slow in tests. - Time-dependent behavior (testing expiry logic without waiting). - Error scenarios that are hard to reproduce (network timeouts, disk full).

When NOT to mock: - The component under test itself — you'd be testing your mock, not your code. - Simple value objects or data structures. - Integration tests — the point is to test real interactions. - When the mock setup becomes more complex than the actual dependency.

Q3.7: Explain the difference between an abstract class and an interface.

Aspect	Abstract Class	Interface
Methods	Can have both abstract and concrete methods	All methods are abstract (Java 8+ allows default methods)
State	Can have instance variables	Only constants (static final)
Constructors	Can have constructors	Cannot have constructors
Inheritance	Single inheritance only	A class can implement multiple interfaces
Access modifiers	Any access modifier	Public by default

Testing framework relevance: - **Abstract class:** Use for base test classes that share setup/teardown logic (`BaseTest` with `setUp()` and `tearDown()` methods) while allowing subclasses to define specific test behavior. - **Interface:** Use for defining contracts (e.g., a `PageObject` interface that all page objects must implement, ensuring consistency across the framework).

---

4. API Testing

Q4.1: What are the different HTTP methods and when do you use each?

Method	Purpose	Idempotent	Safe	Request Body
GET	Retrieve a resource	Yes	Yes	No
POST	Create a new resource	No	No	Yes
PUT	Replace a resource entirely	Yes	No	Yes
PATCH	Partially update a resource	No*	No	Yes
DELETE	Remove a resource	Yes	No	Optional
HEAD	Same as GET but no response body	Yes	Yes	No
OPTIONS	Describe communication options (CORS preflight)	Yes	Yes	No

*PATCH can be idempotent depending on the implementation.

Idempotent means making the same request multiple times produces the same result. This matters for testing: idempotent methods can be safely retried; non-idempotent methods (POST) may create duplicate resources if retried.

Q4.2: How do you test a REST API comprehensively?

A comprehensive API test strategy covers:

1. **Functional testing:** - Happy path: Valid inputs return correct responses (status code, body, headers). - Negative testing: Invalid inputs, missing required fields, wrong data types. - Edge cases: Empty strings, maximum lengths, special characters, Unicode. - Business logic: Verify domain rules are enforced.
2. **Status code validation:** - 200/201 for success, 400 for bad request, 401 for unauthorized, 403 for forbidden, 404 for not found, 409 for conflict, 422 for validation errors, 500 for server errors.

3. **Response validation:** - Schema validation (JSON Schema) — ensures structure doesn't change unexpectedly. - Data type checking, required fields, nullable fields. - Response time assertions.
4. **Authentication & authorization:** - Test with valid/invalid/expired tokens. - Test role-based access — a regular user should not access admin endpoints.
5. **Contract testing:** Verify the API adheres to its OpenAPI/Swagger spec.
6. **Integration points:** Test interactions between dependent APIs.
7. **Idempotency:** Verify that repeated identical requests produce consistent results.

Q4.3: What is the difference between authentication and authorization?

Authentication verifies *who you are* — confirming identity through credentials (username/password, tokens, certificates, biometrics).

Authorization determines *what you can do* — checking whether the authenticated identity has permission to access a resource or perform an action.

Aspect	Authentication	Authorization
Question	"Who are you?"	"What can you do?"
Happens	First	After authentication
Mechanism	Credentials, tokens, OAuth	Roles, permissions, ACLs, policies
HTTP status on failure	401 Unauthorized	403 Forbidden
Example	Login with username/password	Admin can delete users; regular user cannot

Testing implications: - Test authentication: valid credentials, invalid credentials, expired tokens, token refresh flows, multi-factor authentication. - Test authorization: access with different roles, privilege escalation attempts, accessing resources owned by other users.

Q4.4: Explain contract testing. How is it different from integration testing?

Contract testing verifies that two services (consumer and provider) agree on the structure and format of their communication — the "contract." Each side is tested independently against the contract.

Integration testing verifies that two services actually work together by running them simultaneously and testing real interactions.

Aspect	Contract Testing	Integration Testing
Environment	Each service tested in isolation	Both services running together
Speed	Fast (no real service calls)	Slower (real network, real services)
Failure isolation	Pinpoints which side broke the contract	May be harder to isolate
Tools	Pact, Spring Cloud Contract	Testcontainers, Docker Compose
When to use	Microservices with independent deployment	Verifying actual behavior end-to-end

Contract testing workflow (Pact example): 1. **Consumer** writes tests defining expected request/response pairs → generates a contract (pact file). 2. **Provider** runs the pact file against its implementation → verifies it can satisfy the contract. 3. If either side changes in a breaking way, the contract test fails before deployment.

Q4.5: What is GraphQL and how does testing it differ from REST?

GraphQL is a query language for APIs where clients specify exactly what data they need, unlike REST where the server determines the response shape.

Key differences for testing:

Aspect	REST	GraphQL
Endpoint	Multiple endpoints (`/users` , `/orders`)	Single endpoint (`/graphql`)
Response shape	Fixed by server	Defined by client query
Over/under-fetching	Common (get too much or too little data)	Eliminated (get exactly what you request)
Status codes	Uses HTTP status codes semantically	Often returns 200 even for errors (errors in response body)
Versioning	URL versioning (`/v1/` , `/v2/`)	Schema evolution, deprecation of fields

GraphQL testing considerations: - **Query validation:** Test valid and malformed queries. - **Schema testing:** Verify schema changes don't break existing queries. - **Resolver testing:** Each resolver (function that fetches data for a field) should be unit-tested. - **Depth/complexity limits:** Test that deeply nested or overly complex queries are rejected (prevents DoS). - **N+1 query problems:** Verify that batch loading (DataLoader) is working correctly. - **Error handling:** Since GraphQL returns 200 for errors, validate the `errors` array in the response body.

Q4.6: How do you handle API versioning in your test suite?

Strategies for testing versioned APIs:

1. **Parallel test suites:** Maintain separate test suites for each active API version. Tests for v1 and v2 can run concurrently.
2. **Parameterized tests:** Use a version parameter to run the same logical test against multiple versions: `@ParameterizedTest @ValueSource(strings = {"v1", "v2"}) void testGetUser(String version) { response = get("/api/" + version + "/users/1"); // version-specific assertions }`
3. **Contract-based approach:** Maintain contracts per version. When a version is deprecated, archive its contract tests but keep them runnable until the version is sunset.
4. **Backward compatibility tests:** When releasing a new version, run existing tests from the previous version against the new version to catch unintended breaking changes.
5. **Version-specific assertions:** Some tests share the same flow but differ in response structure. Use version-aware assertion helpers.

Q4.7: What is idempotency and how do you test for it?

Idempotency means that making the same request multiple times produces the same result as making it once. The server state doesn't change after the first request.

Testing approach: 1. **Send the same request multiple times** (e.g., PUT the same resource 3 times). 2. **Verify:** The response is identical each time. The resource is in the same state. No duplicate side effects (no duplicate emails sent, no duplicate records created). 3. **Test with idempotency keys:** For POST endpoints that support idempotency keys (common in payment APIs), send the same key twice and verify only one resource is created. 4. **Concurrent requests:** Send the same idempotent request simultaneously from multiple threads and verify only one effect occurs.

Methods that should be idempotent: GET, PUT, DELETE, HEAD, OPTIONS. **Methods typically not idempotent:** POST (each call may create a new resource), PATCH (depends on implementation).

---

5. CI/CD & DevOps

Q5.1: How do you integrate automated tests into a CI/CD pipeline?

A well-structured pipeline runs tests in stages, from fastest to slowest:

```
┌──────────┐    ┌──────────┐    ┌──────────┐    ┌──────────┐    ┌──────────┐
│ Commit   │──▶│ Build    │──▶│ Unit     │──▶│Integration│──▶│ E2E      │
│ Trigger  │    │ & Lint   │    │ Tests    │    │ Tests    │    │ Tests    │
└──────────┘    └──────────┘    └──────────┘    └──────────┘    └──────────┘
                                                                     │
                                                                     ▼
                                                                ┌──────────┐
                                                                │ Deploy   │
                                                                │ (if all  │
                                                                │ pass)    │
                                                                └──────────┘
```

Best practices: - **Fail fast:** Run static analysis and unit tests first. If they fail, don't waste time on slower stages. - **Parallel execution:** Run independent test suites concurrently. - **Test result artifacts:** Publish JUnit XML reports, screenshots of failures, and logs. - **Flaky test management:** Quarantine flaky tests so they don't block deployments. - **Environment provisioning:** Use Docker/containers to spin up consistent test environments. - **Selective testing:** On large codebases, run only tests affected by the changed files (test impact analysis). - **Gate criteria:** Define pass/fail criteria — e.g., all tests pass, code coverage ≥ 80%, no critical security findings.

Q5.2: What is the difference between continuous integration, continuous delivery, and continuous deployment?

Concept	Definition	Automation Level
Continuous Integration (CI)	Developers merge code to a shared branch frequently; each merge triggers automated builds and tests	Build + test automated
Continuous Delivery (CD)	Code is always in a deployable state; deployment to production requires manual approval	Build + test + staging deploy automated; production deploy is manual
Continuous Deployment	Every change that passes all stages is automatically deployed to production with no manual intervention	Everything automated

SDET's role: - **CI:** Ensure unit and integration tests are fast, reliable, and provide clear failure messages. - **Continuous Delivery:** Build acceptance test suites that give confidence the release is production-ready. Implement smoke tests for staging environments. - **Continuous Deployment:** Build robust E2E tests and monitoring/alerting that catch issues post-deployment. Implement canary testing and feature flag verification.

Q5.3: How do you manage test infrastructure at scale?

Challenges: Running thousands of tests across multiple browsers, devices, and environments.

Solutions:

1. **Containerization (Docker):** - Package test environments as Docker images for consistency. - Use Docker Compose for multi-service test setups. - Spin up/tear down environments per pipeline run.
2. **Orchestration (Kubernetes):** - Scale test execution dynamically based on demand. - Run browser tests on Selenium Grid deployed in K8s. - Use ephemeral namespaces for test isolation.
3. **Cloud-based testing platforms:** - Sauce Labs, BrowserStack, LambdaTest for cross-browser/device testing. - AWS Device Farm for mobile testing.

4. **Test parallelization:** - Split tests across multiple workers/machines. - Use test sharding (split by test file, by tag, or by estimated duration).
5. **Caching and optimization:** - Cache dependencies (npm modules, Maven artifacts) between pipeline runs. - Use incremental builds — only rebuild what changed. - Implement test impact analysis — only run tests affected by the change.

Q5.4: What is infrastructure as code and how does it help testing?

Infrastructure as Code (IaC) manages infrastructure (servers, networks, databases) through machine-readable configuration files rather than manual processes.

Tools: Terraform, AWS CloudFormation, Pulumi, Ansible.

Benefits for testing: - **Reproducibility:** Spin up identical test environments every time. "Works on my machine" becomes a solved problem. - **Version control:** Infrastructure configurations are versioned alongside application code. You can track exactly what environment a test ran against. - **Ephemeral environments:** Create a full environment for a feature branch, run tests, tear it down. No shared, long-lived test environments that drift from production. - **Cost control:** Environments exist only for the duration of the test run. - **Testing the infrastructure itself:** IaC can be tested with tools like Terratest — verify that your infrastructure deploys correctly and meets compliance requirements.

Q5.5: How do you handle test failures in CI — do you block the pipeline?

Strategy depends on the test type and failure pattern:

Test Type	Block Pipeline?	Rationale
Unit tests	Yes, always	Fast, reliable, foundational. Failures indicate real bugs.
Integration tests	Yes	Real interaction failures that must be fixed.
E2E / UI tests	Conditional	May be flaky. Use a threshold (e.g., block if >2 failures).
Performance tests	Conditional	Block on significant regression (>10% degradation). Alert on minor changes.
Security scans	Depends on severity	Block on critical/high findings. Warn on medium/low.

Handling flaky failures: - Implement automatic retry (1 retry, not more). - If a test fails on retry, it's likely a real issue — block the pipeline. - Track flaky test metrics. If a test has >5% flakiness rate, quarantine it and fix within a sprint. - Use a "flaky test dashboard" to maintain visibility.

---

6. Performance Testing

Q6.1: What are the different types of performance testing?

Type	Purpose	Approach
Load testing	Verify system behavior under expected load	Simulate expected number of concurrent users
Stress testing	Find the system's breaking point	Gradually increase load beyond expected capacity until failure
Spike testing	Verify behavior under sudden load spikes	Instantly jump to high load, then drop

Type	Purpose	Approach
Soak/Endurance testing	Detect memory leaks, resource exhaustion	Run moderate load for an extended period (hours/days)
Scalability testing	Verify system scales with added resources	Increase load while adding capacity (horizontal/vertical)
Volume testing	Test with large amounts of data	Populate the database with millions of records, then test
Baseline testing	Establish performance benchmarks	Run under standard conditions to set reference metrics

---

Q6.2: Explain the key performance metrics you would measure.

Metric	Description	Why It Matters
Response time	Time from request sent to response received	Primary user experience indicator
Throughput	Requests processed per second (RPS/TPS)	Capacity indicator
Error rate	Percentage of failed requests	Reliability indicator
Latency percentiles (P50, P95, P99)	Response time at various percentiles	Average hides outliers; P99 shows worst-case user experience
Concurrent users	Number of simultaneous active users	Capacity planning
CPU utilization	Server CPU usage under load	Infrastructure sizing
Memory utilization	Server memory usage, GC frequency	Detect memory leaks
Connection pool usage	Database/HTTP connection pool saturation	Bottleneck detection
Apdex score	Application Performance Index (0-1)	Standardized user satisfaction metric

Why percentiles matter: If average response time is 200ms but P99 is 5 seconds, 1% of users have a terrible experience. Averages are misleading — always look at P95 and P99.

Q6.3: How do you identify performance bottlenecks?

Systematic approach:

1. **Establish baseline:** Run performance tests and collect metrics under normal load.
2. **Monitor all layers:** - **Client:** Browser rendering time, JavaScript execution time. - **Network:** DNS resolution, TLS handshake, time to first byte (TTFB). - **Application:** Request processing time, thread pool usage, garbage collection pauses. - **Database:** Query execution time, slow query logs, lock contention, connection pool exhaustion. - **Infrastructure:** CPU, memory, disk I/O, network bandwidth.
3. **Profiling tools:** - APM tools: New Relic, Datadog, Dynatrace — trace requests across services. - Database: `EXPLAIN ANALYZE` for query plans, slow query logs. - Application: Thread dumps, heap dumps, flame graphs.
4. **Isolate the bottleneck:** - If response time increases linearly with load → likely CPU-bound. - If response time increases exponentially → likely contention (locks, connection pools). - If throughput plateaus while response time spikes → saturated resource.
5. **Common bottlenecks:** - N+1 database queries. - Missing database indexes. - Synchronous calls to external services (should be async). - Inefficient serialization/deserialization. - Thread pool exhaustion. - Memory leaks causing GC pauses.

Q6.4: What tools do you use for performance testing?

Tool	Type	Best For
JMeter	Load testing	HTTP/HTTPS, SOAP, REST, JDBC, JMS. Mature, extensible.
Gatling	Load testing	Code-based scenarios (Scala/Java/Kotlin), detailed HTML reports.
k6	Load testing	Developer-friendly (JavaScript), CI/CD integration, cloud execution.
Locust	Load testing	Python-based, distributed, easy to write test scenarios.
Artillery	Load testing	YAML/JS scenarios, serverless-first, good for microservices.
wrk / hey	Benchmarking	Quick HTTP benchmarks from the command line.
Lighthouse	Frontend performance	Core Web Vitals, accessibility, SEO auditing.
Apache Bench (ab)	Benchmarking	Simple, quick HTTP load generation.

Choosing a tool: Consider team expertise (Java → JMeter/Gatling; JS → k6/Artillery; Python → Locust), protocol requirements, CI/CD integration needs, and reporting capabilities.

Q6.5: How do you performance test microservices?

Challenges: Microservices introduce network latency, distributed tracing complexity, and cascading failure risks.

Approach:

1. **Individual service testing:** Load test each service in isolation with mocked dependencies. Identifies per-service bottlenecks.
2. **Integration performance testing:** Test critical paths that span multiple services. Use realistic traffic patterns.
3. **Chaos engineering:** Inject failures (network latency, service crashes) during load testing to verify resilience. Tools: Chaos Monkey, Gremlin, Litmus.
4. **Service mesh observability:** Use Istio/Linkerd metrics to monitor inter-service communication latency, retry rates, and circuit breaker activations.
5. **Distributed tracing:** Use Jaeger or Zipkin to trace requests across services and identify which service contributes most to latency.
6. **Capacity planning per service:** Each service may scale independently. Performance tests should inform auto-scaling thresholds.
7. **Contract-based performance SLAs:** Define performance budgets per service (e.g., “Service A must respond in < 50ms at P99 for GET requests”).

---

7. Security Testing

Q7.1: What are the OWASP Top 10 and how do you test for them?

The OWASP Top 10 (2021 edition) lists the most critical web application security risks:

#	Risk	How to Test
A01	**Broken Access Control**	

#	Risk	How to Test
		Test horizontal/vertical privilege escalation, IDOR, missing function-level access control
A02	**Cryptographic Failures**	Check for sensitive data in plaintext, weak algorithms, missing HTTPS, exposed API keys
A03	**Injection**	SQL injection, XSS, command injection, LDAP injection — test with payloads in all input fields
A04	**Insecure Design**	Review threat models, test business logic flaws, verify rate limiting
A05	**Security Misconfiguration**	Check default credentials, unnecessary features enabled, verbose error messages, missing security headers
A06	**Vulnerable Components**	Scan dependencies (Snyk, Dependabot), check for known CVEs
A07	**Authentication Failures**	Brute force, credential stuffing, session fixation, missing MFA
A08	**Software & Data Integrity Failures**	Verify CI/CD pipeline integrity, check for unsigned updates, test deserialization
A09	**Security Logging & Monitoring Failures**	Verify security events are logged, check log injection, ensure alerts are configured
A10	**Server-Side Request Forgery (SSRF)**	Test URL parameters that fetch external resources with internal URLs

---

Q7.2: How do you test for SQL injection?

SQL injection occurs when user input is incorporated into SQL queries without proper sanitization, allowing attackers to manipulate the query.

Testing approach:

1. **Identify input points:** Form fields, URL parameters, HTTP headers, cookies — anything that might flow into a SQL query.
2. **Manual payloads:** - Basic: `' OR '1'='1` — if the login succeeds, it's vulnerable. - Error-based: `' AND 1=CONVERT(int, @@version)--` — extracts database version through error messages. - Union-based: `' UNION SELECT username, password FROM users--` — extracts data from other tables. - Blind: `' AND SUBSTRING(username,1,1)='a'--` — infers data one character at a time through true/false responses. - Time-based blind: `' AND IF(1=1, SLEEP(5), 0)--` — detects vulnerability through response time differences.
3. **Automated tools:** - SQLMap for comprehensive automated testing. - OWASP ZAP or Burp Suite for proxy-based testing. - Static analysis tools (SonarQube, Semgrep) to detect vulnerable code patterns.
4. **Prevention verification:** - Verify parameterized queries / prepared statements are used. - Verify ORM usage (but test for ORM-specific injection patterns). - Verify input validation is applied (allowlists over denylists). - Verify the principle of least privilege — database user should have minimal permissions.

Q7.3: What is Cross-Site Scripting (XSS) and how do you test for it?

XSS occurs when an application includes untrusted data in a web page without proper sanitization, allowing attackers to execute scripts in victims' browsers.

Types: - **Stored XSS:** Malicious script is permanently stored on the server (e.g., in a comment field) and served to all users who view it. - **Reflected XSS:** Malicious script is reflected off the server in an immediate response (e.g., in a search query parameter). - **DOM-based XSS:** Vulnerability exists in client-side JavaScript that processes untrusted data and updates the DOM.

Testing approach:

1. **Identify injection points:** Any place user input is rendered in the page — form fields, URL parameters, headers.
2. **Test payloads:** - Basic: `<script>alert('XSS')</script>` - Event handlers: `<img src=x onerror=alert('XSS')>` - Attribute injection: `" onmouseover="alert('XSS')` - Encoded payloads: `%3Cscript%3Ealert('XSS')%3C/script%3E` - Polyglot payloads that work across multiple contexts.
3. **Context-aware testing:** Test input rendered in HTML body, attributes, JavaScript, CSS, and URLs — each context requires different escape mechanisms.
4. **Verify protections:** - Content Security Policy (CSP) headers are set. - Output encoding is applied (HTML entities, JavaScript escaping). - `HttpOnly` and `Secure` flags on session cookies. - `X-XSS-Protection` and `X-Content-Type-Options` headers.

Q7.4: What are security headers and which ones should you test for?

Header	Purpose	Recommended Value
`Content-Security-Policy`	Controls which resources the browser can load	Restrict to known sources; block inline scripts
`Strict-Transport-Security`	Forces HTTPS connections	`max-age=31536000; includeSubDomains; preload`
`X-Content-Type-Options`	Prevents MIME type sniffing	`nosniff`
`X-Frame-Options`	Prevents clickjacking via iframes	`DENY` or `SAMEORIGIN`
`Referrer-Policy`	Controls referrer information sent with requests	`strict-origin-when-cross-origin` or `no-referrer`
`Permissions-Policy`	Controls browser features (camera, mic, geolocation)	Restrict to needed features only
`Cache-Control`	Controls caching of sensitive pages	`no-store` for pages with sensitive data

Testing approach: - Inspect response headers for every endpoint. - Automate header checks as part of the API test suite. - Use tools like SecurityHeaders.com or Mozilla Observatory for assessment. - Verify headers are consistent across all environments (dev, staging, production).

Q7.5: How do you integrate security testing into CI/CD?

Shift-left security means integrating security testing early and throughout the pipeline:

```
┌──────────┐   ┌──────────┐   ┌──────────┐   ┌──────────┐   ┌──────────┐
│ Pre-     │──▶│ Static   │──▶│ Dependency│──▶│ Dynamic  │──▶│ Runtime  │
│ commit   │   │ Analysis │   │ Scanning │   │ Analysis │   │ Security │
│ hooks    │   │ (SAST)   │   │ (SCA)    │   │ (DAST)   │   │ Monitoring│
└──────────┘   └──────────┘   └──────────┘   └──────────┘   └──────────┘
  Secrets       SonarQube     Snyk, Dependabot  OWASP ZAP     Falco,
  detection     Semgrep       npm audit         Burp Suite    Wazuh
  (gitleaks)    CodeQL        Trivy (containers) Nuclei       Datadog
```

Pipeline stages: 1. **Pre-commit:** Scan for secrets (API keys, passwords) with gitleaks or truffleHog. 2. **Build stage (SAST):** Static code analysis for security vulnerabilities. Fast, no running application needed. 3. **Dependency scanning (SCA):** Check libraries for known CVEs. Block on critical vulnerabilities. 4. **Staging (DAST):** Run dynamic

scans against the deployed application. Slower, finds runtime vulnerabilities. 5. **Production:** Runtime monitoring, anomaly detection, web application firewalls (WAF).

Gate criteria: Block deployments on critical/high security findings. Alert on medium. Log low.

---

8. Database Testing

Q8.1: What types of database testing should an SDET perform?

Type	What to Test
Schema testing	Tables, columns, data types, constraints, indexes exist as expected
Data integrity	Foreign keys enforced, unique constraints work, NOT NULL constraints respected
CRUD operations	Create, Read, Update, Delete operations through the application layer
Stored procedures / functions	Input/output validation, edge cases, error handling
Migration testing	Schema migrations apply cleanly, data is preserved, rollbacks work
Performance	Query execution times, index effectiveness, query plans
Concurrency	Behavior under concurrent read/write operations, deadlock detection
Backup & recovery	Backups are consistent, restores produce a functional database

---

Q8.2: How do you test database migrations?

Database migrations are versioned changes to the database schema (adding tables, columns, indexes, modifying constraints).

Testing approach:

1. **Forward migration testing:** - Apply migration to a copy of the production schema. - Verify the schema matches the expected state (new tables, columns, indexes exist). - Verify existing data is preserved and correctly transformed.
2. **Rollback testing:** - Apply the migration, then roll it back. - Verify the schema returns to its pre-migration state. - Verify no data loss occurs during rollback.
3. **Idempotency testing:** - Apply the same migration twice. It should either succeed silently or fail gracefully (not corrupt data).
4. **Data migration testing:** - If the migration transforms data (e.g., splitting a name column into first_name and last_name), verify all edge cases: NULL values, empty strings, special characters, extremely long values.
5. **Performance testing:** - Run the migration against a database with production-scale data volumes. - Verify it completes within acceptable time and doesn't lock tables for too long.
6. **Integration testing:** - After migration, run the application's test suite to verify everything still works.

Tools: Flyway, Liquibase, Rails ActiveRecord Migrations, Alembic (Python), each with built-in validation commands.

Q8.3: Explain the difference between SQL joins. When do you use each?

Join Type	Returns	Use Case
INNER JOIN	Only rows with matches in both tables	"Show me orders that have customers"
LEFT (OUTER) JOIN	All rows from the left table + matched rows from the right (NULLs for non-matches)	"Show me all customers and their orders, including customers with no orders"
RIGHT (OUTER) JOIN	All rows from the right table + matched rows from the left	Same as LEFT JOIN but from the other table's perspective
FULL (OUTER) JOIN	All rows from both tables, with NULLs where there's no match	"Show me all customers and all orders, even if unmatched"
CROSS JOIN	Cartesian product — every row from table A combined with every row from table B	Test data generation, combinatorial scenarios
SELF JOIN	A table joined with itself	Hierarchical data (employees and managers in the same table)

Testing relevance: - **Verify data relationships:** LEFT JOIN with a NULL check finds orphaned records (`WHERE right_table.id IS NULL`). - **Data validation:** FULL OUTER JOIN between expected and actual result sets identifies missing and extra rows. - **Test data generation:** CROSS JOIN can generate combinatorial test data sets.

Q8.4: How do you test for data integrity and consistency?

Data integrity ensures that data remains accurate, consistent, and reliable throughout its lifecycle.

Testing strategies:

1. **Constraint validation:** - Insert NULL into a NOT NULL column → should fail. - Insert a duplicate value into a UNIQUE column → should fail. - Insert a foreign key value that doesn't exist in the parent table → should fail. - Insert a value outside a CHECK constraint → should fail.
2. **Referential integrity:** - Delete a parent record with child records → verify cascade/restrict behavior. - Verify orphaned records cannot exist.
3. **Transaction testing:** - Verify ACID properties: Atomicity (all or nothing), Consistency (valid state to valid state), Isolation (concurrent transactions don't interfere), Durability (committed data survives crashes). - Simulate mid-transaction failures and verify rollback.
4. **Data consistency across systems:** - After API operations, query the database directly to verify the data matches the API response. - In event-driven systems, verify that all consumers have consistent data after event processing.
5. **Boundary testing:** - Maximum field lengths (VARCHAR(255) with 256 characters). - Numeric overflow (INT max value + 1). - Date boundaries (leap years, time zones, epoch limits).

Q8.5: What is database connection pooling and why should you test it?

Connection pooling maintains a cache of database connections that can be reused, avoiding the overhead of creating a new connection for each request.

Key parameters: - **Minimum pool size:** Connections kept open even when idle. - **Maximum pool size:** Upper limit on concurrent connections. - **Connection timeout:** How long to wait for an available connection. - **Idle timeout:** How long an unused connection stays in the pool before being closed. - **Max lifetime:** Maximum time a connection can exist before being recycled.

Why test it: - **Pool exhaustion:** If max pool size is too low, requests queue up and eventually time out under load. Performance tests should simulate enough concurrent users to stress the pool. - **Connection leaks:** Code that opens a connection but doesn't close it (missing `finally` block or try-with-resources). Soak tests detect leaks — the application gradually degrades as the pool empties. - **Stale connections:** Connections that are open but invalid

(database restarted, network blip). Test that the pool validates connections before use. - **Configuration testing:** Verify pool settings are tuned for the expected load. Too large a pool wastes database resources; too small causes contention.

---

9. Design Patterns & Architecture

Q9.1: What design patterns are commonly used in test automation?

Pattern	Use Case	Description
Page Object Model	UI testing	Encapsulates page elements and actions into classes
Factory Pattern	Test data creation	Creates test data objects through a centralized factory
Builder Pattern	Complex object construction	Constructs test data step-by-step with a fluent API
Singleton Pattern	WebDriver management	Ensures one browser instance per thread
Strategy Pattern	Multiple test execution strategies	Swap between different browsers, environments, or data sources
Observer Pattern	Test reporting	Listeners observe test events (pass, fail, skip) and generate reports
Decorator Pattern	Adding behaviors to tests	Add logging, screenshots, or retries to test methods without modifying them
Command Pattern	Keyword-driven frameworks	Encapsulate test actions as command objects

---

Q9.2: How do you design a test automation framework from scratch?

Architecture layers:

```
┌──────────────────────────────┐
│        Test Layer            │  ← Test cases, test suites
├──────────────────────────────┤
│     Business Logic Layer     │  ← Reusable test steps, workflows
├──────────────────────────────┤
│     Page/API Object Layer    │  ← Page objects, API clients
├──────────────────────────────┤
│     Core Framework Layer     │  ← Driver management, config, reporting
├──────────────────────────────┤
│     Utility Layer            │  ← Helpers: data gen, file I/O, logging
└──────────────────────────────┘
```

Key decisions:

1. **Technology stack:** Match the application and team skills (Java + TestNG + Selenium, TypeScript + Playwright, Python + pytest).
2. **Configuration management:** External config files (YAML, properties) for URLs, credentials, browser settings. Environment-specific overrides.
3. **Test data management:** Factories for creating data, cleanup mechanisms, data isolation.
4. **Reporting:** Allure, ExtentReports, or custom HTML dashboards. Include screenshots, logs, and execution metadata.
5. **Logging:** Structured logging at appropriate levels (DEBUG for element interactions, INFO for test steps, ERROR for failures).

6. **Parallel execution support:** Thread-safe driver management, isolated test data, no shared mutable state.
7. **CI/CD integration:** JUnit XML output, Docker support, configurable via environment variables.
8. **Retry mechanism:** Configurable retry count for flaky tests, with detailed logging of retry reasons.

Q9.3: What is the Builder pattern and how is it useful in test automation?

The **Builder pattern** constructs complex objects step-by-step using a fluent API, separating construction from representation.

Without Builder:

```
User user = new User("John", "Doe", "john@example.com", "password123",
    "Admin", true, "2024-01-01", "US", "en", null, null, true);
```

With Builder:

```
User user = User.builder()
    .firstName("John")
    .lastName("Doe")
    .email("john@example.com")
    .role("Admin")
    .active(true)
    .build();
```

Benefits in test automation: - **Readable test data:** Tests clearly show which fields matter for each scenario. - **Defaults:** The builder sets sensible defaults — you only override what's relevant to the test. - **Immutability:** The built object can be immutable, preventing accidental modification. - **Variation:** Create similar objects with small differences easily: `User admin = User.builder().role("Admin").build();` `User guest = User.builder().role("Guest").build();`

Q9.4: How do you handle cross-cutting concerns in a test framework?

Cross-cutting concerns are aspects that affect multiple parts of the framework (logging, reporting, screenshot capture, retry logic).

Solutions:

1. **Aspect-Oriented Programming (AOP):** Use aspects/annotations to inject behavior before/after test methods without modifying them. Example: `@TakeScreenshotOnFailure`.
2. **Test listeners/hooks:** - TestNG: `ITestListener`, `ISuiteListener` - JUnit: `TestWatcher`, `Extension` - pytest: fixtures, hooks (`conftest.py`) - Playwright: `beforeEach`, `afterEach`
3. **Middleware/decorators:** Wrap test execution with additional behavior: - Logging decorator that logs entry/exit of every test method. - Retry decorator that re-executes on specific exception types. - Screenshot decorator that captures the browser state on failure.
4. **Base test class:** Common setup and teardown logic in a parent class that all test classes extend. Use sparingly — deep inheritance hierarchies become hard to maintain.

Q9.5: Explain the Strangler Fig pattern and how it applies to testing legacy systems.

The **Strangler Fig pattern** (named after trees that grow around and eventually replace their host) is a migration strategy where you gradually replace a legacy system by building new features alongside it and incrementally routing traffic to the new system.

How it applies to testing:

1. **Dual testing:** Run tests against both the legacy and new systems simultaneously. Compare results to ensure parity.

2. **Contract testing at the boundary:** Define contracts between the legacy and new systems. As you migrate functionality, the contract tests verify the new system behaves identically to the old one.
3. **Feature flag testing:** The migration often uses feature flags to route users. Test both paths — flag on (new system) and flag off (legacy system).
4. **Incremental test migration:** As each feature moves to the new system, migrate its tests too. Don't try to rewrite the entire test suite at once.
5. **Regression safety net:** Maintain the legacy test suite as a safety net until the migration is complete. Only decommission legacy tests after the corresponding feature is fully migrated and verified.

Q9.6: What is the Repository pattern and how does it help with test data management?

The **Repository pattern** abstracts data access behind an interface, decoupling the domain logic from the data source.

In test automation:

```
interface TestDataRepository {
    User createUser(UserProfile profile);
    Order createOrder(User user, Product product);
    void cleanup();
}

class ApiTestDataRepository implements TestDataRepository {
    // Creates test data via API calls
}

class DatabaseTestDataRepository implements TestDataRepository {
    // Creates test data directly in the database
}

class InMemoryTestDataRepository implements TestDataRepository {
    // Creates test data in memory (for unit tests)
}
```

Benefits: - **Swappable backends:** Use API-based setup in E2E tests, direct database inserts in integration tests, and in-memory objects in unit tests — same interface, different implementations. - **Centralized cleanup:** The repository tracks what it created and cleans up after the test. - **Test isolation:** Each test gets its own repository instance, ensuring data isolation. - **Readable tests:** Tests call `testDataRepo.createUser(admin)` instead of raw SQL or API boilerplate.

---

10. Behavioral & Process

Q10.1: How do you prioritize what to test when time is limited?

Framework: Risk-based prioritization

1. **Identify critical paths:** What are the most important user journeys? (login, checkout, payment, core business workflows).
2. **Assess change risk:** What code changed? Areas with the most changes or complex changes get higher priority.
3. **Consider historical defect density:** Which modules have had the most bugs historically?
4. **Evaluate business impact:** A bug in the payment flow is more critical than a bug in the settings page.
5. **Check dependency risk:** Changes to shared libraries or infrastructure affect many features.

Practical approach: - **Must test:** Smoke tests for critical paths, tests for changed areas, tests for areas downstream of changes. - **Should test:** Regression tests for related features, boundary tests for changed logic. - **Nice to test:** Full regression suite, exploratory testing on low-risk areas.

Q10.2: How do you handle disagreements with developers about whether something is a bug?

Approach:

1. **Start with evidence, not opinion:** Show the expected behavior (from requirements, acceptance criteria, or design specs) and the actual behavior. Screenshots, logs, and reproduction steps make the discussion objective.
2. **Reference the source of truth:** If the requirements are ambiguous, escalate to the product owner or business analyst for clarification. The SDET doesn't decide what's a bug — the requirements do.
3. **Separate severity from existence:** You might agree it's a bug but disagree on priority. Focus first on whether the behavior is correct, then discuss priority separately.
4. **Be collaborative, not adversarial:** Frame it as "the system does X but the spec says Y — should we change the system or update the spec?" rather than "you broke this."
5. **Document the decision:** If the team decides it's "working as intended," document that decision. If it comes up again, there's a record of the reasoning.
6. **Know when to escalate:** If the disagreement persists and the potential impact is significant, involve the tech lead, product owner, or architect.

Q10.3: Describe your approach to shifting testing left.

Shift-left means integrating testing earlier in the development lifecycle, rather than treating it as a final gate.

Concrete practices:

1. **Requirement reviews:** Participate in requirement/story refinement. Identify ambiguities, missing acceptance criteria, and edge cases before development starts.
2. **Testability advocacy:** Ensure features are designed to be testable — proper APIs for test setup, observable state, deterministic behavior.
3. **TDD/BDD collaboration:** Work with developers to write test cases or BDD scenarios before coding begins. The tests become the specification.
4. **Code review participation:** Review pull requests for testability, edge case handling, and security concerns.
5. **Unit test support:** Help developers write better unit tests — suggest edge cases, boundary conditions, and negative scenarios they may have missed.
6. **Early automation:** Write automation alongside development, not after. As a feature is built, the tests are built in parallel.
7. **Static analysis:** Integrate linters, SAST tools, and type checking into the development workflow to catch issues before tests even run.

Benefits: Bugs found earlier are cheaper to fix. A bug caught in requirements costs 10x less than one found in production.

Q10.4: How do you measure test effectiveness?

Metrics to track:

Metric	What It Measures	Target
Defect detection rate	Bugs found by tests ÷ total bugs (including production)	> 90%
Test coverage	Lines/branches/conditions covered by tests	Context-dependent; 80%+ for critical code
Escape rate	Bugs that reach production ÷ total bugs	< 10%

Metric	What It Measures	Target
Test execution time	Time to run the full suite	Fast enough to run on every commit
Flaky test rate	Flaky tests ÷ total tests	< 2%
Automation rate	Automated tests ÷ total test cases	Depends on project maturity
Mean time to detect (MTTD)	Time from bug introduction to detection	Minimize
Cost per bug found	Total testing cost ÷ bugs found	Decreasing over time

Beyond numbers: - **Mutation testing:** Introduce small code changes (mutations) and check if tests catch them. If a mutation survives (no test fails), there's a gap in coverage. - **Root cause analysis:** For escaped bugs, analyze why tests didn't catch them and improve the suite accordingly. - **Qualitative feedback:** Are developers confident deploying? Do stakeholders trust the release quality?

Q10.5: How would you introduce test automation in a team that has never done it?

Phased approach:

Phase 1 — Quick wins (weeks 1–4): - Start with API tests — they're faster to write, more stable, and have immediate value. - Pick a small, stable, critical feature (e.g., login, core CRUD operations). - Use a simple tool the team is already comfortable with (same language as the application). - Run tests locally and demonstrate the value with a passing suite.

Phase 2 — Foundation (months 2–3): - Set up CI integration — tests run on every commit. - Establish coding standards for test code (naming, structure, assertions). - Introduce the Page Object Model for UI tests. - Create shared utilities (test data factories, custom assertions). - Train the team through pairing sessions, not just documentation.

Phase 3 — Scaling (months 3–6): - Increase coverage systematically (critical paths first, then edge cases). - Add visual regression testing, performance testing, or security scanning as needed. - Implement parallel execution to keep suite execution time manageable. - Establish ownership — each team member maintains tests for their feature area.

Key success factors: - **Executive buy-in:** Automation requires initial time investment — stakeholders must understand the long-term ROI. - **Developer involvement:** SDETs should not be the only ones writing tests. Developers write unit tests; SDETs own the framework and E2E/integration tests. - **Maintenance culture:** Tests are code. They need refactoring, reviews, and maintenance. Budget for it.

Q10.6: What is exploratory testing and when do you use it?

Exploratory testing is a hands-on approach where the tester simultaneously learns, designs, and executes tests. Unlike scripted testing, there are no predefined steps — the tester uses their knowledge, intuition, and curiosity to probe the system.

When to use: - **New features:** Before automation, explore the feature to understand its behavior and find unexpected issues. - **After a major refactor:** Verify that the system still behaves correctly in ways that automated tests might not cover. - **Time-constrained testing:** When there's no time for comprehensive scripted testing, exploratory testing efficiently covers high-risk areas. - **Finding usability issues:** Automated tests verify correctness but miss "this workflow is confusing." - **Complementing automation:** Automated tests are great for regression; exploratory testing finds new bugs.

Structured approach (Session-Based Test Management): - **Charter:** "Explore the checkout flow with focus on edge cases in discount code application." - **Time-box:** 60-90 minute sessions. - **Notes:** Document what was tested, what was found, and what areas need further investigation. - **Debrief:** Share findings with the team and decide which bugs to log and which areas need automated test coverage.

Q10.7: How do you ensure test maintainability as the codebase grows?

Strategies:

1. **Treat test code as production code:** Apply the same code quality standards — code reviews, refactoring, DRY principles.
2. **Stable locators:** Use `data-testid` attributes for UI elements. Avoid XPath based on DOM structure.
3. **Abstraction layers:** Page objects for UI, API clients for API tests. Tests should read like business workflows.
4. **Single Responsibility:** Each test verifies one behavior. If a test does too much, it's hard to diagnose failures and expensive to maintain.
5. **Avoid test interdependence:** Tests should be runnable in any order. Each test sets up its own state and cleans up afterward.
6. **Regular pruning:** Remove tests that are permanently skipped, test obsolete features, or duplicate coverage. Dead tests are maintenance debt.
7. **Clear naming conventions:** - Bad: `test1`, `testLogin`, `testEdgeCase` - Good: `should_show_error_when_password_is_empty`, `returns_404_for_nonexistent_user`
8. **Parameterization:** When the same logic is tested with different data, use parameterized tests instead of copy-pasting test methods.
9. **Documentation for complex tests:** If a test requires unusual setup or tests a non-obvious scenario, add a brief comment explaining *why*.
10. **Regular refactoring sprints:** Dedicate time each quarter to address test debt — update deprecated APIs, refactor brittle tests, improve reporting.

This document covers the core knowledge areas expected of an SDET. Use it as a study guide, interview preparation resource, or team knowledge-sharing reference.

AWS Cloud Guide

For: Udani Weerasinghe | **Role:** Associate Lead SDET, LSEG **Why this matters:** The JD requires "Familiarity with Cloud technologies such as AWS/Azure." Both interviewers (Amy Lee, Andy Purtell) have infrastructure backgrounds. LSEG runs large-scale financial infrastructure — expect questions about testing in cloud environments.

Table of Contents

1. AWS Core Concepts Every SDET Must Know
2. Compute & Containers — Where Tests Run
3. CI/CD on AWS — Your Strongest Connection
4. Networking & Security — What Interviewers from Infra Backgrounds Ask
5. Storage & Databases — Test Data Management
6. Monitoring, Logging & Observability
7. Serverless & Event-Driven Architectures
8. Infrastructure as Code — Test Environment Provisioning
9. Performance & Load Testing on AWS
10. Security Testing on AWS
11. Financial Services Context (LSEG-Specific)
12. Quick Reference — Services Cheat Sheet
13. Interview Answer Templates

1. AWS Core Concepts Every SDET Must Know

Regions and Availability Zones

- **Region:** A geographical area (e.g., `eu-west-1` for Ireland, `ap-southeast-1` for Singapore). Each region has multiple Availability Zones.
- **Availability Zone (AZ):** An isolated data centre within a region. Applications are deployed across multiple AZs for high availability.
- **Why it matters for testing:** LSEG serves global markets. Tests may need to verify latency from specific regions, data residency compliance, and failover behaviour across AZs.

The Shared Responsibility Model

AWS Manages	You (LSEG) Manage
Physical security, hardware, networking	Data, identity & access, application security
Hypervisor, host OS	Guest OS, patches, firewall rules
Service availability	Configuration, encryption, compliance

SDET angle: You test what LSEG manages — application security, access controls, data encryption, and configuration correctness. AWS handles the infrastructure underneath.

IAM (Identity and Access Management)

This is **the most important AWS service for an SDET to understand**.

- **Users:** Individual identities (human or service accounts).

- **Roles:** Assumed by services, applications, or users temporarily. **Test automation services should use roles, not hardcoded credentials.**
- **Policies:** JSON documents defining what actions are allowed/denied on which resources.
- **Principle of least privilege:** Every user/role should have only the minimum permissions required.

What you should be able to discuss: - How your test automation authenticates to AWS (roles, not access keys). - How you test that IAM policies are correctly configured (e.g., a "reader" role cannot write). - How to test for privilege escalation — can a lower-privileged user access admin resources?

AWS CLI & SDKs

- **AWS CLI:** Command-line tool for interacting with AWS services. SDETs use this in CI/CD scripts.
- **AWS SDKs:** Libraries for Java, Python, JavaScript, etc. Used to interact with AWS programmatically in test code.
- **Boto3** (Python) — the most common for test scripting.
- **AWS SDK for Java** — relevant since Udani's automation stack is Java-based.

---

2. Compute & Containers — Where Tests Run

EC2 (Elastic Compute Cloud)

Virtual servers in the cloud. As an SDET you should know:

Concept	What It Means	Testing Relevance
Instance types	Different CPU/memory configurations (e.g., `t3.medium`, `c5.xlarge`)	Choose the right size for test execution agents — too small = slow tests, too large = wasted cost
AMIs (Amazon Machine Images)	Pre-configured OS templates	Create standardised test environment images
Security Groups	Virtual firewalls controlling inbound/outbound traffic	Test that only expected ports are open
Key Pairs	SSH authentication	Used to access test servers for debugging
Auto Scaling	Automatically add/remove instances based on demand	Test that the application handles scaling events correctly

ECS (Elastic Container Service) & EKS (Elastic Kubernetes Service)

Service	What It Is	When to Use
ECS	AWS-managed container orchestration	Simpler container workloads, tightly integrated with AWS
EKS	Managed Kubernetes	Complex microservices, multi-cloud, or when the team already knows Kubernetes
Fargate	Serverless compute for containers (works with both ECS and EKS)	No server management — just define CPU/memory and run containers

SDET relevance: - **Running test suites in containers:** Package your Selenium/Playwright tests as Docker images, run them on ECS/Fargate. Each CI pipeline run spins up fresh containers. - **Testing containerised applications:** Verify health checks, container startup/shutdown, inter-container networking. - **Selenium Grid on ECS/EKS:** Run a scalable Selenium Grid using containers instead of managing VMs.

Docker & ECR (Elastic Container Registry)

- **ECR** stores Docker images. Your test automation Docker images are pushed here and pulled during CI/CD.
- **Key workflow:** Build test image → Push to ECR → CI/CD pipeline pulls image → Runs tests on ECS/Fargate.

Udani's connection: Your MSc in Cloud Native Computing likely covered containerisation extensively. Link this to your Sysco LABS experience where you integrated tests into CI/CD (Jenkins + Git + AWS).

---

3. CI/CD on AWS — Your Strongest Connection

This is where your experience directly maps. At Sysco LABS, you used Jenkins + Git + AWS for CI/CD.

AWS CI/CD Services

Service	Purpose	Equivalent You Know
CodeCommit	Git repository hosting	Git/GitLab/Bitbucket
CodeBuild	Build and test execution (serverless)	Jenkins build agents
CodePipeline	Orchestrates the CI/CD pipeline	Jenkins Pipeline
CodeDeploy	Automates deployment to EC2, ECS, Lambda	Jenkins deployment steps

Jenkins on AWS (What You Actually Used)

The most common pattern — and likely what LSEG uses:

```
┌──────────────┐      ┌──────────────┐      ┌──────────────┐
│  CodeCommit  │──────▶│   Jenkins    │──────▶│  Test Env    │
│  or GitHub   │      │  (on EC2)    │      │  (EC2/ECS)   │
└──────────────┘      └──────────────┘      └──────────────┘
                             │
                      ┌──────┴──────┐
                      │  Jenkins    │
                      │  Agents     │
                      │  (EC2/ECS)  │
                      └─────────────┘
```

How to explain your experience:

> *"At Sysco LABS, I integrated our Selenium + TestNG + REST Assured automation suites into Jenkins pipelines that deployed and tested on AWS. The pipeline would trigger on Git pushes, build the test artifacts, deploy to test environments, run the full regression suite, and report results. I was responsible for maintaining both the test code and the pipeline configuration."*

Key Things to Know for the Interview

1. **Build specifications:** How to define what CodeBuild/Jenkins does — install dependencies, run tests, publish results.
2. **Artifact management:** Where test results, screenshots, and logs are stored (S3).
3. **Environment variables and secrets:** Using AWS Systems Manager Parameter Store or Secrets Manager — never hardcode credentials.
4. **Parallel test execution:** Running test suites across multiple build agents to reduce execution time.
5. **Pipeline gating:** Tests that must pass before deployment proceeds (quality gates).

---

4. Networking & Security — What Interviewers from Infra Backgrounds Ask

Amy was a Unix admin and Andy tested across infrastructure. Expect at least one question here.

VPC (Virtual Private Cloud)

A logically isolated network within AWS. Think of it as your own private data centre in the cloud.

```
┌──────────────────── VPC (10.0.0.0/16) ────────────────────┐
│                                                            │
│  ┌─── Public Subnet ───┐   ┌─── Private Subnet ───┐       │
│  │  • Load balancers   │   │  • Application servers│      │
│  │  • Bastion hosts    │   │  • Databases          │      │
│  │  • NAT Gateways     │   │  • Internal services  │      │
│  └─────────────────────┘   └───────────────────────┘      │
│                                                            │
│  Internet Gateway ←→ Public Subnet                         │
│  NAT Gateway: Private Subnet → Internet (outbound only)    │
└────────────────────────────────────────────────────────────┘
```

SDET relevance: - **Test environments** typically live in private subnets (not directly accessible from the internet). - **VPN or bastion hosts** are used to access test environments for debugging. - **Security groups and NACLs** control which traffic flows between subnets — test that internal services are not accidentally exposed.

Key Networking Concepts

Concept	What It Is	Testing Relevance
Security Groups	Stateful firewall rules at the instance level	Verify only required ports are open
NACLs	Stateless firewall rules at the subnet level	Additional layer of network testing
Route Tables	Define how traffic flows between subnets	Verify test environment routing is correct
Elastic Load Balancer (ELB/ALB/NLB)	Distributes traffic across instances	Test load balancing behaviour, health checks, sticky sessions
Route 53	DNS service	Test DNS resolution, failover routing
CloudFront	CDN for content delivery	Test caching behaviour, cache invalidation

What You Should Be Able to Answer

- *"How do you access a test environment that's in a private subnet?"* → Through a VPN connection or a bastion/jump host in the public subnet. In CI/CD, the build agents run inside the same VPC so they have direct access.
- *"How do you test an application behind a load balancer?"* → Test through the load balancer URL (as users would), but also test individual instances directly to isolate issues. Verify health check endpoints return correct status. Test behaviour when one instance goes down.

---

5. Storage & Databases — Test Data Management

S3 (Simple Storage Service)

Object storage for anything — test artifacts, screenshots, logs, test data files.

Feature	Testing Use
Buckets	Store test reports, failure screenshots, log files
Versioning	Track changes to test data files over time

Feature	Testing Use
Lifecycle policies	Auto-delete old test artifacts after 30 days (cost control)
Pre-signed URLs	Generate temporary access links to test reports for stakeholders
Event notifications	Trigger a Lambda function when a new test report is uploaded

Common SDET tasks: - Upload test results to S3 after each CI/CD run. - Download test data files from S3 before test execution. - Verify that the application correctly handles S3 file uploads/downloads.

RDS (Relational Database Service)

Managed databases: PostgreSQL, MySQL, Oracle, SQL Server.

SDET relevance: - **Test database provisioning:** Spin up a dedicated RDS instance for test runs, tear it down afterward. - **Snapshots:** Create a snapshot of a known good database state. Restore it before each test run for consistent test data. - **Read replicas:** Test that the application correctly reads from replicas and writes to the primary. - **Connection testing:** Verify connection pooling, timeouts, and failover behaviour.

Udani's connection: Your CV lists PostgreSQL and Oracle experience. RDS is simply managed PostgreSQL/Oracle in the cloud — same SQL, same concepts, AWS handles the patching and backups.

DynamoDB

Fully managed NoSQL database. Key-value and document data model.

Aspect	What to Know
Tables, items, attributes	Core data model (no fixed schema)
Partition key / sort key	How data is distributed and queried
Provisioned vs on-demand capacity	Cost and performance model
DynamoDB Streams	Capture changes to items (for event-driven testing)

Testing relevance: If the application uses DynamoDB, test data modelling, query patterns, error handling for throttled requests, and consistency modes (eventually consistent vs strongly consistent reads).

---

6. Monitoring, Logging & Observability

CloudWatch

The central monitoring service. SDETs use it heavily.

Feature	SDET Use
Metrics	Monitor application performance during test runs (CPU, memory, request count, error rate)
Logs	View application logs to debug test failures — no need to SSH into servers
Alarms	Set up alerts if error rate exceeds a threshold during testing
Dashboards	Create test monitoring dashboards showing test execution metrics
Log Insights	Query and analyse logs with SQL-like syntax — useful for investigating intermittent failures

X-Ray

Distributed tracing for microservices. Traces a request as it flows through multiple services.

Testing relevance: - Debug slow API responses by identifying which downstream service is the bottleneck. - Verify that requests flow through the expected services. - Identify N+1 query patterns and unnecessary service calls.

CloudTrail

Logs every API call made in the AWS account. An audit trail.

Testing relevance: - Verify that security-sensitive actions are logged (who created/deleted a resource). - Test compliance requirements — financial regulators often require complete audit trails. - Investigate test environment issues — "who changed this configuration?"

---

7. Serverless & Event-Driven Architectures

Lambda

Run code without managing servers. You upload a function, AWS runs it in response to events.

Trigger	Testing Scenario
API Gateway request	Test the API endpoint that invokes the Lambda
S3 file upload	Test that processing triggers correctly when files are uploaded
SQS message	Test that messages are consumed and processed correctly
CloudWatch scheduled event	Test that scheduled tasks (cron jobs) execute as expected
DynamoDB Stream	Test that data changes trigger downstream processing

SDET testing considerations: - **Cold starts:** First invocation after idle period is slower. Test that cold start latency is acceptable. - **Timeouts:** Lambda has a maximum execution time (15 min). Test behaviour when functions approach the limit. - **Concurrency limits:** Test what happens when too many invocations occur simultaneously. - **Error handling:** Test retry behaviour for failed invocations (Lambda retries automatically for async invocations). - **Local testing:** Use AWS SAM (Serverless Application Model) or LocalStack to test Lambda functions locally before deploying.

SQS (Simple Queue Service) & SNS (Simple Notification Service)

Service	What It Does	Testing Focus
SQS	Message queue (decouple services)	Test message ordering, visibility timeout, dead letter queues, duplicate handling
SNS	Pub/sub notifications (fan-out)	Test that all subscribers receive messages, test filtering, test failure scenarios

SDET relevance for LSEG: Financial systems often use message queues for order processing, trade execution, and market data distribution. Testing message flow integrity is critical.

API Gateway

Managed service for creating, publishing, and managing APIs.

Testing focus: - Request validation and throttling. - Authentication (API keys, IAM, Cognito, Lambda authorizers). - Response caching behaviour. - Rate limiting — especially important for financial APIs. - CORS configuration.

---

8. Infrastructure as Code — Test Environment Provisioning

CloudFormation

AWS-native IaC tool. Defines infrastructure in YAML/JSON templates.

```yaml
# Example: A test environment definition
Resources:
  TestDatabase:
    Type: AWS::RDS::DBInstance
    Properties:
      DBInstanceClass: db.t3.micro
      Engine: postgres
      MasterUsername: testuser
      MasterUserPassword: !Ref DBPassword

  TestServer:
    Type: AWS::EC2::Instance
    Properties:
      InstanceType: t3.small
      ImageId: ami-0abcdef1234567890
```

SDET relevance: - **Ephemeral test environments:** Spin up a complete environment from a template before tests, tear it down after. Every test run gets a clean, identical environment. - **Environment parity:** Same template for dev, test, staging — only parameters differ (instance size, database name). - **Testing the infrastructure itself:** Verify that CloudFormation templates deploy correctly and produce the expected resources.

Terraform (not AWS-native but widely used)

Multi-cloud IaC tool. Many organisations (including those on AWS) prefer Terraform for its cloud-agnostic approach.

Same principles as CloudFormation but: - Uses HCL (HashiCorp Configuration Language) instead of YAML/JSON. - Can manage resources across AWS, Azure, GCP, and other providers. - Has a stronger community ecosystem and module registry.

Udani's connection: Your MSc in Cloud Native Computing almost certainly covered IaC. Mention it as a tool for ensuring consistent, reproducible test environments.

---

9. Performance & Load Testing on AWS

Running Load Tests on AWS

Approach	How It Works
Distributed JMeter on EC2	Run JMeter master + multiple slave instances across regions
k6 on EC2/ECS	Containerised k6 workers for scalable load generation
Gatling on CodeBuild	Serverless execution — no infrastructure to manage
AWS Distributed Load Testing	AWS solution using Fargate + Taurus for managed load tests

Key Considerations

1. **Test from the right region:** If LSEG serves European clients, run load tests from `eu-west-1`, not `us-east-1`. Latency matters.
2. **Avoid testing AWS limits, not your app:** AWS services have default quotas (e.g., API Gateway: 10,000 requests/sec). Know the difference between your application bottleneck and an AWS throttle.

3. **Cost awareness:** Large-scale load tests on EC2 can be expensive. Use spot instances for load generators (they're disposable anyway).
4. **Auto-scaling verification:** Load test to verify that Auto Scaling policies trigger correctly and the application handles scale-out/scale-in without dropping requests.
5. **Monitor during tests:** Use CloudWatch dashboards to watch metrics in real-time during load tests.

---

10. Security Testing on AWS

AWS Security Services an SDET Should Know

Service	Purpose	SDET Use
IAM Access Analyzer	Identifies resources shared externally	Verify no test resources are accidentally public
GuardDuty	Threat detection (monitors for malicious activity)	Verify alerts fire during security testing
Inspector	Vulnerability scanning for EC2 and containers	Include in CI/CD pipeline for automated vulnerability checks
Secrets Manager	Stores and rotates secrets (API keys, passwords)	Test code should retrieve credentials from here, never hardcode
KMS (Key Management Service)	Encryption key management	Verify data is encrypted at rest and in transit
WAF (Web Application Firewall)	Protects against common web exploits	Test that WAF rules block SQL injection, XSS, etc.
Config	Tracks resource configuration changes	Verify compliance rules are enforced

Security Testing Checklist for AWS Applications

- [ ] All data encrypted at rest (S3 bucket encryption, RDS encryption, EBS encryption).
- [ ] All data encrypted in transit (TLS/HTTPS, no HTTP endpoints).
- [ ] IAM roles follow least privilege — test with restricted roles and verify access is denied where expected.
- [ ] S3 buckets are not publicly accessible (unless explicitly intended).
- [ ] Security groups don't have overly permissive rules (no `0.0.0.0/0` on sensitive ports).
- [ ] Secrets are not hardcoded — pulled from Secrets Manager or Parameter Store.
- [ ] CloudTrail is enabled for audit logging.
- [ ] VPC flow logs are enabled for network monitoring.

---

11. Financial Services Context (LSEG-Specific)

LSEG operates critical financial infrastructure. AWS usage in financial services has specific requirements:

Compliance & Regulatory

Requirement	AWS Mechanism	Testing Focus
Data residency	Deploy in specific regions (e.g., EU data stays in EU regions)	Verify data doesn't leave required regions
Audit trails	CloudTrail + CloudWatch Logs	Verify all actions are logged and tamper-proof
Encryption	KMS + S3/RDS/EBS encryption	Verify encryption is enabled everywhere

Requirement	AWS Mechanism	Testing Focus
Disaster recovery	Multi-AZ deployments, cross-region replication	Test failover scenarios
Access controls	IAM, VPC, Security Groups	Test that only authorised users access sensitive data

High Availability & Disaster Recovery

Financial systems cannot have downtime. Testing DR scenarios is critical:

1. **Multi-AZ failover:** Kill a database in one AZ → verify automatic failover to standby AZ.
2. **Cross-region failover:** Simulate an entire region outage → verify traffic routes to DR region.
3. **RTO/RPO testing:** - **RTO (Recovery Time Objective):** How quickly must the system recover? Test that failover completes within the target. - **RPO (Recovery Point Objective):** How much data loss is acceptable? Test that replicated data is current.
4. **Chaos engineering:** Use AWS Fault Injection Simulator (FIS) to inject failures and verify resilience.

Real-Time Data Processing (LSEG Context)

LSEG's real-time market data systems likely use:

AWS Service	Purpose	Testing Focus
Kinesis Data Streams	Real-time data ingestion	Test message ordering, throughput, shard splitting
MSK (Managed Kafka)	Event streaming	Test consumer lag, partition rebalancing, exactly-once semantics
ElastiCache (Redis)	Low-latency caching	Test cache hit/miss ratios, TTL behaviour, failover
DynamoDB	Low-latency key-value lookups	Test read/write capacity, throttling behaviour

---

12. Quick Reference — Services Cheat Sheet

Services You MUST Know (High Priority)

Service	One-Line Description
IAM	Who can do what
EC2	Virtual servers
S3	Object/file storage
RDS	Managed relational databases
VPC	Your private network
CloudWatch	Monitoring and logging
Lambda	Serverless functions
ECS/Fargate	Container orchestration
CodeBuild/CodePipeline	CI/CD services
Secrets Manager	Secure credential storage

Services You Should Know (Medium Priority)

Service	One-Line Description
DynamoDB	NoSQL database
SQS/SNS	Message queuing and notifications
API Gateway	Managed API endpoints
CloudFormation	Infrastructure as code
ECR	Docker image registry
X-Ray	Distributed tracing
Route 53	DNS management
ELB/ALB	Load balancing

Services to Be Aware Of (Lower Priority)

Service	One-Line Description
Kinesis	Real-time data streaming
Step Functions	Workflow orchestration
CloudFront	CDN
ElastiCache	In-memory caching (Redis/Memcached)
CloudTrail	API audit logging
WAF	Web application firewall
Inspector	Vulnerability scanning
Config	Compliance monitoring

13. Interview Answer Templates

"How do you manage test environments in the cloud?"

> *"At Sysco LABS, our test environments were provisioned on AWS. I integrated our automation suites — Selenium with TestNG and REST Assured — into Jenkins pipelines that deployed and tested against AWS-hosted environments. The pipeline would trigger on code push, deploy to a test environment, execute the regression suite, and publish results.*
> *In my MSc in Cloud Native Computing, I went deeper into containerisation and infrastructure-as-code, which are the foundation for modern test environment management. The ideal approach is to define test environments as code — using CloudFormation or Terraform — so every CI/CD run gets an identical, fresh environment. Containers (Docker on ECS or Fargate) make this fast and cost-effective because you only pay for the compute time you use.*
> *For a financial institution like LSEG, I'd also emphasise environment isolation and security — test environments in private subnets, credentials stored in Secrets Manager, and strict IAM roles so test automation only has the access it needs."*

"How do you ensure test automation is secure on AWS?"

> *"There are several layers I'd focus on. First, **credentials management** — test automation should never use hardcoded AWS access keys. Instead, use IAM roles for EC2/ECS instances, or retrieve secrets from AWS Secrets Manager at runtime. Second, **least privilege** — the IAM role for test automation should only have permissions for what it needs (read from S3, write to RDS, etc.), nothing more. Third, **test data security** —*

> *especially in financial services, test environments should never contain real customer data. We use synthetic data generators and ensure test databases are encrypted. Finally,* ***network isolation*** *— test environments should be in their own VPC or subnets, with security groups restricting access."*

"What's your experience with AWS?"

> *"I have both practical and academic experience with AWS. At Sysco LABS, I integrated test automation suites into CI/CD pipelines using Jenkins deployed on AWS, with test environments hosted on AWS infrastructure. I worked with EC2 instances, S3 for artifact storage, and AWS-integrated Jenkins for pipeline orchestration.*
> *My MSc in Cloud Native Computing gave me deeper knowledge of cloud architecture — containerisation with Docker and Kubernetes, infrastructure as code, microservices testing, and cloud-native CI/CD patterns. I understand how services like VPC, IAM, CloudWatch, and RDS work together to support a testing infrastructure.*
> *While my primary hands-on experience is with the CI/CD and compute layer, I'm confident in my understanding of the broader AWS ecosystem and how it supports quality engineering at scale."*

"How would you test a microservices application deployed on AWS?"

> *"I'd approach this in layers. First,* ***individual service testing*** *— each microservice gets its own unit and integration tests, running in containers on ECS or CodeBuild. Dependencies are mocked or stubbed using tools like WireMock or LocalStack.*
> *Second,* ***contract testing*** *— since microservices communicate via APIs, I'd implement Pact or similar contract tests to ensure services agree on their interfaces without needing to deploy everything together.*
> *Third,* ***end-to-end testing*** *in a staging environment — deploy all services to a test environment and run key user journeys. Use AWS X-Ray for distributed tracing to debug any issues across service boundaries.*
> *Fourth,* ***resilience testing*** *— use AWS Fault Injection Simulator to test how the system handles failures: what happens if a downstream service goes down? Does the circuit breaker activate? Does the system degrade gracefully?*
> *Finally,* ***monitoring-based testing*** *— even after deployment, CloudWatch alarms and dashboards verify the system is healthy. This is especially important for financial systems where real-time data integrity is critical."*

Focus your study on Sections 1–6 and 11 — these are the most likely interview topics given the LSEG role and the interviewers' infrastructure backgrounds. Sections 7–10 are valuable for demonstrating depth if the conversation goes there.

DevOps Questions

Context: LSEG Associate Lead SDET role — the JD places this role "as a member of a DevOps team" and requires CI/CD, cloud, and infrastructure familiarity.

Table of Contents

1. DevOps Culture & Principles
2. Version Control & Branching
3. CI/CD Pipelines
4. Containerisation & Orchestration
5. Infrastructure as Code
6. Configuration Management
7. Monitoring, Logging & Alerting
8. Networking & Linux
9. Cloud & Scalability
10. DevOps for Quality Engineering

1. DevOps Culture & Principles

Q1.1: What is DevOps and why does it matter?

DevOps is a set of practices, cultural philosophies, and tools that unify software **development** (Dev) and IT **operations** (Ops) to shorten the development lifecycle and deliver high-quality software continuously.

Core pillars (CALMS):

Pillar	Meaning
C — Culture	Shared responsibility between dev, ops, and QA; no "throw it over the wall"
A — Automation	Automate builds, tests, deployments, and infrastructure
L — Lean	Eliminate waste, reduce batch sizes, deliver incrementally
M — Measurement	Measure everything — lead time, deployment frequency, MTTR, change failure rate
S — Sharing	Shared tools, knowledge, and visibility across teams

Why it matters for SDETs: In a DevOps team, the SDET is not a gatekeeper at the end of the pipeline. You're embedded in the team, contributing to CI/CD pipelines, writing automated quality gates, and ensuring that the speed of delivery doesn't compromise quality.

Q1.2: What are the four key DORA metrics?

The **DORA (DevOps Research and Assessment)** metrics measure software delivery performance:

Metric	What It Measures	Elite Performance
Deployment Frequency	How often code is deployed to production	Multiple times per day
Lead Time for Changes	Time from commit to production deployment	Less than 1 hour

Metric	What It Measures	Elite Performance
Change Failure Rate	Percentage of deployments causing failures	0–15%
Mean Time to Recovery (MTTR)	Time to restore service after a failure	Less than 1 hour

SDET connection: Your test automation directly impacts all four metrics: - Fast, reliable tests → shorter lead time. - Comprehensive tests → lower change failure rate. - Good test observability → faster MTTR (tests help pinpoint what broke). - Stable CI/CD → enables higher deployment frequency.

Q1.3: What is the difference between DevOps, SRE, and Platform Engineering?

Discipline	Focus	Key Responsibility
DevOps	Culture and practices for fast, reliable delivery	CI/CD, automation, breaking silos between dev and ops
SRE (Site Reliability Engineering)	Reliability of production systems	SLOs, error budgets, incident response, toil reduction
Platform Engineering	Building self-service internal platforms	Developer experience, internal tooling, golden paths

Overlap: All three value automation and measurement. An SDET in a DevOps team may interact with SRE practices (e.g., testing against SLOs) and platform engineering (e.g., using an internal test infrastructure platform).

Q1.4: Explain the concept of “shift-left” and “shift-right” in DevOps.

Direction	Meaning	Practices
Shift-left	Move testing and quality earlier in the lifecycle	TDD, BDD, static analysis, code reviews, unit tests, contract tests
Shift-right	Extend testing and monitoring into production	Canary deployments, feature flags, A/B testing, production monitoring, chaos engineering

Both together: Shift-left catches bugs early (cheaper to fix). Shift-right catches issues that only appear in production (real traffic patterns, edge cases, infrastructure differences). A mature DevOps team does both.

Q1.5: What is “Infrastructure as Code” and why is it a DevOps principle?

Infrastructure as Code (IaC) means managing and provisioning infrastructure through machine-readable configuration files rather than manual processes.

Why it’s a DevOps principle: - **Reproducibility:** Spin up identical environments every time — eliminates “works on my machine.” - **Version control:** Infrastructure definitions live in Git alongside application code. Changes are reviewed, tracked, and auditable. - **Automation:** Environments can be created/destroyed automatically in CI/CD pipelines. - **Self-service:** Developers and testers can provision their own environments without waiting for an ops team. - **Drift detection:** Compare the actual state of infrastructure against the desired state defined in code.

Tools: Terraform, AWS CloudFormation, Pulumi, Ansible, Chef, Puppet.

---

2. Version Control & Branching

Q2.1: Explain Git branching strategies. Which would you recommend and why?

Strategy	How It Works	Best For
Git Flow	Long-lived `develop` and `main` branches; feature, release, and hotfix branches	Scheduled releases, larger teams, strict release process
GitHub Flow	Single `main` branch; short-lived feature branches merged via PRs	Continuous deployment, small teams, SaaS products
Trunk-Based Development	Everyone commits to `main` (trunk) directly or via very short-lived branches (< 1 day)	High-performing DevOps teams, continuous deployment
GitLab Flow	Environment branches (`staging` , `production`) alongside feature branches	Teams that need environment-specific deployments

Recommendation for an SDET: - For a financial institution like LSEG with strict release processes → **Git Flow** or **GitLab Flow** (structured, auditable). - For test automation repository specifically → **GitHub Flow** (simpler, feature branches merged after review).

Q2.2: What is a merge conflict and how do you resolve it?

A **merge conflict** occurs when two branches modify the same lines of a file, and Git cannot automatically determine which change to keep.

Resolution process: 1. Git marks conflicts in the file with `<<<<<<<` , `=======` , `>>>>>>>` markers. 2. Open the file and understand both changes. 3. Decide which change to keep, or combine both. 4. Remove the conflict markers. 5. Stage the resolved file and commit.

Prevention strategies: - Pull frequently from the main branch to stay current. - Keep branches short-lived — merge within 1–2 days. - Communicate with teammates about who's working on what. - Use code ownership / CODEOWNERS files to route reviews appropriately.

Q2.3: What are Git hooks and how are they useful in DevOps?

Git hooks are scripts that run automatically at specific points in the Git workflow.

Hook	When It Runs	DevOps Use
`pre-commit`	Before a commit is created	Run linters, formatters, static analysis; block commits with secrets
`commit-msg`	After commit message is written	Enforce commit message format (e.g., Jira ticket prefix)
`pre-push`	Before pushing to remote	Run unit tests; block pushes to protected branches
`post-merge`	After a merge completes	Notify CI/CD system, rebuild dependencies
`pre-receive` (server-side)	Before accepting a push on the server	Enforce branch protection, run security checks

Tools that leverage hooks: Husky (Node.js), pre-commit (Python), Lefthook (polyglot).

Q2.4: What is the difference between `git rebase` and `git merge`?

Aspect	`git merge`	`git rebase`
History	Preserves all branch history with a merge commit	Rewrites history to create a linear sequence
Merge commits	Creates a merge commit	No merge commit — commits are replayed on top of the target branch
Safety	Safe for shared branches	Dangerous on shared branches (rewrites public history)
Use case	Merging feature branches into main	Keeping a feature branch up to date with main before merging

Golden rule: Never rebase commits that have been pushed to a shared branch. Rebase is for cleaning up your local history before merging.

3. CI/CD Pipelines

Q3.1: Design a CI/CD pipeline for a microservices application. What stages would you include?

```
┌──────┐   ┌──────────┐   ┌──────────┐   ┌──────────┐   ┌──────────┐   ┌──────────┐
┌──────────┐
│ Code │──▶│  Static  │──▶│  Build   │──▶│   Unit   │──▶│Integration│──▶│ Security │──▶│
Deploy  │
│ Push │   │ Analysis │   │ & Package│   │  Tests   │   │  Tests   │   │  Scans   │   │
Staging  │
└──────┘   └──────────┘   └──────────┘   └──────────┘   └──────────┘   └──────────┘
└──────────┘
                                                                                        │
            ┌──────────┐   ┌──────────┐   ┌──────────┐                                 │
            │  Deploy  │◀──│ Manual   │◀──│   E2E &
│◀──────────────────────────────────────┘
            │Production│   │ Approval │   │  Smoke   │
            └──────────┘   └──────────┘   └──────────┘
```

Stage details:

Stage	What Happens	Gate
Static Analysis	Linting, code style, SonarQube, type checking	Fail on critical issues
Build & Package	Compile code, build Docker image, push to registry	Build must succeed
Unit Tests	Run unit test suite	All must pass; coverage ≥ threshold
Integration Tests	Test service interactions, database queries, API contracts	All must pass
Security Scans	SAST, dependency scanning (Snyk/Trivy), secret detection	Fail on critical/high CVEs
Deploy to Staging	Deploy to a pre-production environment	Deployment must succeed
E2E & Smoke Tests	Run end-to-end tests against staging	All must pass
Manual Approval	Human gate for production deployment	Required for financial systems
Deploy to Production	Blue/green or canary deployment	Health checks must pass

Q3.2: What is the difference between blue/green deployment and canary deployment?

Blue/Green Deployment:

```
                    ┌──────────────┐
                    │ Load         │
                    │ Balancer     │
                    └──────────────┘
                           │
                ┌──────────┼──────────┐
                ▼                     ▼
┌────────────────┐       ┌────────────────┐
│  Blue (v1)     │       │  Green (v2)    │
│  (current)     │       │  (new)         │
└────────────────┘       └────────────────┘
```

- Two identical environments. The load balancer switches all traffic from blue (old) to green (new) at once.
- **Rollback:** Switch the load balancer back to blue. Instant.
- **Drawback:** Requires double the infrastructure. All-or-nothing switch — if v2 has a subtle bug, all users are affected.

Canary Deployment:

```
100% traffic          90% → v1,  10% → v2         100% → v2

┌──────┐             ┌──────┐  ┌──────┐          ┌──────┐
│ v1   │    ──▶      │ v1   │  │ v2   │   ──▶    │ v2   │
└──────┘             └──────┘  └──────┘          └──────┘
```

- Route a small percentage of traffic to the new version. Gradually increase if metrics look good.
- **Rollback:** Route all traffic back to v1. Only a small percentage of users were affected.
- **Advantage:** Real-world validation with limited blast radius.

For LSEG (financial systems): Canary deployments are preferred because they limit risk. A bug in a trading system affecting 100% of users could have catastrophic financial impact.

SDET role: Write automated health checks and smoke tests that run against the canary. If error rates or latency increase, the canary is automatically rolled back.

Q3.3: How do you handle secrets in a CI/CD pipeline?

Never do: - Hardcode secrets in source code. - Store secrets in plain text config files committed to Git. - Pass secrets as plain text environment variables visible in CI logs.

Best practices:

Approach	Tool	How It Works
Secret management service	AWS Secrets Manager, HashiCorp Vault, Azure Key Vault	Secrets stored encrypted, retrieved at runtime via API
CI/CD built-in secrets	Jenkins Credentials, GitHub Secrets, GitLab CI Variables	Secrets configured in the CI platform, injected as masked env vars
Environment-specific configs	AWS Parameter Store, Consul	Different secrets per environment (dev/staging/prod)
Secret scanning	gitleaks, truffleHog, git-secrets	Pre-commit hooks that block commits containing secrets

SDET concern: Test automation often needs credentials (database passwords, API keys, service accounts). Store these in the CI platform's secret store and inject them at runtime. Never log them.

Q3.4: What is a pipeline-as-code? Give an example.

Pipeline-as-code defines CI/CD pipelines in version-controlled configuration files rather than through a UI.

Benefits: - Versioned alongside application code — changes to the pipeline are reviewed and tracked. - Reproducible — the pipeline definition is the same across environments. - Portable — moving to a new CI server means copying the config file.

Examples by tool:

Jenkinsfile (Declarative):

```
pipeline {
    agent any
    stages {
        stage('Build') {
            steps {
                sh 'mvn clean compile'
            }
        }
        stage('Test') {
            steps {
                sh 'mvn test'
            }
            post {
                always {
                    junit 'target/surefire-reports/*.xml'
                }
            }
        }
        stage('Deploy') {
            when { branch 'main' }
            steps {
                sh 'deploy.sh staging'
            }
        }
    }
}
```

GitHub Actions:

```yaml
name: CI
on: [push, pull_request]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: npm install
      - run: npm test
```

GitLab CI (.gitlab-ci.yml):

```yaml
stages:
  - test
  - deploy

unit-tests:
  stage: test
  script:
    - pytest tests/

deploy-staging:
  stage: deploy
  script:
    - ./deploy.sh staging
```

```
  only:
    - main
```

Q3.5: What is artifact management and why is it important?

An **artifact** is any output of the build process — compiled binaries, Docker images, test reports, JAR/WAR files, npm packages.

Artifact management is the practice of storing, versioning, and distributing these artifacts reliably.

Tool	What It Stores
JFrog Artifactory	Maven/Gradle JARs, Docker images, npm packages, generic files
Nexus Repository	Same as Artifactory — Maven, npm, Docker, PyPI
AWS ECR	Docker images
AWS S3	Generic artifacts (test reports, binaries)
GitHub Packages	npm, Maven, Docker, NuGet

Why it matters: - **Reproducibility:** Deploy the exact same artifact that was tested, not a rebuild. - **Traceability:** Every artifact is tagged with its Git commit, build number, and test results. - **Speed:** Download pre-built artifacts instead of rebuilding from source. - **Rollback:** Previous versions are stored and can be redeployed instantly.

SDET relevance: Store test reports, failure screenshots, and performance baselines as artifacts. When a test fails in CI, the artifact (screenshot, log) is the first thing you examine.

Q3.6: How do you implement quality gates in a CI/CD pipeline?

A **quality gate** is a checkpoint in the pipeline that must pass before proceeding to the next stage.

Gate	Criteria	Tool
Code quality	No critical SonarQube issues, code coverage ≥ 80%	SonarQube, CodeClimate
Unit tests	100% pass rate	JUnit, TestNG, pytest
Integration tests	100% pass rate	REST Assured, Testcontainers
Security	No critical/high vulnerabilities	Snyk, Trivy, OWASP Dependency-Check
Performance	Response time < threshold, no regression > 10%	k6, Gatling, JMeter
E2E tests	All critical paths pass	Selenium, Playwright, Cypress
Manual approval	Human sign-off for production	Jenkins input step, GitHub environment protection

Implementation pattern:

```
Stage → Run checks → Evaluate against criteria → Pass? → Next stage
                                                 → Fail? → Block pipeline + notify team
```

SDET ownership: The SDET typically owns the test-related quality gates — defining the criteria, maintaining the tests, and ensuring gates are reliable (not flaky).

---

4. Containerisation & Orchestration

Q4.1: What is Docker and why is it important for DevOps?

Docker packages an application and its dependencies into a **container** — a lightweight, portable unit that runs consistently anywhere.

Key concepts:

Concept	Description
Image	A read-only template with the application + dependencies. Built from a `Dockerfile`.
Container	A running instance of an image. Isolated process with its own filesystem and network.
Dockerfile	Instructions to build an image (`FROM`, `COPY`, `RUN`, `CMD`).
Registry	Where images are stored (Docker Hub, AWS ECR, private registries).
Volume	Persistent storage that survives container restarts.
Network	Virtual networks connecting containers together.

Why it matters for DevOps: - **Consistency:** "It works on my machine" is eliminated — the container is the same everywhere. - **Isolation:** Each service runs in its own container, preventing dependency conflicts. - **Speed:** Containers start in seconds (vs. minutes for VMs). - **Scalability:** Spin up multiple container instances behind a load balancer.

SDET use cases: - Run test suites inside containers for consistent execution. - Use Docker Compose to spin up the application + database + message queue for integration testing. - Run Selenium Grid as containers (selenium/hub + selenium/node-chrome).

Q4.2: Write a Dockerfile for a test automation project.

```
# Base image with Java (for TestNG/Selenium projects)
FROM maven:3.9-eclipse-temurin-17

# Install Chrome for Selenium tests
RUN apt-get update && apt-get install -y \
    wget \
    gnupg2 \
    && wget -q -O - https://dl.google.com/linux/linux_signing_key.pub | apt-key add - \
    && echo "deb http://dl.google.com/linux/chrome/deb/ stable main" >> /etc/apt/
sources.list.d/google.list \
    && apt-get update \
    && apt-get install -y google-chrome-stable \
    && rm -rf /var/lib/apt/lists/*

# Set working directory
WORKDIR /tests

# Copy project files
COPY pom.xml .
COPY src/ ./src/

# Download dependencies (cached layer if pom.xml hasn't changed)
RUN mvn dependency:resolve

# Default command: run tests
CMD ["mvn", "test", "-Dbrowser=chrome-headless"]
```

Key practices: - **Layer caching:** Copy `pom.xml` first, then `src/`. Dependencies are only re-downloaded if `pom.xml` changes. - **Headless browser:** Run Chrome in headless mode inside the container (no display). - **Clean up:** Remove apt caches to keep image size small.

Q4.3: What is Docker Compose and how do you use it for testing?

Docker Compose defines and runs multi-container applications using a YAML file.

```yaml
version: '3.8'
services:
  app:
    build: ./app
    ports:
      - "8080:8080"
    environment:
      - DB_HOST=db
      - DB_PORT=5432
    depends_on:
      db:
        condition: service_healthy

  db:
    image: postgres:15
    environment:
      POSTGRES_DB: testdb
      POSTGRES_USER: testuser
      POSTGRES_PASSWORD: testpass
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U testuser"]
      interval: 5s
      timeout: 5s
      retries: 5

  tests:
    build: ./tests
    depends_on:
      app:
        condition: service_started
    environment:
      - BASE_URL=http://app:8080
```

Testing workflow:

```bash
# Start everything, run tests, tear down
docker compose up --abort-on-container-exit --exit-code-from tests
docker compose down -v
```

Benefits: The entire application stack (app + database + tests) is defined in one file. Anyone can run the full test suite with a single command — no environment setup needed.

Q4.4: What is Kubernetes and why does an SDET need to know it?

Kubernetes (K8s) is a container orchestration platform that automates deployment, scaling, and management of containerised applications.

Core concepts:

Concept	Description
Pod	The smallest deployable unit — one or more containers sharing network/storage
Deployment	Manages a set of identical pods, handles rolling updates

Concept	Description
Service	Stable network endpoint to access a set of pods
Namespace	Logical isolation within a cluster (e.g., `dev`, `staging`, `prod`)
ConfigMap / Secret	Externalised configuration and sensitive data
Ingress	HTTP/HTTPS routing from external traffic to services
Horizontal Pod Autoscaler	Automatically scales pods based on CPU/memory or custom metrics

Why SDETs need it:

1. **Testing K8s-deployed applications:** Verify rolling updates don't cause downtime, health checks work, pods restart on failure.
2. **Test infrastructure on K8s:** Run Selenium Grid, test databases, and test agents as K8s workloads.
3. **Ephemeral test environments:** Create a namespace per feature branch, deploy the application, run tests, delete the namespace.
4. **Chaos testing:** Kill pods, drain nodes, inject network latency — verify the application is resilient.

Q4.5: What is the difference between a Docker container and a virtual machine?

Aspect	Container	Virtual Machine
Isolation	Process-level (shares host OS kernel)	Hardware-level (runs its own OS kernel)
Size	Megabytes (app + dependencies only)	Gigabytes (full OS + app + dependencies)
Startup time	Seconds	Minutes
Resource usage	Lightweight, many containers per host	Heavy, fewer VMs per host
Portability	Highly portable (runs anywhere Docker runs)	Less portable (hypervisor-dependent)
Use case	Microservices, CI/CD, test execution	Legacy applications, OS-level isolation, multi-OS environments

When to use VMs for testing: When you need to test on a specific OS (e.g., Windows Server testing on a Linux host), or when the application requires kernel-level features not available in containers.

Q4.6: What is a container registry and how do you manage image security?

A **container registry** stores and distributes Docker images.

Registry	Type
Docker Hub	Public/private
AWS ECR	Private (AWS-native)
Google Artifact Registry	Private (GCP-native)
Azure Container Registry	Private (Azure-native)
Harbor	Self-hosted, open-source
JFrog Artifactory	Universal artifact management

Image security practices:

1. **Use minimal base images:** `alpine` or `distroless` instead of `ubuntu` — smaller attack surface.

2. **Scan for vulnerabilities:** Use Trivy, Snyk, or AWS ECR scanning to detect known CVEs in image layers.
3. **Don't run as root:** Use `USER nonroot` in Dockerfile.
4. **Pin image versions:** Use `node:18.17.0-alpine` instead of `node:latest` — reproducible builds.
5. **Sign images:** Use Docker Content Trust or Sigstore to verify image integrity.
6. **Secrets never in images:** Never `COPY` secret files or set credentials as `ENV` in the Dockerfile.

---

5. Infrastructure as Code

Q5.1: Compare Terraform and CloudFormation.

Aspect	Terraform	CloudFormation
Provider	HashiCorp (open-source)	AWS (native)
Cloud support	Multi-cloud (AWS, Azure, GCP, etc.)	AWS only
Language	HCL (HashiCorp Configuration Language)	YAML or JSON
State management	External state file (S3, Terraform Cloud)	Managed by AWS automatically
Drift detection	`terraform plan` shows drift	Built-in drift detection
Modularity	Modules from Terraform Registry	Nested stacks, modules
Preview changes	`terraform plan`	Change sets
Rollback	Manual (apply previous state)	Automatic rollback on failure

When to choose: - **Terraform:** Multi-cloud environments, team prefers HCL, need to manage non-AWS resources. - **CloudFormation:** Pure AWS environment, want native integration, prefer AWS-managed state.

Q5.2: What is Terraform state and why is it important?

Terraform state is a file (`terraform.tfstate`) that maps your Terraform configuration to real-world resources. It tracks what Terraform has created so it knows what to update or delete.

Why it matters: - Without state, Terraform doesn't know which resources it manages — it would try to create duplicates. - State enables `terraform plan` to show what will change before applying. - State contains sensitive data (resource IDs, outputs, sometimes passwords).

Best practices: - **Remote state:** Store state in S3 (with DynamoDB locking) or Terraform Cloud — never in local files for team projects. - **State locking:** Prevent two people from applying changes simultaneously (DynamoDB lock table). - **State encryption:** Enable S3 server-side encryption for the state bucket. - **Never edit state manually:** Use `terraform state mv` , `terraform import` , or `terraform state rm` .

Q5.3: How do you test Infrastructure as Code?

Level	What to Test	Tool
Static analysis	Syntax errors, security misconfigurations, best practice violations	`tflint` , `checkov` , `tfsec` , `cfn-lint`
Unit testing	Individual module logic (does this module create the right resources?)	Terratest (Go), `terraform validate` , `cfn-guard`
Integration testing	Deploy infrastructure to a test account, verify it works, tear it down	Terratest, Kitchen-Terraform, `taskcat` (CloudFormation)
Compliance testing		

Level	What to Test	Tool
	Verify infrastructure meets security/compliance policies	Open Policy Agent (OPA), Sentinel (Terraform Enterprise), AWS Config Rules
Drift detection	Verify actual infrastructure matches the defined state	`terraform plan` (should show no changes if in sync)

SDET angle: Testing IaC is a natural extension of the SDET role — you're applying the same principles (automated validation, shift-left, quality gates) to infrastructure.

---

6. Configuration Management

Q6.1: What is configuration management and which tools are used?

Configuration management ensures that systems are configured consistently and maintained in a desired state.

Tool	Approach	Language	Agentless?
Ansible	Procedural (do this, then that)	YAML (playbooks)	Yes (SSH-based)
Chef	Declarative (desired state)	Ruby (recipes)	No (agent on each node)
Puppet	Declarative	Puppet DSL (manifests)	No (agent on each node)
Salt	Both	YAML/Python	Both (agent or SSH)

Ansible is the most common choice today because it's agentless, uses simple YAML, and can handle both configuration and orchestration.

SDET relevance: - Provision test environments with the correct software versions and configurations. - Verify server configurations (e.g., ensure the test database has the correct settings). - Automate test environment setup/teardown.

Q6.2: What is the difference between mutable and immutable infrastructure?

Aspect	Mutable	Immutable
Updates	Modify existing servers in-place (install patches, change configs)	Never modify existing servers — build new ones and replace
Drift risk	High — servers diverge over time ("snowflake servers")	None — every deployment is a fresh, identical image
Rollback	Complex — undo changes on live servers	Simple — deploy the previous image
Tools	Ansible, Chef, Puppet (configuration management)	Packer + Terraform, Docker, AMI-based deployments
Analogy	Repairing a car part by part	Replacing the entire car with a new one

Modern DevOps favours immutable infrastructure. Build a new server image (AMI, Docker image) with every change. Deploy the new image. If something breaks, deploy the previous image.

Testing benefit: Immutable infrastructure guarantees that the test environment is identical every time — no configuration drift between test runs.

---

7. Monitoring, Logging & Alerting

Q7.1: What are the three pillars of observability?

Pillar	What It Provides	Tools
Metrics	Quantitative measurements over time (CPU, request count, error rate)	Prometheus, Grafana, CloudWatch, Datadog
Logs	Detailed event records (structured text with timestamps)	ELK Stack (Elasticsearch, Logstash, Kibana), Fluentd, CloudWatch Logs, Splunk
Traces	End-to-end request flow across services	Jaeger, Zipkin, AWS X-Ray, Datadog APM

Why SDETs care: - **Metrics** tell you if something is wrong (error rate spiked). - **Logs** tell you what went wrong (error message, stack trace). - **Traces** tell you where it went wrong (which service in the chain failed).

When a test fails in CI/CD, the first debugging step is checking logs and metrics — not re-running the test.

Q7.2: What is the ELK Stack?

ELK = **Elasticsearch** + **Logstash** + **Kibana** (now often called the Elastic Stack, with Beats added).

Component	Role
Elasticsearch	Search and analytics engine — stores and indexes logs
Logstash	Data processing pipeline — collects, transforms, and sends logs to Elasticsearch
Kibana	Visualisation dashboard — search logs, create charts, set up alerts
Beats	Lightweight agents that ship data (Filebeat for logs, Metricbeat for metrics)

Flow:

```
Application → Filebeat → Logstash → Elasticsearch → Kibana
```

SDET use: Query application logs during test execution to correlate test actions with server-side behaviour. Set up Kibana dashboards showing test execution metrics.

Q7.3: What is Prometheus and Grafana?

Prometheus is an open-source monitoring system that collects metrics via a pull model (scrapes HTTP endpoints).

Grafana is a visualisation platform that creates dashboards from Prometheus (and other) data sources.

How they work together:

```
Application ──▶ /metrics endpoint ──▶ Prometheus (scrapes & stores) ──▶ Grafana (visualises)
```

Key Prometheus concepts: - **Metrics types:** Counter (monotonically increasing), Gauge (can go up or down), Histogram (distribution), Summary. - **PromQL:** Query language for metrics (`rate(http_requests_total[5m])`). - **Alertmanager:** Routes alerts to Slack, email, PagerDuty based on Prometheus rules.

SDET relevance: - Monitor application metrics during performance tests. - Create Grafana dashboards for test execution metrics (pass rate over time, test duration trends). - Set up alerts for metric thresholds (response time > 500ms triggers an alert during load testing).

Q7.4: What is structured logging and why does it matter?

Unstructured log:

```
2024-03-15 10:30:45 ERROR - Failed to process order 12345 for user john@example.com
```

Structured log (JSON):

```
{
  "timestamp": "2024-03-15T10:30:45Z",
  "level": "ERROR",
  "message": "Failed to process order",
  "orderId": "12345",
  "userId": "john@example.com",
  "service": "order-service",
  "traceId": "abc-123-def",
  "error": "PaymentDeclined",
  "duration_ms": 230
}
```

Why structured logging matters: - **Searchable:** Query by any field (`orderId=12345 AND level=ERROR`). - **Parseable:** Machines can process structured logs automatically. - **Correlatable:** Use `traceId` to follow a request across services. - **Aggregatable:** Calculate error rates, average durations, counts per service.

SDET impact: When debugging a test failure, structured logs let you filter by test run ID or trace ID and see exactly what happened server-side during your test execution.

8. Networking & Linux

Q8.1: Explain the essential Linux commands an SDET should know.

Command	Purpose	Example
`ssh`	Connect to remote servers	`ssh user@test-server.internal`
`scp` / `rsync`	Copy files to/from remote servers	`scp report.html user@server:/tmp/`
`curl`	Make HTTP requests from the command line	`curl -X GET https://api.example.com/health`
`grep`	Search text in files/output	`grep -r "ERROR" /var/log/app/`
`tail -f`	Follow log files in real time	`tail -f /var/log/app/application.log`
`ps aux`	List running processes	`ps aux | grep java`
`top` / `htop`	Monitor CPU and memory usage	`htop`
`netstat` / `ss`	View network connections	`ss -tlnp` (listening ports)
`df -h`	Check disk space	`df -h /`
`chmod` / `chown`	Change file permissions/ownership	`chmod +x run-tests.sh`
`systemctl`	Manage services	`systemctl restart nginx`
`journalctl`	View systemd logs	`journalctl -u app-service -f`

Command	Purpose	Example
`awk` / `sed`	Text processing	`awk '{print $1}' access.log`
`crontab`	Schedule recurring tasks	`crontab -e`

Q8.2: What is the difference between TCP and UDP?

Aspect	TCP	UDP
Connection	Connection-oriented (three-way handshake)	Connectionless
Reliability	Guaranteed delivery, ordering, retransmission	No guarantee — packets may be lost or arrive out of order
Speed	Slower (overhead for reliability)	Faster (minimal overhead)
Use case	HTTP/HTTPS, SSH, database connections	DNS lookups, video streaming, real-time market data feeds
Header size	20 bytes minimum	8 bytes

LSEG relevance: Real-time market data feeds often use UDP (or multicast UDP) for speed. Trade execution uses TCP for reliability. An SDET testing LSEG systems may need to understand both protocols.

Q8.3: What is DNS and how does it work?

DNS (Domain Name System) translates human-readable domain names (e.g., `api.lseg.com`) to IP addresses (e.g., `52.84.123.45`).

Resolution flow:

```
Browser → Local cache → OS cache → Router cache → ISP DNS → Root DNS →
TLD DNS (.com) → Authoritative DNS (lseg.com) → Returns IP
```

Key record types:

Record	Purpose	Example
A	Maps domain to IPv4 address	`api.lseg.com → 52.84.123.45`
AAAA	Maps domain to IPv6 address	`api.lseg.com → 2001:db8::1`
CNAME	Alias for another domain	`www.lseg.com → lseg.com`
MX	Mail server	`lseg.com → mail.lseg.com`
TXT	Text records (SPF, DKIM, verification)	Domain verification strings
NS	Name server for the domain	`lseg.com → ns1.awsdns.com`

Testing relevance: - Verify DNS resolution works correctly for test environments. - Test failover — when primary DNS record changes, does traffic route correctly? - Test TTL (Time to Live) — how quickly do DNS changes propagate? - DNS-based load balancing (Route 53 weighted routing).

Q8.4: What happens when you type a URL in a browser? (Common interview question)

1. **DNS resolution:** Browser resolves the domain name to an IP address.
2. **TCP connection:** Browser establishes a TCP connection with the server (three-way handshake: SYN → SYN-ACK → ACK).

3. **TLS handshake:** If HTTPS, negotiate encryption (exchange certificates, agree on cipher suite, generate session keys).
4. **HTTP request:** Browser sends an HTTP GET request with headers (User-Agent, Accept, cookies).
5. **Server processing:** Server receives the request, processes it (query database, run business logic), generates a response.
6. **HTTP response:** Server sends back the response (status code, headers, HTML body).
7. **Rendering:** Browser parses HTML, requests CSS/JS/images (parallel requests), builds the DOM, paints the page.
8. **JavaScript execution:** Browser executes JavaScript, which may make additional API calls (AJAX/fetch).

SDET angle: Each step is a potential failure point you can test — DNS misconfiguration, certificate expiry, slow server processing, broken rendering, JavaScript errors.

---

9. Cloud & Scalability

Q9.1: What is horizontal vs vertical scaling?

Aspect	Vertical Scaling (Scale Up)	Horizontal Scaling (Scale Out)
Method	Add more resources to a single machine (more CPU, RAM)	Add more machines behind a load balancer
Limit	Hardware ceiling (can't infinitely upgrade one server)	Virtually unlimited (add as many instances as needed)
Downtime	Usually requires restart	No downtime (add instances while running)
Complexity	Simple (same application, bigger machine)	Complex (need stateless design, load balancing, data synchronisation)
Cost	Exponentially expensive at the top end	Linear cost scaling
Example	Upgrade EC2 from `t3.medium` to `t3.xlarge`	Add more `t3.medium` instances behind an ALB

SDET testing considerations: - **Vertical:** Test application behaviour before and after scaling. Verify it utilises added resources. - **Horizontal:** Test load balancer health checks. Test session management (sticky sessions vs. stateless). Test data consistency across instances. Test auto-scaling triggers.

Q9.2: What is a load balancer and what types exist?

A **load balancer** distributes incoming traffic across multiple servers to ensure no single server is overwhelmed.

AWS Load Balancer types:

Type	Layer	Best For
Application Load Balancer (ALB)	Layer 7 (HTTP/HTTPS)	Web applications, API routing, path-based routing
Network Load Balancer (NLB)	Layer 4 (TCP/UDP)	High-performance, low-latency, real-time data
Gateway Load Balancer (GLB)	Layer 3	Third-party virtual appliances (firewalls, inspection)
Classic Load Balancer (CLB)	Layer 4/7	Legacy (avoid for new applications)

Testing focus: - **Health checks:** Verify the load balancer correctly detects unhealthy instances and stops routing traffic to them. - **Routing rules:** Test path-based routing (`/api/*` → API servers, `/static/*` → CDN). - **SSL**

termination: Verify HTTPS is handled correctly at the load balancer. - **Sticky sessions:** If enabled, verify the same user hits the same server. - **Failover:** Kill a backend instance and verify traffic seamlessly shifts.

Q9.3: What is a CDN and how do you test caching behaviour?

A **Content Delivery Network (CDN)** caches content at edge locations worldwide, serving users from the nearest location to reduce latency.

AWS CloudFront is AWS's CDN service.

Testing caching behaviour: 1. **Cache hit/miss:** Check response headers (`X-Cache: Hit from cloudfront` vs `Miss from cloudfront`). 2. **TTL (Time to Live):** Verify content expires and refreshes after the configured TTL. 3. **Cache invalidation:** After deploying new content, verify invalidation clears stale cache. 4. **Cache-Control headers:** Verify the application sets appropriate headers (`Cache-Control: max-age=3600`, `no-cache`, `no-store`). 5. **Dynamic vs static content:** Verify dynamic content (API responses) is not cached when it shouldn't be. 6. **Geographic testing:** Verify users in different regions receive content from their nearest edge location.

Q9.4: What are microservices and how do they differ from monoliths?

Aspect	Monolith	Microservices
Structure	Single deployable unit	Multiple independently deployable services
Scaling	Scale the entire application	Scale individual services independently
Deployment	All-or-nothing deployment	Deploy services independently
Technology	Single tech stack	Each service can use different languages/ databases
Failure	One bug can crash the entire application	Failure isolated to one service (if designed correctly)
Communication	In-process function calls	Network calls (HTTP/REST, gRPC, message queues)
Testing	Simpler integration testing (everything in one process)	Complex — need contract testing, service virtualisation

SDET challenges with microservices: - **Test environment complexity:** Need to run multiple services for end-to-end testing. - **Contract testing:** Ensure services agree on API contracts (Pact, Spring Cloud Contract). - **Service virtualisation:** Mock unavailable or unstable services (WireMock, Mountebank). - **Distributed tracing:** Debug failures that span multiple services. - **Data consistency:** Eventual consistency between services needs specific test strategies.

---

10. DevOps for Quality Engineering

Q10.1: How does an SDET contribute to a DevOps team?

Area	SDET Contribution
CI/CD pipeline	Build and maintain automated test stages, define quality gates
Test automation	Write and maintain unit, integration, API, and E2E tests
Infrastructure testing	Test environment provisioning, IaC validation, disaster recovery testing
Monitoring	

Area	SDET Contribution
	Define test-related metrics, create dashboards, set up alerts for quality regressions
Security	Integrate security testing into the pipeline (SAST, DAST, dependency scanning)
Performance	Build and run performance tests, establish baselines, detect regressions
Incident response	Provide test data and analysis during incidents; write regression tests for post-mortems
Process improvement	Analyse test metrics, reduce flakiness, optimise test execution time

---

Q10.2: What is ChatOps and how does it relate to DevOps?

ChatOps integrates DevOps tools with team chat platforms (Slack, Teams) so that operations are visible and executable from chat.

Examples: - `/deploy staging` in Slack triggers a deployment pipeline. - Bot posts test results in a channel after each CI/CD run. - Alerts from monitoring tools appear in a dedicated channel. - `/rollback production v2.3.1` triggers a rollback.

Benefits: - **Visibility:** Everyone sees what's happening — deployments, test results, incidents. - **Auditability:** Chat history provides a log of who did what and when. - **Speed:** Execute operations without switching to a separate tool. - **Collaboration:** Incident response happens in a shared channel, not in silos.

Q10.3: What is GitOps?

GitOps uses Git as the single source of truth for both application code and infrastructure. All changes (code, configuration, infrastructure) go through Git pull requests.

Principles: 1. **Declarative:** The entire system is described declaratively (YAML/HCL files in Git). 2. **Versioned:** All changes are tracked in Git history. 3. **Automated:** Approved changes are automatically applied by an operator (ArgoCD, Flux). 4. **Self-healing:** If the actual state drifts from the desired state in Git, the operator corrects it automatically.

Workflow:

```
Developer pushes change → PR reviewed & approved → Merged to main →
GitOps operator detects change → Applies to cluster → Verifies convergence
```

Tools: ArgoCD (Kubernetes), Flux (Kubernetes), Jenkins X.

SDET relevance: In a GitOps workflow, test environment configurations are also in Git. Changes to test infrastructure go through the same PR process, ensuring review and traceability.

Q10.4: What is chaos engineering and how do you implement it?

Chaos engineering is the practice of intentionally injecting failures into a system to test its resilience.

Principles (from Netflix): 1. Define the system's "steady state" (normal behaviour in terms of metrics). 2. Hypothesise that steady state will continue during the experiment. 3. Introduce real-world failures (server crash, network latency, disk full). 4. Observe whether the hypothesis holds — does the system remain stable?

Types of failures to inject:

Failure	What to Test
Instance termination	Does auto-scaling replace it? Do requests failover?

Failure	What to Test
Network latency	Does the application handle slow responses gracefully (timeouts, circuit breakers)?
DNS failure	Does the application retry? Does it fail with a useful error?
Disk full	Does the application handle write failures without corruption?
Dependency unavailable	Does the circuit breaker open? Does the application degrade gracefully?
CPU/memory stress	Does the application remain responsive under resource pressure?

Tools:

Tool	Platform
AWS Fault Injection Simulator (FIS)	AWS-native chaos engineering
Chaos Monkey	Netflix OSS — randomly kills instances
Gremlin	Commercial chaos engineering platform
Litmus	Kubernetes-native chaos engineering
Toxiproxy	Simulates network conditions (latency, timeouts)

Q10.5: What is a service mesh and why should an SDET know about it?

A **service mesh** is a dedicated infrastructure layer that handles service-to-service communication in microservices architectures.

Popular implementations: Istio, Linkerd, Consul Connect.

What it provides:

Feature	Description	Testing Relevance
Traffic management	Routing, load balancing, traffic splitting	Test canary deployments, A/B routing
Security	Mutual TLS between services, access policies	Verify encryption in transit, test access controls
Observability	Metrics, logs, traces for every service call	Debug test failures across services
Resilience	Circuit breakers, retries, timeouts	Test fault tolerance behaviour
Rate limiting	Control request rates between services	Test throttling behaviour

SDET relevance: If LSEG uses a service mesh, the SDET can leverage its built-in observability for debugging, test traffic routing configurations, and verify security policies between services.

This guide covers DevOps topics from an SDET perspective — focusing on what you need to understand, discuss, and test rather than deep operational expertise. For the LSEG interview, prioritise Sections 1, 3, 4, and 7 — these are the most likely discussion areas given the DevOps team structure mentioned in the JD.